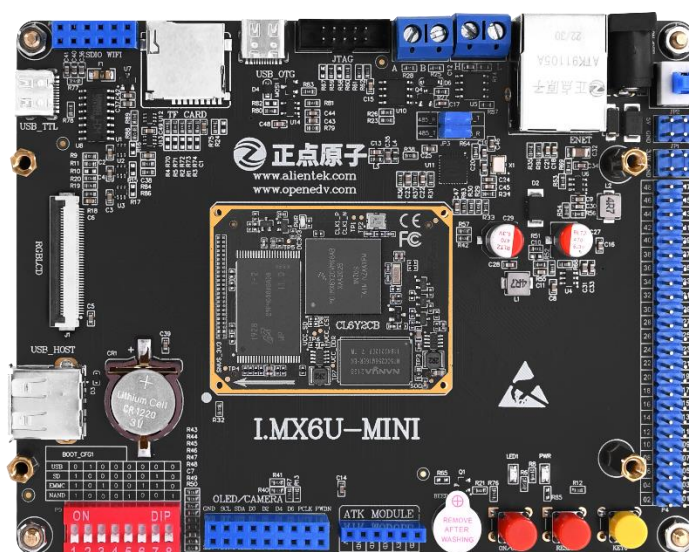
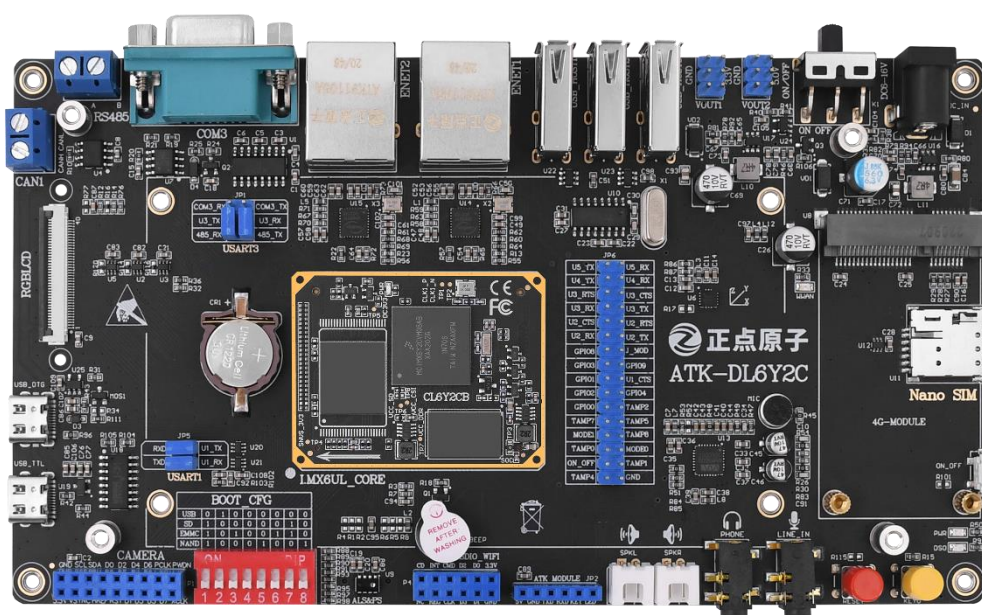


I.MX6U 嵌入式 Linux 驱动 开发指南 V1.81

-正点原子 I.MX6U ALPHA/Mini 开发板教程





正点原子公司名称 : 广州市星翼电子科技有限公司

原子哥在线教学平台 : www.yuanzige.com

开源电子网 / 论坛 : <http://www.openedv.com/forum.php>

正点原子淘宝店铺 : <https://openedv.taobao.com>

正点原子官方网站 : www.alientek.com

正点原子 B 站视频 : <https://space.bilibili.com/394620890>

电话: 020-38271790 传真: 020-36773971

请关注正点原子公众号, 资料发布更新我们会通知。

请下载原子哥 APP, 数千讲视频免费学习, 更快更流畅。



扫码关注正点原子公众号



扫码下载“原子哥”APP

文档更新说明

版本	版本更新说明	负责人	校审	发布日期
V1.0	初稿:	左忠凯	左忠凯	2019.10.9
V1.1	修改: 修改文档中的部分错误。 添加: 1、添加 4.9 MobaXterm 软件安装和使用 2、添加第 A3 章 Ubuntu-base 根文件系统构建 3、添加 B1 章 开发板 FTP 服务器移植与搭建	左忠凯	左忠凯	2019.12.1
V1.2	修改: 修改文档中的部分错误。 修改 A3.2.3.2 中 ubuntu arm 软件源 添加: 1、添加 71.2.5 ME3630 4G 模块 GNSS 定位测试 2、添加第六十四章 Linux 多点电容触摸屏实验	左忠凯	左忠凯	2020.1.15
V1.3	修改: 1、修改文档中的部分错误 2、修改 B1 章 开发板 FTP 服务器移植与搭建 3、修改 30.4.5 小节, 添加在 uboot 更新 EMMC 的 u-boot.imx。 4、修改 30.4.8 小节, 删除 uboot 中更新 NAND 版的 u-boot.imx, 并加以说明。 5、修改第五章, 添加对于 I.MX6U-Mini 开发板的支持, 调整了第五章的章节顺序。 6、修改 64.6 小节, 修改编译内核自带 FT5X06 驱动的配置路径。	左忠凯	左忠凯	2020.3.18

	7、修改 70.3.1 libopenssl 移植小节，本小节名字改为“70.3.1 openssl 移植”。修改了 openssl 移植的配置命令，非常重要！ 添加： 1、第六十五章 Linux 音频驱动实验 2、第 B2 章节 开发板 OpenSSH 移植与使用 3、第 B3 章节 嵌入式 GDB 调试搭建与使用 4、第 B4 章节 VSCode+gdbserver 图形化调试			
V1.4	修改： 1、修改文档中的部分错误 2、修改第七十章 Linux WIFI 驱动试验，添加对 RTL8188CUS WIFI 驱动的支持。 3、修改 37.4.3 小节中的 linux 内核网络驱动移植章节 4、修改 64.3.1 小节中触摸屏复位引脚 pinctrl 配置信息。 添加： 1、第六十六章 Linux CAN 驱动实验 2、第六十七章 Linux USB 驱动实验 3、第七十二章 RGB 转 HDMI 实验 4、第七十三章 Linux PWM 驱动实验			
V1.5	修改： 1、修改文档中的部分错误 2、修改 37.4.1 小节中关于 ALPHA 开发主频的描述。 添加： 1、第六十八章 Linux 块设备驱动实验 2、第七十章 Linux 网络驱动实验	左忠凯	左忠凯	2020.7.2
V1.5.1	修改：	左忠凯	左忠凯	2021.4.30

	1、第三十章 U-Boot 使用实验 中不合理的地方 2、修改大量客户反馈错误 3、修改了一些资料路径错误			
V1.5.2	修改: 1、修改一些错误内容 添加: 1、第 C1 章 ADC 实验	左忠凯	左忠凯	2021.6.21
V1.6	修改: 1、修改了第六十二章 Linux SPI 驱动实验, 使用 Linux 内核软件片选功能, 驱动中不再需要自己手动控制片选信号 添加: 1、第七十四章 Regmao API 实验 2、第七十五章 Linux IIO 驱动实验 3、第七十六章 Linux ADC 驱动实验	左忠凯	左忠凯	2021.10.20
V1.7	修改: 1、V2.4 新底板网络以及音频相关内容修改	左忠凯	左忠凯	2022.8.1
V1.71	删除: 1、68.5 修改 EMMC 驱动 修改: 1、修改 37.4.2 小节, EMMC 设备树要加入 “no-1-8-v” 选项, 因为核心板 EMMC 采用的 3.3V 电压, 因此不能使用 1.8V	左忠凯	左忠凯	2022.8.29
V1.8	添加: 1、添加了 V2.2 新版 Mini 底板描述 2、添加《64.8 7 寸触摸 GT911 驱动》	左忠凯	左忠凯	2022.12.09
V1.81	添加:	左忠凯	左忠凯	2023.1.11

	1、V2.2 版 Mini 底板的网络移植注释			
--	-------------------------	--	--	--

ALPHA/Mini 开发板教程适配表

正点原子目前有两款 I.MX6U 开发板: ALPHA 和 Mini, 本教程同时适用于这两款开发板。但是, 由于 Mini 开发板是 ALPHA 开发板的精简板, 因此相比 ALPHA 开发板会少一些外设, 所以本教程有些章节实验就无法在 Mini 板上完成, 关于本教程在两款开发板的适配情况请参考下表:

注意: ●表示适配, 否则表示不适配

章节名称	ALPHA 开发板	Mini 开发板	备注
第一章 Ubuntu 系统安装	●	●	
第二章 Ubuntu 系统入门	●	●	
第三章 Linux C 编程入门	●	●	
第四章 开发环境搭建	●	●	
第五章 I.MX6U-ALPHA/Mini 开发平台介绍	●	●	
第六章 Cortex-A7 MPCore 架构	●	●	
第七章 ARM 汇编基础	●	●	
第八章 汇编 LED 灯实验	●	●	
第九章 I.MX6U 启动方式详解	●	●	
第十章 C 语言版 LED 灯实验	●	●	
第十一章 模仿 STM32 驱动开发格式实验	●	●	
第十二章 官方 SDK 移植实验	●	●	
第十三章 BSP 工程管理实验	●	●	
第十四章 蜂鸣器实验	●	●	
第十五章 按键输入实验	●	●	
第十六章 主频和时钟配置实验	●	●	
第十七章 GPIO 中断实验	●	●	
第十八章 EPIT 定时器实验	●	●	
第十九章 定时器按键消抖实验	●	●	
第二十章 高精度延时实验	●	●	
第二十一章 UART 串口通信实验	●	●	
第二十二章 串口格式化函数移植实验	●	●	
第二十三章 DDR3 实验	●	●	
第二十四章 RGBLCD 显示实验	●	●	
第二十五章 RTC 实时时钟实验	●	●	
第二十六章 I2C 实验	●		
第二十七章 SPI 实验	●		
第二十八章 多点电容触摸屏实验	●	●	
第二十九章 LCD 背光调节实验	●	●	
第三十章 U-Boot 使用实验	●	●	
第三十一章 U-Boot 顶层 Makefile 详解	●	●	
第三十二章 U-Boot 启动流程详解	●	●	
第三十三章 U-Boot 移植	●	●	

第三十四章 U-Boot 图形化配置及其原理	●	●	
第三十五章 Linux 内核顶层 Makefile 详解	●	●	
第三十六章 Linux 内核启动流程	●	●	
第三十七章 Linux 内核移植	●	●	
第三十八章 根文件系统构建	●	●	
第三十九章 系统烧写	●	●	
第四十章 字符设备驱动开发	●	●	
第四十一章 嵌入式 Linux LED 驱动开发实验	●	●	
第四十二章 新字符设备驱动实验	●	●	
第四十三章 Linux 设备树	●	●	
第四十四章 设备树下的 LED 驱动实验	●	●	
第四十五章 pinctrl 和 gpio 子系统实验	●	●	
第四十六章 Linux 蜂鸣器实验	●	●	
第四十七章 Linux 并发与竞争	●	●	
第四十八章 Linux 并发与竞争实验	●	●	
第四十九章 Linux 按键输入实验	●	●	
第五十章 Linux 内核定时器实验	●	●	
第五十一章 Linux 中断实验	●	●	
第五十二章 Linux 阻塞和非阻塞 IO 实验	●	●	
第五十三章 异步通知实验	●	●	
第五十四章 platform 设备驱动实验	●	●	
第五十五章 设备树下的 platform 驱动编写	●	●	
第五十六章 Linux 自带的 LED 灯驱动实验	●	●	
第五十七章 Linux MISC 驱动实验	●	●	
第五十八章 Linux INPUT 子系统实验	●	●	
第五十九章 Linux LCD 驱动实验	●	●	
第六十章 Linux RTC 驱动实验	●	●	
第六十一章 Linux I2C 驱动实验	●		
第六十二章 Linux SPI 驱动实验	●		
第六十三章 Linux RS232/485/GPS 驱动实验	●	●	Mini 板只能完成 RS485 和 GPS
第六十四章 Linux 多点电容触摸屏实验	●	●	
第六十五章 Linux 音频驱动实验	●		
第六十六章 Linux CAN 驱动实验	●	●	
第六十七章 Linux USB 驱动实验	●	●	
第六十八章 Linux 块设备驱动实验	●	●	
第六十九章 Linux 网络驱动实验	●	●	
第七十章 Linux WIFI 驱动实验	●	●	
第七十一章 Linux 4G 通信实验	●		
第七十二章 RGB 转 HDMI 实验	●	●	
第七十三章 Linux PWM 驱动实验	●	●	
第七十四章 Regmap API 实验	●		

第七十五章 Linux IIO 驱动实验	●		
第七十六章 Linux ADC 驱动实验	●	●	
第 A1 章 Buildroot 根文件系统构建	●	●	
第 A2 章 Yocto 根文件系统构建	●	●	
第 A3 章 Ubuntu-base 根文件系统构建	●	●	
第 B1 章 开发板 FTP 服务器移植与搭建	●	●	
第 B2 章节 开发板 OpenSSH 移植与使用	●	●	
第 B3 章节 嵌入式 GDB 调试搭建与使用	●	●	
第 B4 章节 VSCode+gdbserver 图形化调试	●	●	

目录

ALPHA/Mini 开发板教程适配表	7
前言	45
第一篇 Ubuntu 系统入门篇	47
第一章 Ubuntu 系统安装	48
1.1 安装虚拟机软件 VMware	49
1.2 创建虚拟机	55
1.3 安装 Ubuntu 操作系统	66
1.3.1 获取 Ubuntu 系统	66
1.3.2 安装 Ubuntu 操作系统	67
1.3.3 弹出系统镜像	79
第二章 Ubuntu 系统入门	81
2.1 Ubuntu 系统初体验	82
2.1.1 Hello Ubuntu	82
2.1.2 系统设置	84
2.1.3 系统注销与关机	87
2.1.4 中文输入测试	87
2.2 Ubuntu 终端操作	90
2.3 Shell 操作	91
2.3.1 Shell 简介	91
2.3.2 Shell 基本操作	91
2.2.4 常用 Shell 命令	93
2.4 APT 下载工具	100
2.5 Ubuntu 下文本编辑	103
2.5.1 Gedit 编辑器	103
2.5.2 VI/VIM 编辑器	104
2.6 Linux 文件系统	109
2.6.1 Linux 文件系统简介以及类型	109
2.6.2 Linux 文件系统结构	111
2.6.2 文件操作命令	114
2.6.3 文件压缩和解压缩	118
2.6.4 文件查询和搜索	124

2.6.5 文件类型.....	125
2.7 Linux 用户权限管理.....	126
2.7.1 Ubuntu 用户系统.....	126
2.7.2 权限管理.....	126
2.7.3 权限管理命令.....	129
2.8 Linux 磁盘管理.....	131
2.8.1 Linux 磁盘管理基本概念.....	131
2.8.2 磁盘管理命令.....	132
第三章 Linux C 编程入门.....	136
3.1 Hello World!	137
3.1.1 编写代码.....	137
3.1.2 编译代码.....	138
3.2 GCC 编译器.....	140
3.2.1 gcc 命令.....	140
3.2.2 编译错误警告.....	140
3.2.3 编译流程.....	141
3.3 Makefile 基础.....	141
3.3.1 何为 Makefile	141
3.3.2 Makefile 的引入.....	142
3.4 Makefile 语法.....	145
3.4.1 Makefile 规则格式.....	145
3.4.2 Makefile 变量.....	147
3.4.3 Makefile 模式规则.....	149
3.4.4 Makefile 自动化变量.....	150
3.4.5 Makefile 伪目标.....	151
3.4.6 Makefile 条件判断.....	151
3.4.7 Makefile 函数使用.....	152
第二篇 裸机开发篇.....	155
第四章 开发环境搭建.....	156
4.1 Ubuntu 和 Windows 文件互传.....	157
4.2 Ubuntu 下 NFS 和 SSH 服务开启	163
4.2.1 NFS 服务开启	163
4.2.2 SSH 服务开启	164

4.3 Ubuntu 交叉编译工具链安装.....	164
4.3.1 交叉编译器安装.....	164
4.3.2 安装相关库.....	168
4.3.3 交叉编译器验证.....	168
4.4 Source Insight 软件安装和使用.....	170
4.4.1 Source Insight 安装.....	170
4.4.2 Source Insight 新建工程.....	176
4.4.3 Source Insight 解决中文乱码.....	184
4.5 Visual Studio Code 软件的安装和使用.....	186
4.5.1 Visual Studio Code 的安装.....	186
4.5.2 Visual Studio Code 插件的安装.....	190
4.5.3 Visual Studio Code 新建工程.....	193
4.6 CH340 串口驱动安装.....	199
4.7 SecureCRT 软件安装和使用.....	202
4.7.1 SecureCRT 安装.....	202
4.7.2 SecureCRT 使用.....	207
4.8 Putty 软件的安装和使用.....	212
4.8.1 Putty 软件安装.....	212
4.8.2 Putty 软件使用.....	215
4.9 MobaXterm 软件安装和使用.....	218
4.9.1 MobaXterm 软件安装.....	218
4.9.2 MobaXterm 软件使用.....	219
第五章 I.MX6U-ALPHA/Mini 开发平台介绍.....	223
5.1 正点原子 I.MX6U-ALPHA/Mini 开发板资源初探.....	224
5.1.1 I.MX6U-ALPHA 开发板底板资源.....	224
5.1.2 I.MX6U-Mini 开发板底板资源.....	226
5.1.3 I.MX6U 核心板资源.....	229
5.2 正点原子 I.MX6U-ALPHA/Mini 开发板资源说明.....	230
5.2.1 I.MX6U-ALPHA 硬件资源说明.....	230
5.2.2 I.MX6U-ALPHA 软件资源说明.....	234
5.2.3 I.MX6U-Mini 硬件资源说明.....	236
5.2.4 I.MX6U-Mini 软件资源说明.....	239
5.2.5 I.MX6ULL 核心板硬件资源说明.....	239

5.3 I.MX6U-ALPHA 开发板底板原理图详解	240
5.3.1 核心板接口	240
5.3.2 引出 IO 口	242
5.3.3 USB 串口/串口 1 选择接口	242
5.3.4 RGB LCD 模块接口	243
5.3.5 复位电路	244
5.3.6 启动模式设置接口	244
5.3.7 VBAT 供电接口	245
5.3.8 RS232 串口	245
5.3.9 RS485 接口	246
5.3.10 CAN 接口	246
5.3.11 USB HUB 接口	247
5.3.12 USB OTG 接口	248
5.3.13 光环境传感器	249
5.3.14 六轴传感器	250
5.3.15 LED	250
5.3.16 按键	251
5.3.17 摄像头模块接口	251
5.3.18 有源蜂鸣器	251
5.3.19 TF 卡接口	252
5.3.20 SDIO WIFI 接口	252
5.3.21 4G 模块接口	253
5.3.22 ATK 模块接口	254
5.3.23 以太网接口 (RJ45)	255
5.3.24 SAI 音频编解码器	259
5.3.25 电源	261
5.3.26 电源输入输出接口	261
5.3.27 USB 串口	262
5.4 I.MX6U-Mini 开发板底板原理图详解	263
5.4.1 核心板接口	263
5.4.2 引出 IO 口	265
5.4.3 USB 串口	265
5.4.4 RGB LCD 模块接口	267
5.4.5 复位电路	268

5.4.6 启动模式设置接口	268
5.4.7 VBAT 供电接口.....	268
5.4.8 RS485 接口.....	269
5.4.9 CAN 接口	269
5.4.10 USB HOST 接口.....	270
5.4.11 USB OTG 接口	270
5.4.12 LED	271
5.4.13 按键	272
5.4.14 摄像头模块接口	272
5.4.15 有源蜂鸣器.....	273
5.4.16 TF 卡接口	273
5.4.17 SDIO WIFI 接口	274
5.4.18 ATK 模块接口	274
5.4.19 以太网接口 (RJ45)	274
5.4.20 电源	275
5.4.21 电源输入输出接口	276
5.5 I.MX6U 核心板原理图详解.....	276
5.5.1 SOC	276
5.5.2 BTB 接口.....	280
5.5.3 NAND FLASH.....	282
5.5.4 EMMC	282
5.5.5 DDR3L	283
5.5.6 核心板电源.....	284
5.6 开发板使用注意事项	287
第六章 Cortex-A7 MPCore 架构.....	288
6.1 Cortex-A7 MPCore 简介.....	289
6.2 Cortex-A 处理器运行模型	290
6.3 Cortex-A 寄存器组.....	290
6.3.1 通用寄存器.....	292
6.3.2 程序状态寄存器	293
第七章 ARM 汇编基础.....	295
7.1 GNU 汇编语法	296
7.2 Cortex-A7 常用汇编指令	298

7.2.1 处理器内部数据传输指令	298
7.2.2 存储器访问指令	298
7.2.3 压栈和出栈指令	299
7.2.4 跳转指令	301
7.2.5 算术运算指令	302
7.2.6 逻辑运算指令	302
第八章 汇编 LED 灯实验	304
8.1 I.MX6U GPIO 详解	305
8.1.1 STM32 GPIO 回顾	305
8.1.2 I.MX6U IO 命名	305
8.1.3 I.MX6U IO 复用	307
8.1.4 I.MX6U IO 配置	308
8.1.5 I.MX6U GPIO 配置	311
8.1.6 I.MX6U GPIO 时钟使能	315
8.2 硬件原理分析	317
8.3 实验程序编写	317
8.4 编译下载验证	322
8.4.1 编译代码	322
8.4.2 创建 Makefile 文件	326
8.4.3 代码烧写	327
8.4.4 代码验证	331
第九章 I.MX6U 启动方式详解	332
9.1 启动方式选择	333
9.1.1 串行下载	333
9.1.2 内部 BOOT 模式	334
9.2 BOOT ROM 初始化内容	334
9.3 启动设备	334
9.4 镜像烧写	338
9.4.1 IVT 和 Boot Data 数据	339
9.4.2 DCD 数据	341
第十章 C 语言版 LED 灯实验	344
10.1 C 语言版 LED 灯简介	345
10.2 硬件原理分析	345

10.3 实验程序编写	345
10.3.1 汇编部分实验程序编写	345
10.3.2 C 语言部分实验程序编写	347
10.4 编译下载验证	352
10.4.1 编写 Makefile	352
10.4.2 链接脚本	353
10.4.3 修改 Makefile	355
10.4.4 下载验证	355
第十一章 模仿 STM32 驱动开发格式实验	356
11.1 模仿 STM32 寄存器定义	357
11.1.1 STM32 寄存器定义简介	357
11.1.2 I.MX6U 寄存器定义	358
11.2 硬件原理分析	359
11.3 实验程序编写	359
11.4 编译下载验证	364
11.4.1 编写 Makefile 和链接脚本	364
11.4.2 编译下载	365
第十二章 官方 SDK 移植实验	366
12.1 I.MX6ULL 官方 SDK 包简介	367
12.2 硬件原理图分析	368
12.3 试验程序编写	368
12.3.1 SDK 文件移植	368
12.3.2 创建 cc.h 文件	368
12.3.3 编写实验代码	369
12.4 编译下载验证	375
12.4.1 编写 Makefile 和链接脚本	375
12.4.2 编译下载	376
第十三章 BSP 工程管理实验	377
13.1 工程管理简介	378
13.2 硬件原理分析	378
13.3 实验程序编写	379
13.3.1 创建 imx6ul.h 文件	379
13.3.2 编写 led 驱动代码	379

13.3.3 编写时钟驱动代码	381
13.3.4 编写延时驱动代码	382
13.3.5 修改 main.c 文件	384
13.4 编译下载验证	385
13.4.1 编写 Makefile 和链接脚本	385
13.4.2 编译下载	388
第十四章 蜂鸣器实验	389
14.2 有源蜂鸣器简介	390
14.3 硬件原理分析	390
14.3 试验程序编写	391
14.4 编译下载验证	393
14.4.1 编写 Makefile 和链接脚本	393
14.4.2 编译下载	394
第十五章 按键输入实验	395
15.1 按键输入简介	396
15.2 硬件原理分析	396
15.3 实验程序编写	396
15.4 编译下载验证	404
15.4.1 编写 Makefile 和链接脚本	404
15.4.2 编译下载	405
第十六章 主频和时钟配置实验	406
16.1 I.MX6U 时钟系统详解	407
16.1.1 系统时钟来源	407
16.1.2 7 路 PLL 时钟源	407
16.1.3 时钟树简介	409
16.1.4 内核时钟设置	411
16.1.5 PFD 时钟设置	415
16.1.6 AHB、IPG 和 PERCLK 根时钟设置	417
16.2 硬件原理分析	421
16.3 实验程序编写	421
16.4 编译下载验证	424
16.4.1 编写 Makefile 和链接脚本	424
16.4.2 编译下载	425

第十七章 GPIO 中断实验.....	426
17.1 Cortex-A7 中断系统详解	427
17.1.1 STM32 中断系统回顾	427
17.1.2 Cortex-A7 中断系统简介	429
17.1.3 GIC 控制器简介	432
17.1.4 CP15 协处理器	438
17.1.5 中断使能.....	442
17.1.6 中断优先级设置	442
17.2 硬件原理分析.....	444
17.3 试验程序编写.....	444
17.3.1 移植 SDK 包中断相关文件	444
17.3.2 重新编写 start.S 文件	444
17.3.3 通用中断驱动文件编写.....	449
17.3.4 修改 GPIO 驱动文件	453
17.3.5 按键中断驱动文件编写.....	458
17.3.6 编写 main.c 文件	461
17.4 编译下载验证.....	462
17.4.1 编写 Makefile 和链接脚本	462
17.4.2 编译下载.....	463
第十八章 EPIT 定时器实验.....	464
18.1 EPIT 定时器简介.....	465
18.2 硬件原理分析.....	468
18.3 实验程序编写.....	468
18.4 编译下载验证.....	471
18.4.1 编写 Makefile 和链接脚本	471
18.4.2 编译下载.....	472
第十九章 定时器按键消抖实验	473
19.1 定时器按键消抖简介	474
19.2 硬件原理分析.....	475
19.3 试验程序编写.....	475
19.4 编译下载验证.....	480
19.4.1 编写 Makefile 和链接脚本	480
19.4.2 编译下载.....	481

第二十章 高精度延时实验	482
20.1 高精度延时简介	483
20.1.1 GPT 定时器简介	483
20.1.2 定时器实现高精度延时原理	486
20.2 硬件原理分析	487
20.3 实验程序编写	487
20.4 编译下载验证	493
20.4.1 编写 Makefile 和链接脚本	493
20.4.2 编译下载	493
第二十一章 UART 串 口 通 信 实 验	496
21.1 I.MX6U 串口简介	497
21.1.1 UART 简介	497
21.1.2 I.MX6U UART 简介	499
21.2 硬件原理分析	502
21.3 实验程序编写	504
21.4 编译下载验证	513
21.4.1 编写 Makefile 和链接脚本	513
21.4.2 编译下载	515
第二十二章 串口格式化函数移植实验	519
22.1 串口格式化函数简介	520
22.2 硬件原理分析	520
22.3 实验程序编写	520
22.4 编译下载验证	522
22.4.1 编写 Makefile 和链接脚本	522
22.4.2 编译下载	523
第二十三章 DDR3 实验	525
23.1 DDR3 内存简介	526
23.1.1 何为 RAM 和 ROM?	526
23.1.2 SRAM 简介	526
23.1.3 SDRAM 简介	528
23.1.4 DDR 简介	531
23.2 DDR3 关键时间参数	533

23.3 I.MX6U MMDC 控制器简介	536
23.3.1 MMDC 控制器	536
23.3.2 MMDC 控制器信号引脚	536
23.3.3 MMDC 控制器时钟源	537
23.4 ALPHA 开发板 DDR3L 原理图	537
23.5 DDR3L 初始化与测试	538
23.5.1 ddr_stress_tester 简介	538
23.5.2 DDR3L 驱动配置	539
23.5.3 DDR3L 校准	545
23.5.4 DDR3L 超频测试	549
23.5.5 DDR3L 驱动总结	550
第二十四章 RGBLCD 显示实验	553
24.1 LCD 和 eLCDIF 简介	554
24.1.1 LCD 简介	554
24.1.2 eLCDIF 接口	562
24.2 硬件原理分析	567
24.3 实验程序编写	569
24.4 编译下载验证	590
24.4.1 编写 Makefile 和链接脚本	590
24.4.2 编译下载	591
第二十五章 RTC 实时时钟实验	592
25.1 I.MX6U RTC 简介	593
25.2 硬件原理分析	595
25.3 实验程序编写	595
25.3.1 修改文件 MCIMX6Y2.h	596
25.3.2 编写实验程序	596
25.4 编译下载验证	606
25.4.1 编写 Makefile 和链接脚本	606
25.4.2 编译下载	607
第二十六章 I2C 实验	609
26.1 I2C & AP3216C 简介	610
26.1.1 I2C 简介	610
26.1.2 I.MX6U I2C 简介	612

26.1.3 AP3216C 简介	615
26.2 硬件原理分析	616
26.3 实验程序编写	617
26.4 编译下载验证	632
26.4.1 编写 Makefile 和链接脚本	632
26.4.2 编译下载	633
第二十七章 SPI 实验	635
27.1 SPI & ICM-20608 简介	636
27.1.1 SPI 简介	636
27.1.2 I.MX6U ECSPI 简介	637
27.1.3 ICM-20608 简介	641
27.2 硬件原理分析	644
27.3 实验程序编写	644
27.4 编译下载验证	659
27.4.1 编写 Makefile 和链接脚本	659
27.4.2 编译下载	661
第二十八章 多点电容触摸屏实验	662
28.1 多点电容触摸简介	663
28.2 硬件原理分析	664
28.3 实验程序编写	665
28.4 编译下载验证	675
28.4.1 编写 Makefile 和链接脚本	675
28.4.2 编译下载	676
第二十九章 LCD 背光调节实验	679
29.1 LCD 背光调节简介	680
29.2 硬件原理分析	685
29.3 实验程序编写	685
29.4 编译下载验证	691
29.4.1 编写 Makefile 和链接脚本	691
29.4.2 编译下载	693
第三篇 系统移植篇	695
第三十章 U-Boot 使用实验	696
30.1 U-Boot 简介	697

30.2 U-Boot 初次编译	700
30.3 U-Boot 烧写与启动	702
30.4 U-Boot 命令使用	704
30.4.1 信息查询命令	705
30.4.2 环境变量操作命令	707
30.4.3 内存操作命令	708
30.4.4 网络操作命令	712
30.4.5 EMMC 和 SD 卡操作命令	718
30.4.6 FAT 格式文件系统操作命令	724
30.4.7 EXT 格式文件系统操作命令	727
30.4.8 NAND 操作命令	728
30.4.9 BOOT 操作命令	732
30.4.10 其他常用命令	737
第三十一章 U-Boot 顶层 Makefile 详解	740
31.1 U-Boot 工程目录分析	741
31.2 VScode 工程创建	751
31.3 U-Boot 顶层 Makefile 分析	757
31.3.1 版本号	757
31.3.2 MAKEFLAGS 变量	758
31.3.3 命令输出	758
31.3.4 静默输出	760
31.3.5 设置编译结果输出目录	762
31.3.6 代码检查	763
31.3.7 模块编译	764
31.3.8 获取主机架构和系统	765
31.3.9 设置目标架构、交叉编译器和配置文件	766
31.3.10 调用 scripts/Kbuild.include	767
31.3.11 交叉编译工具变量设置	767
31.3.12 导出其他变量	768
31.3.13 make xxx_defconfig 过程	772
31.3.14 Makefile.build 脚本分析	777
31.3.15 make 过程	780
第三十二章 U-Boot 启动流程详解	789

32.1 链接脚本 u-boot.lds 详解	790
32.2 U-Boot 启动流程详解	794
32.2.1 reset 函数源码详解	794
32.2.2 lowlevel_init 函数详解	799
32.2.3 s_init 函数详解	802
32.2.4 _main 函数详解	804
32.2.5 board_init_f 函数详解	811
32.2.6 relocate_code 函数详解	819
32.2.7 relocate_vectors 函数详解	825
32.2.8 board_init_r 函数详解	826
32.2.9 run_main_loop 函数详解	830
32.2.10 cli_loop 函数详解	836
32.2.11 cmd_process 函数详解	838
32.3 bootz 启动 Linux 内核过程	844
32.3.1 images 全局变量	844
32.3.2 do_bootz 函数	845
32.3.3 bootz_start 函数	846
32.3.4 do_bootm_states 函数	849
32.3.5 bootm_os_get_boot_func 函数	854
32.3.6 do_bootm_linux 函数	855
第三十三章 U-Boot 移植	860
33.1 NXP 官方开发板 uboot 编译测试	861
33.1.1 查找 NXP 官方的开发板默认配置文件	861
33.1.2 编译 NXP 官方开发板对应的 uboot	862
33.1.3 烧写验证与驱动测试	864
33.2 在 U-Boot 中添加自己的开发板	866
33.2.1 添加开发板默认配置文件	867
33.2.2 添加开发板对应的头文件	867
33.2.3 添加开发板对应的板级文件夹	875
33.2.4 修改 U-Boot 图形界面配置文件	877
33.2.5 使用新添加的板子配置编译 uboot	877
33.2.6 LCD 驱动修改	879
33.2.7 V2.4 版本以前底板网络驱动修改	882

33.2.8 V2.4 及以后版本底板网络驱动修改	892
33.2.9 其他需要修改的地方	901
33.3 bootcmd 和 bootargs 环境变量	902
33.3.1 环境变量 bootcmd	903
33.3.2 环境变量 bootargs	908
33.4 uboot 启动 Linux 测试.....	909
33.4.1 从 EMMC 启动 Linux 系统	909
33.4.2 从网络启动 Linux 系统	910
第三十四章 U-Boot 图形化配置及其原理	912
34.1 U-Boot 图形化配置体验	913
34.2 menuconfig 图形化配置原理.....	917
34.2.1 make menuconfig 过程分析	917
34.2.2 Kconfig 语法简介	918
34.3 添加自定义菜单	927
第三十五章 Linux 内核顶层 Makefile 详解	930
35.1 Linux 内核获取	931
35.2 Linux 内核初次编译.....	931
35.3 Linux 工程目录分析.....	933
35.4 VSCode 工程创建.....	939
35.5 顶层 Makefile 详解.....	941
35.5.1 make xxx_defconfig 过程.....	946
35.5.2 Makefile.build 脚本分析	948
35.5.3 make 过程	950
35.5.4 built-in.o 文件编译生成过程	955
35.5.5 make zImage 过程.....	958
第三十六章 Linux 内核启动流程	960
36.1 链接脚本 vmlinux.lds	961
36.2 Linux 内核启动流程分析	961
36.2.1 Linux 内核入口 stext	961
36.2.2 __mmap_switched 函数	964
36.2.3 start_kernel 函数	964
36.2.4 rest_init 函数.....	968
36.2.5 init 进程	970

第三十七章 Linux 内核移植.....	974
37.1 创建 VSCode 工程.....	975
37.2 NXP 官方开发板 Linux 内核编译	975
37.2.1 修改顶层 Makefile.....	975
37.2.2 配置并编译 Linux 内核	975
37.2.3 Linux 内核启动测试.....	976
37.2.4 根文件系统缺失错误	977
37.3 在 Linux 中添加自己的开发板	978
37.3.1 添加开发板默认配置文件.....	978
37.3.2 添加开发板对应的设备树文件	979
37.3.3 编译测试.....	980
37.4 CPU 主频和网络驱动修改.....	981
37.4.1 CPU 主频修改	981
37.4.2 修改 EMMC 驱动	987
37.4.3 V2.4 版本以前底板网络驱动修改	989
37.4.4 V2.4 及以后版本底板网络驱动修改	1000
37.4.5 保存修改后的图形化配置文件	1007
第三十八章 根文件系统构建	1010
38.1 根文件系统简介	1011
38.2 BusyBox 构建根文件系统.....	1013
38.2.1 BusyBox 简介	1013
38.2.2 编译 BusyBox 构建根文件系统	1014
38.2.3 向根文件系统添加 lib 库	1022
38.2.4 创建其他文件夹	1024
38.3 根文件系统初步测试	1024
38.4 完善根文件系统	1026
38.4.1 创建/etc/init.d/rcS 文件.....	1026
38.4.2 创建/etc/fstab 文件.....	1027
38.4.3 创建/etc/inittab 文件	1028
38.5 根文件系统其他功能测试.....	1029
38.5.1 软件运行测试.....	1029
38.5.2 中文字符测试.....	1031
38.5.3 开机自启动测试.....	1032

38.5.4 外网连接测试.....	1033
第三十九章 系统烧写.....	1035
39.1 MfgTool 工具简介.....	1036
39.2 MfgTool 工作原理简介	1037
39.2.1 烧写方式.....	1037
39.2.2 系统烧写原理.....	1039
39.3 烧写 NXP 官方系统	1043
39.4 烧写自制的系统.....	1044
39.4.1 系统烧写.....	1044
39.4.2 网络开机自启动设置	1045
39.5 改造我们自己的烧写工具.....	1047
39.5.1 改造 MfgTool.....	1047
39.5.2 烧写测试.....	1050
39.5.3 解决 Linux 内核启动失败	1051
第四篇 ARM Linux 驱动开发篇.....	1054
第四十章 字符设备驱动开发	1055
40.1 字符设备驱动简介	1056
40.2 字符设备驱动开发步骤	1058
40.2.1 驱动模块的加载和卸载.....	1059
40.2.2 字符设备注册与注销	1060
40.2.3 实现设备的具体操作函数.....	1062
40.2.4 添加 LICENSE 和作者信息	1064
40.3 Linux 设备号	1064
40.3.1 设备号的组成.....	1064
40.3.2 设备号的分配.....	1065
40.4 chrdevbase 字符设备驱动开发实验	1066
40.4.1 实验程序编写.....	1066
40.4.2 编写测试 APP.....	1072
40.4.3 编译驱动程序和测试 APP	1076
40.4.4 运行测试.....	1078
第四十一章 嵌入式 Linux LED 驱动开发实验	1082
41.1 Linux 下 LED 灯驱动原理	1083
41.1.1 地址映射.....	1083

41.1.2 I/O 内存访问函数.....	1085
41.2 硬件原理图分析.....	1085
41.3 实验程序编写.....	1085
41.3.1 LED 灯驱动程序编写.....	1085
41.3.2 编写测试 APP.....	1091
41.4 运行测试.....	1093
41.4.1 编译驱动程序和测试 APP	1093
41.4.2 运行测试.....	1093
第四十二章 新字符设备驱动实验.....	1095
42.1 新字符设备驱动原理	1096
42.1.1 分配和释放设备号	1096
42.1.2 新的字符设备注册方法.....	1097
42.2 自动创建设备节点	1098
42.2.1 mdev 机制.....	1098
42.2.1 创建和删除类.....	1099
42.2.2 创建设备.....	1099
42.2.3 参考示例.....	1100
42.3 设置文件私有数据	1100
42.4 硬件原理图分析.....	1101
42.5 实验程序编写.....	1101
42.5.1 LED 灯驱动程序编写.....	1101
42.5.2 编写测试 APP.....	1108
42.6 运行测试.....	1108
42.6.1 编译驱动程序和测试 APP	1108
42.6.2 运行测试.....	1108
第四十三章 Linux 设备树	1110
43.1 什么是设备树?	1111
43.2 DTS、DTB 和 DTC.....	1112
43.3 DTS 语法.....	1114
43.3.1 .dtsi 头文件.....	1114
43.3.2 设备节点.....	1117
43.3.3 标准属性.....	1118
43.3.4 根节点 compatible 属性.....	1122

43.3.5 向节点追加或修改内容.....	1128
43.4 创建小型模板设备树	1130
43.5 设备树在系统中的体现	1136
43.6 特殊节点.....	1137
43.6.1 aliases 子节点	1137
43.6.2 chosen 子节点	1138
43.7 Linux 内核解析 DTB 文件	1140
43.8 绑定信息文档.....	1141
43.9 设备树常用 OF 操作函数	1142
43.9.1 查找节点的 OF 函数	1143
43.9.2 查找父/子节点的 OF 函数	1144
43.9.3 提取属性值的 OF 函数	1145
43.9.4 其他常用的 OF 函数	1148
第四十四章 设备树下的 LED 驱动实验	1151
44.1 设备树 LED 驱动原理.....	1152
44.2 硬件原理图分析	1152
44.3 实验程序编写	1152
44.3.1 修改设备树文件	1152
44.3.2 LED 灯驱动程序编写.....	1153
44.3.3 编写测试 APP.....	1161
44.4 运行测试.....	1161
44.4.1 编译驱动程序和测试 APP	1161
44.4.2 运行测试.....	1161
第四十五章 pinctrl 和 gpio 子系统实验	1163
45.1 pinctrl 子系统	1164
45.1.1 pinctrl 子系统简介	1164
45.1.2 I.MX6ULL 的 pinctrl 子系统驱动	1164
45.1.3 设备树中添加 pinctrl 节点模板.....	1173
45.2 gpio 子系统.....	1174
45.2.1 gpio 子系统简介	1174
45.2.2 I.MX6ULL 的 gpio 子系统驱动	1174
45.2.3 gpio 子系统 API 函数.....	1182
45.2.4 设备树中添加 gpio 节点模板.....	1183

45.2.5 与 gpio 相关的 OF 函数	1184
45.3 硬件原理图分析	1185
45.4 实验程序编写	1185
45.4.1 修改设备树文件	1185
45.4.2 LED 灯驱动程序编写	1187
45.4.3 编写测试 APP	1192
45.5 运行测试	1193
45.5.1 编译驱动程序和测试 APP	1193
45.5.2 运行测试	1193
第四十六章 Linux 蜂鸣器实验	1195
46.1 蜂鸣器驱动原理	1196
46.2 硬件原理图分析	1196
46.3 实验程序编写	1196
46.3.1 修改设备树文件	1196
46.3.2 蜂鸣器驱动程序编写	1197
46.3.3 编写测试 APP	1202
46.4 运行测试	1204
46.4.1 编译驱动程序和测试 APP	1204
46.4.2 运行测试	1204
第四十七章 Linux 并发与竞争	1206
47.1 并发与竞争	1207
47.2 原子操作	1208
47.2.1 原子操作简介	1208
47.2.2 原子整形操作 API 函数	1209
47.2.3 原子位操作 API 函数	1210
47.3 自旋锁	1211
47.3.1 自旋锁简介	1211
47.3.2 自旋锁 API 函数	1212
47.3.3 其他类型的锁	1214
47.3.4 自旋锁使用注意事项	1215
47.4 信号量	1216
47.4.1 信号量简介	1216
47.4.2 信号量 API 函数	1217

47.5 互斥体	1217
47.5.1 互斥体简介	1217
47.5.2 互斥体 API 函数	1218
第四十八章 Linux 并发与竞争实验	1219
48.1 原子操作实验	1220
48.1.1 实验程序编写	1220
48.1.2 运行测试	1227
48.2 自旋锁实验	1229
48.2.1 实验程序编写	1229
48.2.2 运行测试	1233
48.3 信号量实验	1234
48.3.1 实验程序编写	1234
48.3.2 运行测试	1237
48.4 互斥体实验	1238
48.4.1 实验程序编写	1238
48.4.2 运行测试	1241
第四十九章 Linux 按键输入实验	1243
49.1 Linux 下按键驱动原理	1244
49.2 硬件原理图分析	1244
49.3 实验程序编写	1244
49.3.1 修改设备树文件	1244
49.3.2 按键驱动程序编写	1245
49.3.3 编写测试 APP	1250
49.4 运行测试	1252
49.4.1 编译驱动程序和测试 APP	1252
49.4.2 运行测试	1252
第五十章 Linux 内核定时器实验	1254
50.1 Linux 时间管理和内核定时器简介	1255
50.1.1 内核时间管理简介	1255
50.1.2 内核定时器简介	1258
50.1.3 Linux 内核短延时函数	1260
50.2 硬件原理图分析	1260
50.3 实验程序编写	1261

50.3.1 修改设备树文件	1261
50.3.2 定时器驱动程序编写	1261
50.3.3 编写测试 APP.....	1267
50.4 运行测试.....	1269
50.4.1 编译驱动程序和测试 APP	1269
50.4.2 运行测试.....	1270
第五十一章 Linux 中断实验.....	1271
51.1 Linux 中断简介	1272
51.1.1 Linux 中断 API 函数	1272
51.1.2 上半部与下半部	1274
51.1.3 设备树中断信息节点	1280
51.1.4 获取中断号.....	1282
51.2 硬件原理图分析.....	1282
51.3 实验程序编写.....	1282
51.3.1 修改设备树文件	1282
51.3.2 按键中断驱动程序编写.....	1283
51.3.3 编写测试 APP.....	1291
51.4 运行测试.....	1293
51.4.1 编译驱动程序和测试 APP	1293
51.4.2 运行测试.....	1293
第五十二章 Linux 阻塞和非阻塞 IO 实验	1295
52.1 阻塞和非阻塞 IO.....	1296
52.1.1 阻塞和非阻塞简介	1296
52.1.2 等待队列.....	1297
52.1.3 轮询	1299
52.1.4 Linux 驱动下的 poll 操作函数	1303
52.2 阻塞 IO 实验.....	1304
52.2.1 硬件原理图分析	1304
52.2.2 实验程序编写	1304
52.2.3 运行测试.....	1311
52.3 非阻塞 IO 实验.....	1312
52.3.1 硬件原理图分析	1312
52.3.2 实验程序编写	1313

52.3.3 运行测试.....	1318
第五十三章 异步通知实验	1321
53.1 异步通知.....	1322
53.1.1 异步通知简介	1322
53.1.2 驱动中的信号处理	1324
53.1.3 应用程序对异步通知的处理.....	1326
53.2 硬件原理图分析	1326
53.3 实验程序编写	1326
53.3.1 修改设备树文件	1327
53.3.2 程序编写.....	1327
53.3.3 编写测试 APP.....	1330
53.4 运行测试.....	1332
53.4.1 编译驱动程序和测试 APP	1332
53.4.2 运行测试.....	1333
第五十四章 platform 设备驱动实验	1334
54.1 Linux 驱动的分离与分层	1335
54.1.1 驱动的分隔与分离	1335
54.1.2 驱动的分层.....	1337
54.2 platform 平台驱动模型简介	1337
54.2.1 platform 总线	1337
54.2.2 platform 驱动	1339
54.2.3 platform 设备	1344
54.3 硬件原理图分析.....	1347
54.4 试验程序编写	1347
54.4.1 platform 设备与驱动程序编写	1347
54.4.2 测试 APP 编写.....	1357
54.5 运行测试.....	1358
54.5.1 编译驱动程序和测试 APP	1358
54.4.2 运行测试.....	1359
第五十五章 设备树下的 platform 驱动编写	1361
55.1 设备树下的 platform 驱动简介	1362
55.2 硬件原理图分析.....	1363
55.3 实验程序编写.....	1363

55.3.1 修改设备树文件	1363
55.3.2 platform 驱动程序编写	1363
55.3.3 编写测试 APP	1369
55.4 运行测试	1369
55.4.1 编译驱动程序和测试 APP	1369
55.4.2 运行测试	1370
第五十六章 Linux 自带的 LED 灯驱动实验	1371
56.1 Linux 内核自带 LED 驱动使能	1372
56.2 Linux 内核自带 LED 驱动简介	1373
56.2.1 LED 灯驱动框架分析	1373
56.2.2 module_platform_driver 函数简析	1374
56.2.3 gpio_led_probe 函数简析	1375
56.3 设备树节点编写	1378
56.4 运行测试	1378
第五十七章 Linux MISC 驱动实验	1380
57.1 MISC 设备驱动简介	1381
57.2 硬件原理图分析	1382
57.3 实验程序编写	1382
57.3.1 修改设备树	1382
57.3.2 beep 驱动程序编写	1382
57.3.3 编写测试 APP	1388
57.4 运行测试	1389
57.4.1 编译驱动程序和测试 APP	1389
57.4.2 运行测试	1390
第五十八章 Linux INPUT 子系统实验	1391
58.1 input 子系统	1392
58.1.1 input 子系统简	1392
58.1.2 input 驱动编写流程	1392
58.1.3 input_event 结构体	1398
58.2 硬件原理图分析	1399
58.3 实验程序编写	1399
58.3.1 修改设备树文件	1399
58.3.2 按键 input 驱动程序编写	1399

58.3.3 编写测试 APP.....	1405
58.4 运行测试.....	1407
58.4.1 编译驱动程序和测试 APP	1407
58.4.2 运行测试.....	1408
58.5 Linux 自带按键驱动程序的使用	1410
58.5.1 自带按键驱动程序源码简析.....	1410
58.5.2 自带按键驱动程序的使用.....	1415
第五十九章 Linux LCD 驱动实验	1417
59.1 Linux 下 LCD 驱动简析	1418
59.1.1 Framebuffer 设备	1418
59.1.2 LCD 驱动简析	1419
59.2 硬件原理图分析.....	1424
59.3 LCD 驱动程序编写	1425
59.4 运行测试.....	1430
59.4.1 LCD 屏幕基本测试	1430
59.4.2 设置 LCD 作为终端控制台	1431
59.4.3 LCD 背光调节	1432
59.4.4 LCD 自动关闭解决方法.....	1433
第六十章 Linux RTC 驱动实验	1435
60.1 Linux 内核 RTC 驱动简介	1436
60.2 I.MX6U 内部 RTC 驱动分析.....	1440
60.3 RTC 时间查看与设置.....	1445
第六十一章 Linux I2C 驱动实验.....	1448
61.1 Linux I2C 驱动框架简介	1449
61.1.1 I2C 总线驱动.....	1449
61.1.2 I2C 设备驱动	1450
61.1.3 I2C 设备和驱动匹配过程.....	1454
61.2 I.MX6U 的 I2C 适配器驱动分析	1455
61.3 I2C 设备驱动编写流程	1461
61.3.1 I2C 设备信息描述	1462
61.3.2 I2C 设备数据收发处理流程.....	1463
61.4 硬件原理图分析.....	1466
61.5 实验程序编写.....	1466

61.5.1 修改设备树.....	1466
61.5.2 AP3216C 驱动编写.....	1468
61.5.3 编写测试 APP.....	1478
61.6 运行测试.....	1480
61.6.1 编译驱动程序和测试 APP	1480
61.6.2 运行测试.....	1480
第六十二章 Linux SPI 驱动实验	1482
62.1 Linux 下 SPI 驱动框架简介	1483
62.1.1 SPI 主机驱动	1483
62.1.2 SPI 设备驱动	1485
62.1.3 SPI 设备和驱动匹配过程.....	1487
62.2 I.MX6U SPI 主机驱动分析	1488
62.3 SPI 设备驱动编写流程.....	1490
62.3.1 SPI 设备信息描述	1490
62.3.2 SPI 设备数据收发处理流程.....	1491
62.4 硬件原理图分析	1495
62.5 试验程序编写	1495
62.5.1 修改设备树.....	1495
62.5.2 编写 ICM20608 驱动	1496
62.5.3 编写测试 APP.....	1507
62.6 运行测试.....	1510
62.6.1 编译驱动程序和测试 APP	1510
62.6.2 运行测试.....	1511
第六十三章 Linux RS232/485/GPS 驱动实验	1512
63.1 Linux 下 UART 驱动框架	1513
63.2 I.MX6U UART 驱动分析	1516
63.3 硬件原理图分析	1522
63.4 RS232 驱动编写	1523
63.5 移植 minicom.....	1525
63.6 RS232 驱动测试.....	1528
63.6.1 RS232 连接设置	1528
63.6.2 minicom 设置.....	1529
63.6.3 RS232 收发测试	1531

63.7 RS485 测试.....	1533
63.7.1 RS485 连接设置	1533
63.7.2 RS485 收发测试	1534
63.8 GPS 测试.....	1535
63.8.1 GPS 连接设置.....	1535
63.8.2 GPS 数据接收测试.....	1536
第六十四章 Linux 多点电容触摸屏实验	1538
64.1 Linux 下电容触摸屏驱动框架简介.....	1539
64.1.1 多点触摸(MT)协议详解.....	1539
64.1.2 Type A 触摸点信息上报时序	1540
64.1.3 Type B 触摸点信息上报时序	1542
64.1.4 MT 其他事件的使用	1544
64.1.5 多点触摸所使用到的 API 函数.....	1544
64.1.6 多点电容触摸驱动框架.....	1546
64.2 硬件原理图分析.....	1550
64.3 试验程序编写.....	1550
64.3.1 修改设备树.....	1551
64.3.2 编写多点电容触摸驱动.....	1552
64.4 运行测试.....	1562
64.4.1 编译驱动程序.....	1562
64.4.2 运行测试.....	1563
64.4.3 将驱动添加到内核中	1564
64.5 tslib 移植与使用	1566
64.5.1 tslib 移植.....	1566
64.5.2 tslib 测试.....	1567
64.6 使用内核自带的驱动	1569
64.7 4.3 寸屏触摸驱动实验	1571
64.8 7 寸触摸 GT911 驱动	1574
第六十五章 Linux 音频驱动实验.....	1576
65.1 音频接口简介.....	1577
65.1.1 为何需要音频编解码芯片?	1577
65.1.2 WM8960 简介.....	1577
65.1.3 I2S 总线接口	1579

65.1.4 I.MX6ULL SAI 简介	1580
65.2 硬件原理图分析	1581
65.3 音频驱动使能	1582
65.3.1 修改设备树	1582
65.3.2 使能内核的 WM8960 驱动	1587
65.4 alsa-lib 和 alsa-utils 移植	1589
65.4.1 alsa-lib 移植	1590
65.4.2 alsa-utils 移植	1591
65.5 声卡设置与测试	1592
65.5.1 amixer 使用方法	1592
65.5.2 音乐播放测试	1594
65.5.3 MIC 录音测试	1595
65.5.4 LINE IN 录音测试	1598
65.6 开机自动配置声卡	1600
65.7 mplayer 播放器移植与使用	1601
65.7.1 mplayer 移植	1601
65.7.2 mplayer 使用	1603
65.8 alsamixer 简介	1604
第六十六章 Linux CAN 驱动实验	1606
66.1 CAN 协议简析	1607
66.1.1 何为 CAN?	1607
66.1.2 CAN 电气属性	1608
66.1.3 CAN 协议	1609
66.1.4 CAN 速率	1615
66.1.5 I.MX6ULL FlexCAN 简介	1617
66.2 硬件原理图分析	1619
66.3 实验程序编写	1619
66.3.1 修改设备树	1619
66.3.2 使能 Linux 内核自带的 FlexCAN 驱动	1621
66.4 FlexCAN 测试	1622
66.4.1 检查 CAN 网卡设备是否存在	1622
66.4.2 移植 iproute2	1623
66.4.3 移植 can-utils 工具	1625

66.4.4 CAN 通信测试.....	1625
第六十七章 Linux USB 驱动实验	1629
67.1 USB 接口简介	1630
67.1.1 什么是 USB?	1630
67.1.2 USB 电气特性	1631
67.1.3 USB 拓扑结构	1632
67.1.4 什么是 USB OTG?	1633
67.1.5 I.MX6ULL USB 接口简介	1634
67.2 硬件原理图分析	1635
67.2.1 USB HUB 原理图分析	1635
67.2.2 V2.4 版本以前底板 USB OTG 原理图分析	1638
67.2.3 V2.4 及以后版本底板 USB OTG 原理图分析	1639
67.3 USB 协议简析	1640
67.3.1 USB 描述符	1640
67.3.3 USB 数据包类型	1643
67.3.4 USB 传输类型	1644
67.3.5 USB 枚举	1645
67.4 Linux 内核自带 HOST 实验	1646
67.4.1 USB 鼠标键盘测试	1646
67.4.2 U 盘实验	1649
67.5 Linux 内核自带 USB OTG 实验	1651
67.5.1 修改设备树	1651
67.5.2 OTG 主机实验	1652
67.5.3 OTG 从机实验	1652
第六十八章 Linux 块设备驱动实验	1657
68.1 什么是块设备?	1658
68.2 块设备驱动框架	1658
68.2.1 block_device 结构体	1658
68.2.2 gendisk 结构体	1660
68.2.3 block_device_operations 结构体	1662
68.2.4 块设备 I/O 请求过程	1663
68.3 使用请求队列实验	1669
68.3.1 实验程序编写	1669

68.3.2 运行测试.....	1675
68.4 不使用请求队列实验	1676
68.4.1 实验程序编写	1676
68.4.2 运行测试.....	1678
第六十九章 Linux 网络驱动实验	1680
69.1 嵌入式网络简介	1681
69.1.1 嵌入式下的网络硬件接口.....	1681
69.1.2 MII/RMII 接口	1682
69.1.3 MDIO 接口	1684
69.1.4 RJ45 接口	1684
69.1.5 I.MX6ULL ENET 接口简介	1685
69.2 PHY 芯片详解.....	1686
69.2.1 PHY 基础知识简介	1686
69.2.2 LAN8720A 详解.....	1687
69.2.3 SR8201F 详解.....	1693
69.3 Linux 内核网络驱动框架	1697
69.3.1 net_device 结构体.....	1697
69.3.2 net_device_ops 结构体.....	1703
69.3.3 sk_buff 结构体.....	1705
69.3.4 网络 NAPI 处理机制	1710
69.4 I.MX6ULL 网络驱动简介	1712
69.4.1 I.MX6ULL 网络外设设备树	1712
69.4.2 I.MX6ULL 网络驱动源码简析	1716
69.4.3 fec_netdev_ops 操作集	1729
69.4.3 Linux 内核 PHY 子系统与 MDIO 总线简析.....	1734
69.5 网络驱动实验测试	1743
69.5.1 LAN8720 PHY 驱动测试	1743
69.5.2 通用 PHY 驱动测试	1743
69.5.3 DHCP 功能配置	1744
69.6 单网卡使用.....	1744
69.6.1 只使用 ENET2 网卡	1745
69.6.2 只使用 ENET1 网卡	1746
第七十章 Linux WIFI 驱动实验	1749

70.1 WIFI 驱动添加与编译	1750
70.1.1 向 Linux 内核添加 WIFI 驱动	1750
70.1.2 配置 Linux 内核	1753
70.1.3 编译 WIFI 驱动	1754
70.1.4 驱动加载测试	1756
70.2 wireless tools 工具移植与测试	1759
70.2.1 wireless tools 移植	1759
70.2.2 wireless tools 工具测试	1759
70.3 wpa_supplicant 移植	1761
70.3.1 openssl 移植	1761
70.3.2 libnl 库移植	1761
70.3.3 wpa_supplicant 移植	1762
70.4 WIFI 联网测试	1764
70.4.1 RTL8188 USB WIFI 联网测试	1764
70.4.2 RTL8189 SDIO WIFI 联网测试	1766
第七十一章 Linux 4G 通信实验	1768
71.1 4G 网络连接简介	1769
71.2 高新兴 ME3630 4G 模块实验	1771
71.2.1 ME3630 4G 模块简介	1771
71.2.2 ME3630 4G 模块驱动修改	1772
71.2.3 ME3630 4G 模块 ppp 联网测试	1775
71.2.4 ME3630 4G 模块 ECM 联网测试	1780
71.2.5 ME3630 4G 模块 GNSS 定位测试	1781
71.3 EC20 4G 模块实验	1783
71.3.1 EC20 4G 模块简介	1783
71.3.2 EC20 4G 模块驱动修改	1784
71.3.3 quectel-CM 移植	1789
71.3.4 EC20 上网测试	1789
第七十二章 RGB 转 HDMI 实验	1791
72.1 RGB 转 HDMI 简介	1792
72.2 硬件原理图分析	1792
72.3 实验驱动编写	1793
72.3.1 修改设备树	1793

72.3.2 使能内核自带的 sii902x 驱动	1796
72.3.3 修改 sii902x 驱动	1796
72.4 RGB 转 HDMI 测试	1797
第七十三章 Linux PWM 驱动实验	1799
73.1 PWM 驱动简析	1800
73.1.1 设备树下的 PWM 控制器节点	1800
73.1.2 PWM 子系统	1800
73.1.3 PWM 驱动源码分析	1801
73.2 PWM 驱动编写	1805
73.2.1 修改设备树	1805
73.2.2 使能 PWM 驱动	1807
73.3 PWM 驱动测试	1807
73.4 PWM 背光设置	1809
第七十四章 Regmap API 实验	1810
74.1 Regmap API 简介	1811
74.1.1 什么是 Regmap	1811
74.1.2 Regmap 驱动框架	1811
74.1.3 Regmap 操作函数	1815
74.1.4 regmap_config 掩码设置	1817
74.2 实验程序编写	1819
74.3 运行测试	1823
第七十五章 Linux IIO 驱动实验	1824
75.1 IIO 子系统简介	1825
75.1.1 iio_dev	1825
75.1.2 iio_info	1828
75.1.3 iio_chan_spec	1829
75.2 IIO 驱动框架创建	1833
75.2.1 基础驱动框架建立	1833
75.2.2 IIO 设备申请与初始化	1835
75.3 实验程序编写	1838
75.3.1 使能内核 IIO 相关配置	1838
75.3.2 ICM20608 的 IIO 驱动框架搭建	1839
75.3.3 完善 icm20608_read_raw 函数	1852

75.3.4 完善 icm20608_write_raw 函数.....	1858
75.4 测试应用程序编写	1862
75.4.1 linux 文件流读取.....	1862
75.4.2 编写测试 APP.....	1864
75.4.3 运行测试.....	1870
第七十六章 Linux ADC 驱动实验.....	1871
76.1 ADC 简介	1872
76.2 ADC 驱动源码简析	1872
76.2.1 设备树下的 ADC 节点	1872
76.2.2 ADC 驱动源码分析	1872
76.3 硬件原理图分析.....	1878
76.4 ADC 驱动编写.....	1879
76.4.1 修改设备树.....	1879
76.4.2 使能 ADC 驱动.....	1880
76.4.3 编写测试 APP.....	1881
76.5 运行测试.....	1884
76.5.1 编译驱动程序和测试 APP	1884
76.5.2 运行测试.....	1885
附录 A 其他根文件系统构建.....	1886
第 A1 章 Buildroot 根文件系统构建	1887
A1.1 何为 buildroot?	1888
A1.1.1 buildroot 简介	1888
A1.1.2 buildroot 下载	1888
A1.2 buildroot 构建根文件系统.....	1889
A1.2.1 配置 buildroot	1889
A1.2.2 编译 buildroot	1894
A1.2.3 buildroot 根文件系统测试	1895
A1.3 buildroot 第三方软件和库的配置	1895
A1.4 buildroot 下的 busybox 配置.....	1897
A1.4.1 busybox 配置	1897
A1.4.2 busybox 中文字符的支持	1898
A1.4.3 编译 busybox	1898
A1.5 根文件系统测试.....	1899

第 A2 章 Yocto 根文件系统构建	1901
第 A3 章 Ubuntu-base 根文件系统构建.....	1902
A3.1 ubuntu-base 获取.....	1903
A3.2 ubuntu 根文件系统构建	1905
A3.2.1 解压缩 ubuntu base 根文件系统	1905
A3.2.2 安装 qemu.....	1905
A3.2.3 设置软件源.....	1906
A3.2.4 在主机挂载并配置根文件系统	1906
A3.3 ubuntu 根文件系统测试	1908
A3.3.1 nfs 挂载测试	1908
A3.3.2 ubuntu 根文件系统烧写.....	1909
A3.4 ubuntu 系统使用	1910
A3.4.1 添加新用户	1910
A3.4.2 网络 DHCP 配置.....	1911
A3.4.3 安装软件.....	1911
A3.4.4 FTP 服务器搭建	1911
A3.4.5 gcc 和 make 工具安装	1913
附录 B 其他第三方软件移植.....	1915
第 B1 章 开发板 FTP 服务器移植与搭建	1916
B1.1 vsftpd 源码下载	1917
B1.2 vsftpd 移植	1918
B1.3 vsftpd 服务器测试	1918
B1.3.1 配置 vsftpd	1918
B1.3.2 添加新用户	1919
B1.3.3 Filezilla 连接测试	1920
第 B2 章节 开发板 OpenSSH 移植与使用	1923
B2.1 OpenSSH 简介	1924
B2.2 OpenSSH 移植	1924
B2.2.1 OpenSSH 源码获取.....	1924
B2.2.2 移植 zlib 库	1924
B2.2.3 移植 openssl 库	1924
B2.2.4 移植 openssh 库.....	1924
B2.3 openssh 设置	1926

B2.4 openssh 使用	1926
B2.4.1 ssh 登录	1926
B2.4.2 scp 命令拷贝文件	1928
第 B3 章节 嵌入式 GDB 调试搭建与使用	1930
B3.1 GDB 简介	1931
B3.2 GDB 移植	1931
B3.2.1 获取 gdb 和 gdbserver 源码	1931
B3.2.2 编译 gdb	1931
B3.2.3 移植 gdbserver	1933
B3.3 使用 GDB 进行嵌入式程序调试	1934
B3.3.1 编写一个测试应用	1934
B3.3.2 GDB 调试程序	1935
第 B4 章节 VSCode+gdbserver 图形化调试	1938
B4.1 VSCode 设置	1939
B4.2 VSCode 调试方法	1940
附录 C 裸机例程补充	1943
第 C1 章 ADC 实验	1944
C1.1 ADC 简介	1945
C1.1.1 什么是 ADC	1945
C1.1.2 I.MX6ULL ADC 简介	1945
C1.2 硬件原理分析	1949
C1.3 实验程序编写	1950
C1.4 编译下载验证	1956
C1.4.1 编写 Makefile 和链接脚本	1956
C1.4.2 编译下载	1956

前言

当今社会是一个电子信息技术飞速发展的年代,小到玩具,家电,大到手机、飞机、潜艇等,都离不开电子信息技术。社会上对于电子信息方面的人才需求也很大,从就业的角度而言电子、计算机、通信以及信息方面专业的毕业生工资都比较高,就业也比较容易,尤其是嵌入式、Linux 和 Android 等相关开发工作。

本书主要讲解嵌入式 Linux 中的驱动开发,也会涉及到裸机开发的内容,相信大部分读者和作者一样,以前都是做单片机开发的工作,比如 51 或者 STM32 等。单片机开发很难接触到更高层次的系统方面的知识,单片机用到的系统都很简单,比如 UCOS、FreeRTOS 等等,这些操作系统都是一个 kernel,如果需要网络、文件系统、GUI 等这些就需要开发者自行移植。而移植又是非常痛苦的一件事情,而且移植完成以后的稳定性也无法保证。即使移植成功以后后续的开发工作也比较繁琐,因为不同的组件其 API 操作函数都不同,没有一个统一的标准,使用起来的话学习成本比较高。这个时候一个功能完善的操作系统显得尤为重要:具有统一的标准;提供完善的多任务管理、存储管理、设备管理、文件管理和网络等。Linux 就是这样一个系统,当然这样的系统还有很多,比如 Windows, MacOS, UNIX 等等。本书我们讲解 Linux,而 Linux 开发可以分为底层驱动开发和应用开发,本书讲解的是 Linux 的驱动开发,主要面向与那些此前使用 STM32 的开发者。平心而论,如果此前只会 51 单片机开发的话我是非常不建议直接上手 Linux 驱动开发的,因为 51 和 Linux 驱动开发的差异太大了!笔者建议在学习嵌入式 Linux 驱动开发之前一定要学一下 STM32 这种 Cortex-M 内核的 MCU,因为 STM32 这样的 MCU 其内部资源基本和可以运行 Linux 的 CPU 差不多,如果会 STM32 的话上手 Linux 驱动开发就会容易很多。笔者就是此前做了 4 年 STM32 开发工作,然后转的 Linux 驱动开发,整个过程比较顺畅。

鉴于当前 STM32 非常火爆,学习者众多,如何帮助 STM32 学习者顺利的转入 Linux 驱动开发是笔者思考了很久的问题。为此笔者本书和相应例程的安排如下:

1、选取合适的 CPU

理论上来讲,如果 ST 公司有可以运行的 Linux 的芯片那再好不过了,因为大家对 STM32 很熟悉,但是在写本书的时候 ST 也没有可以运行 Linux 的 CPU。Linux 驱动开发入门的 CPU 一定不能复杂!!! 比如像三星的 Exynos 4412、Exynos 4418 等这些手机 CPU,这些 CPU 性能很强大,带有 GPU、支持硬件视频解码、可以运行 Android。但是正是它们的性能过于强大,功能过于繁杂,所以不适合 Linux 驱动开发入门。一款外设和 STM32H7 这样的 MCU 差不多的 CPU 就非常适合 Linux 入门,三星的 2440 就非常合适,但是 2440 早已停产了,学了以后工作上肯定又用不到了,又得学习其他的 CPU,有点浪费时间。作者花了不少时间终于找到了一款合适的 CPU,那就是 NXP 的 I.MX6UL! 可以认为 I.MX6UL 就是一款可以跑 Linux 的 STM32,外设功能和 STM32 相似,如果此前学习过 STM32 的话会非常容易上手 I.MX6UL。而且目前 I.MX6UL 正在大量出货,这是一款工业级的 CPU,大量的以前三星 2440、6410 做的产品更新换代的绝佳之选,学习完 I.MX6UL 以后工作就可以直接使用了。本书选取开发平台为正点原子的 I.MX6U-ALPHA 开发板,其他厂商的 I.MX6UL 开发板也可以参考本书。

2、开发环境讲解

STM32 的开发都是在 Windows 系统下进行的,使用 MDK 或者 IAR 这样的集成 IDE,但是嵌入式 Linux 驱动开发需要的主机是 Linux 平台的,也就是你必须先在自己的电脑上安装 Linux 系统,Linux 系统发行版有 Ubuntu、CentOS、Fedora、Debian 等等,本书我们使用 Ubuntu 操作系统。本书假设大家此前从来没有接触过 Ubuntu 操作系统,因此会有详细的 Ubuntu 操作

系统安装、使用教程,帮助大家熟悉开发环境。

3、合理的裸机例程

学习嵌入式 Linux 驱动开发建议大家先学习裸机开发(当然了,如果学习过 STM32 的话可以跳过裸机学习),Linux 驱动开发非常庞大、繁琐。要想进行 Linux 驱动开发,必须要先移植 Uboot、然后移植 Linux 系统和根文件系统到你的开发平台上。而 Uboot 又是一个超大的裸机综合例程,因此如果你没有学习过裸机例程那么 Uboot 移植将会有点困难,尤其是当要修改 Uboot 代码的时候。做 STM32 开发的话基本都是裸机开发,在 IDE 平台下编写代码,可以使用 ST 提供的库。但是在 Ubuntu 下编写 I.MX6UL 裸机例程的话就没有这么方便了,没有 MDK 和 IAR 这样的 IDE,没有 ST 提供的库。所有的一切都需要我们自己搭建,大家不用担心,本书包括视频会有详细的讲解。在裸机例程内容方面,我们提供了数十个裸机例程,由浅入深,涵盖了大部分常用的功能,比如 IO 输入和输出、中断、串口、定时器、DDR、LCD、I2C 等。学习完裸机例程以后基本就对 I.MX6UL 这颗 CPU 非常了解了。再去学习 Linux 驱动开发的话就很轻松了。

4、Uboot、Linux 和根文件系统移植

学习完裸机例程以后就是 Linux 驱动开发了,但是在进行 Linux 驱动开发之前要先在使用的开发板平台上移植好 Uboot, Linux 和根文件系统。这是 Linux 驱动开发的第一个拦路虎,因此本书和相应的视频会着重讲解 Uboot/Linux 和根文件系统的移植。

5、嵌入式 Linux 驱动开发

当我们把 Uboot, Linux 和根文件系统都在开发板上移植好了以后就可以开始 Linux 驱动开发了。Linux 驱动有三大类:字符设备驱动、块设备驱动和网络设备驱动,这三大类我们都会详细的讲解,并且配有数十个相应的例程,由简入深,从最简单的点灯,到最后的网络设备驱动。

本书一共分四篇,每篇对应一个不同的阶段:

第一篇: Ubuntu 操作系统入门

本篇主要讲解 Ubuntu 操作系统的使用,本篇不涉及到任何嵌入式方面的知识,全部是在 PC 上完成的,只要安装好 Ubuntu 操作系统即可。

第二篇: ARM 裸机开发

从本篇开始我们就正式开始使用开发板学习了,本篇通过数十个裸机例程来帮助大家了解 I.MX6UL 这颗 CPU。为以后的 Linux 驱动开发做准备,通过本篇大家可以掌握在 Ubuntu 下进行 ARM 开发的方法。

第三篇: Uboot、Linux 和根文件系统移植

本篇讲解如何将 Uboot、Linux 和根文件系统移植到我们的开发板上,为后面的 Linux 驱动开发做准备。

第四篇: Linux 驱动开发

前面做了那么多的工作就是为了本篇,因此本篇注定了将是本书重中之重,大家应该投入精力最多的一篇,望大家做好准备

通过上面四篇的学习,大家基本掌握了嵌入式 Linux 驱动的开发流程,本书旨在引导大家入门 Linux 驱动开发,更加深入的研究就需要大家自行查阅其他更加专业的书籍了,祝愿大家学习顺利。

第一篇 Ubuntu 系统入门篇

本篇主要讲解 Ubuntu 系统,包括如何在虚拟机上安装 Ubuntu 操作系统,安装好以后 Ubuntu 的设置、基本操作等等。在正式进行嵌入式开发之前肯定是要先学会 Ubuntu 系统如何使用的,由于 Ubuntu 系统功能很庞大,如果要详细讲解的话一本书都讲不完,因此本篇只做 Ubuntu 系统入门讲解,掌握我们进行嵌入式开发所需的技能即可,如果想详细的学习 Ubuntu 系统的话可以参考其他的书籍,比如经典的《鸟哥的 linux 私房菜》,《鸟哥的 linux 私房菜》这本书使用的 CentOS 操作系统,但是 Ubuntu 下完全可以使用。

当 Ubuntu 系统入门以后,我们重点要学的就是如何在 Linux 下进行 C 语言开发,如何使用 gcc 编译器、如何编写 Makefile 文件等等。

如果此前已经使用过 Ubuntu 操作系统,并且从事过 Linux C 编程工作的话本篇就不需要看了,可以直接跳到第二篇,开始 ARM 裸机开发篇的学习。

第一章 Ubuntu 系统安装

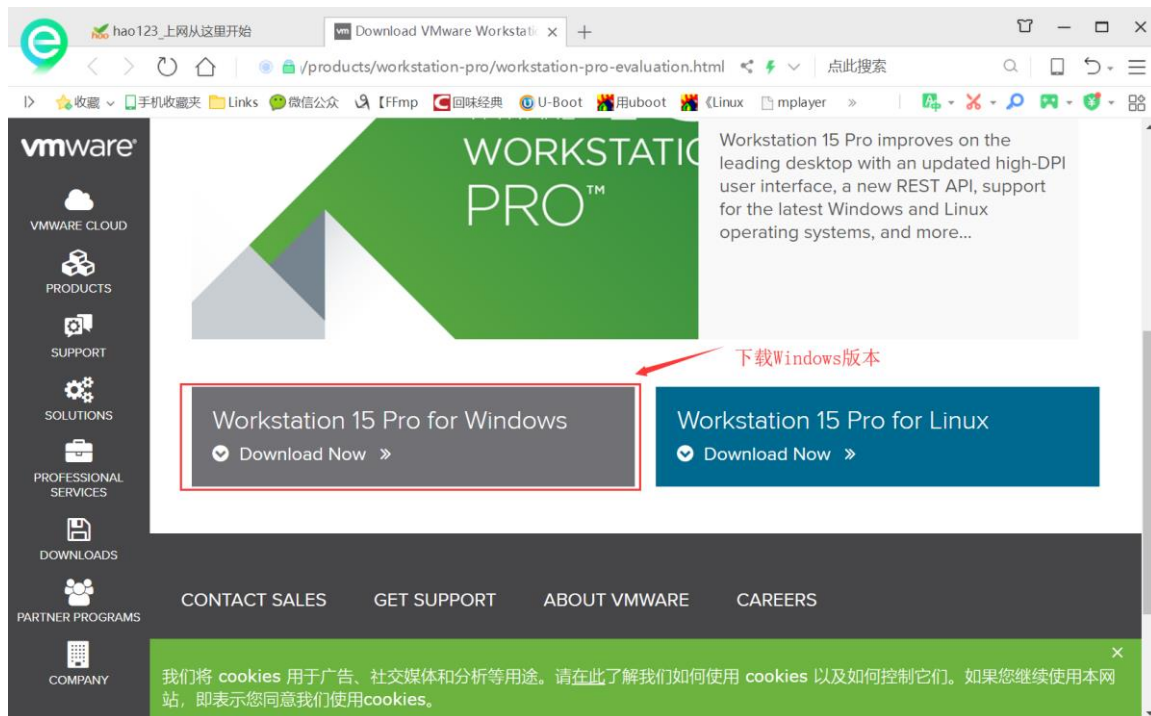
Linux 的开发需要在 Linux 系统下进行,这就要求我们的 PC 主机安装 Linux 系统,本书我们选择 Ubuntu 这个 Linux 发行版系统。本章讲解如何安装虚拟机,以及如何在虚拟机中安装 Ubuntu 系统,安装完成以后如何做简单的设置。如果已经对于虚拟机以及 Ubuntu 基础操作已经熟悉的话就可以跳过本章。

1.1 安装虚拟机软件 VMware

不是安装 Ubuntu 吗？怎么要先安装虚拟机呢？虚拟机是个啥？相信大部分第一次安装 Ubuntu 的人都会有这个疑问。我不能直接安装 Ubuntu 吗？能不能不要虚拟机呢？答案是肯定可以的！直接在电脑上安装 Ubuntu 以后你的电脑就是一个真真正正的 Ubuntu 电脑了，你可以再安装一个 Windows 系统，这样你的电脑就是双系统了，在开机的时候可以选择不同的系统启动。但是这样的话会有一个问题，那就是你每次只能选择其中的一个系统启动，要么 Windows 要么 Ubuntu，但是我们在再开发的时候很多时候需要再 Windows 和 Ubuntu 下来回切换，Windows 系统下的软件资源要比 Ubuntu 下丰富的多，比如我们在 Windows 用 Source Insight 这个神器编写代码，然后拿到 Ubuntu 下编译。这个就涉及到两个系统切换问题，显然如果你直接在电脑上安装 Ubuntu 以后就没法做到，因为你每次开机只能在 Windows 和 Ubuntu 下二选一。

如果 Ubuntu 系统能作为 Windows 下的一个软件就好了，我们默认启动 Windows 系统，需要用到 Ubuntu 的话直接打开这个软件就行了。当然可以！这个就要借助虚拟机了，虚拟机顾名思义就是虚拟出一个机器，然后你就可以在这个机器上安装任何你想要的系统，相当于在克隆出一个你的电脑，这样在主机上运行 Windows 系统，当我们需要用到 Ubuntu 的话就打开安装有 Ubuntu 系统的虚拟机。

虚拟机的实现我们可以借助其他软件，比如 VMware Workstation，Vmware Workstation 是收费软件，免费的虚拟机软件有 Virtualbox。本书我们使用 Vmware Workstation 软件来做虚拟机。Vmware Workstation 软件可以在 Vmware 官网下载，下载地址：<https://www.vmware.com/products/workstation-pro/workstation-pro-evaluation.html>，当前最新的版本是 VMware Workstation Pro 15，我们下载 Windows 版本的，如下图所示：



我们已经在开发板光盘里面提供了 VMware Workstation 软件，大家可以直接使用，在光盘目录：[开发板光盘->3、软件->VMware-workstation-full-15.5.0-14665864.exe](#)。VMware Workstation 的安装和普通软件安装一样，双击 VMware-workstation-full-15.5.0-14665864.exe 进入安装界面，如图 1.1.1 所示：



图 1.1.1 VMware 安装界面

点击图 1.1.1 中的“下一步”，进入图 1.1.2 所示步骤：

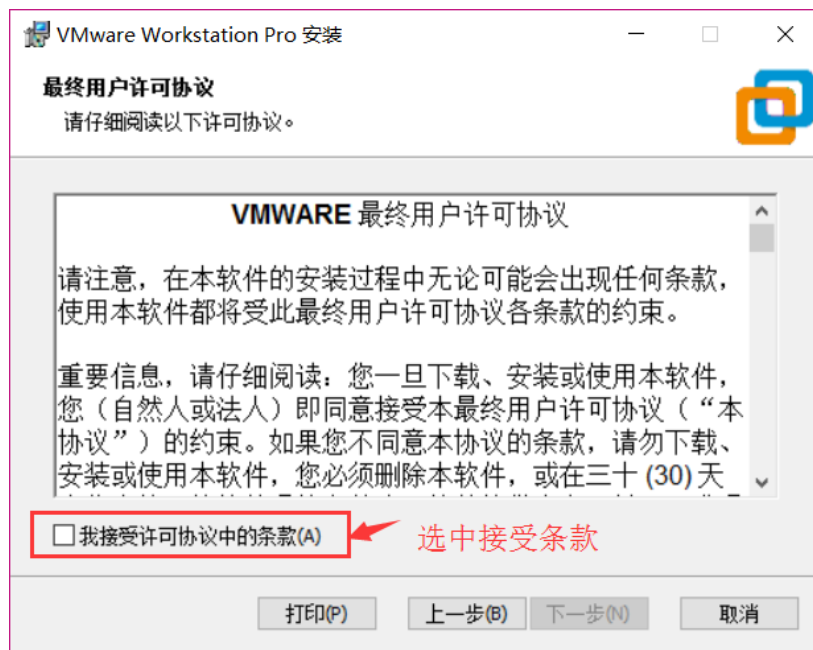


图 1.1.2 VMware 条款

先选择图 1.1.2 中的“我接受许可协议中的条款”，然后在选择“下一步”，进入图 1.1.3 所示步骤：

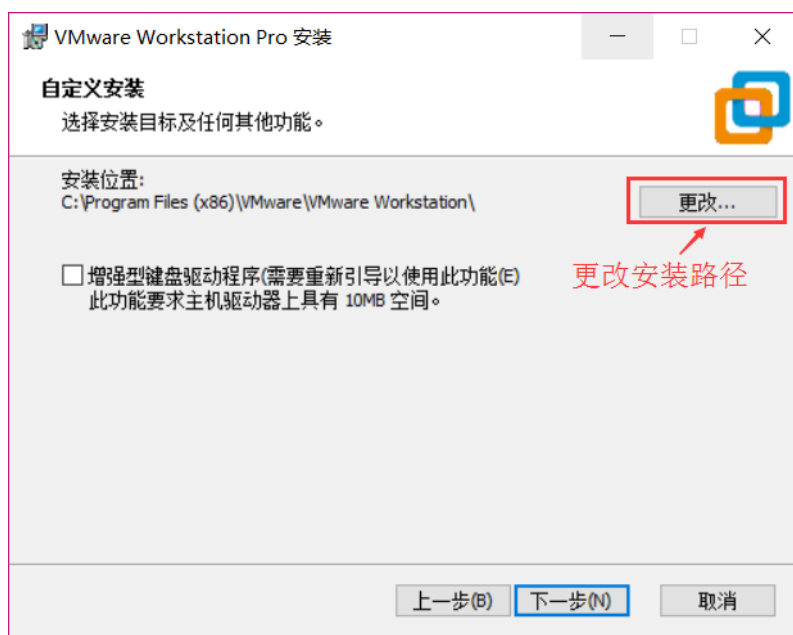


图 1.1.3 选择安装路径

图 1.1.3 中选择软件的安装路径，点击“更改”按钮，然后根据自己的实际需要选择合适路径即可,我的安装路径如图 1.1.4 所示：

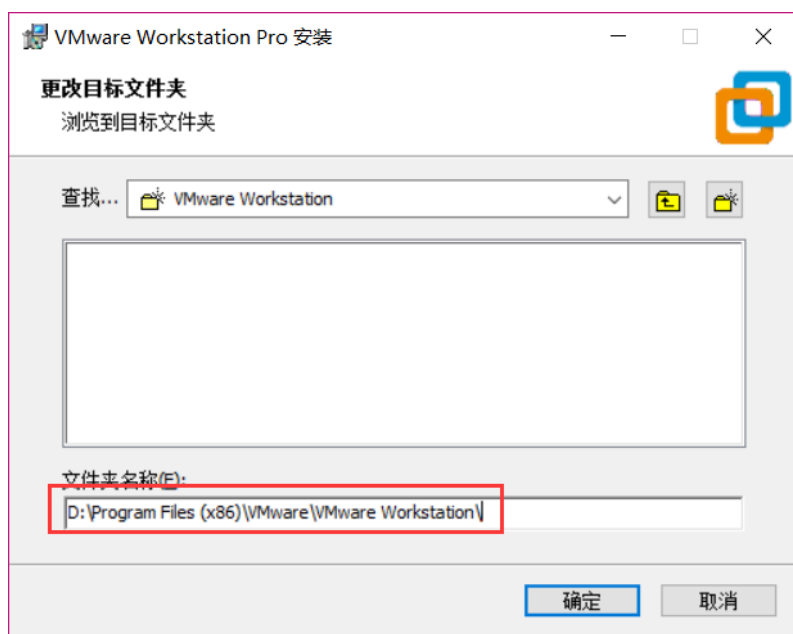


图 1.1.4 安装路径

选择好路径以后点击图 1.1.4 中的“确定”按钮，然后回到图 1.1.3 所示界面，点击图 1.1.3 中的“下一步”，进入图 1.1.5 所示界面：

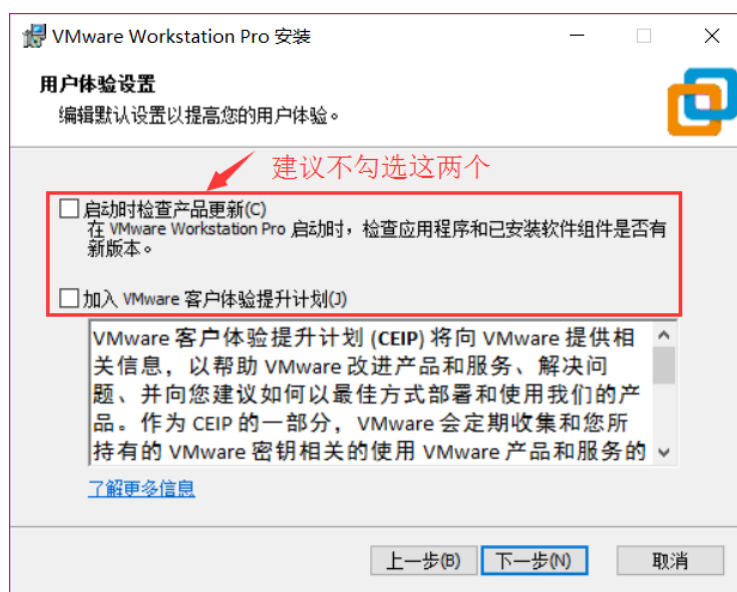


图 1.1.5 检查更新界面

在图 1.1.5 中，会有两个复选框，默认都是选中的，建议不要选中！然后点击图 1.1.5 中的“下一步”按钮，进入图 1.1.6 所示界面：

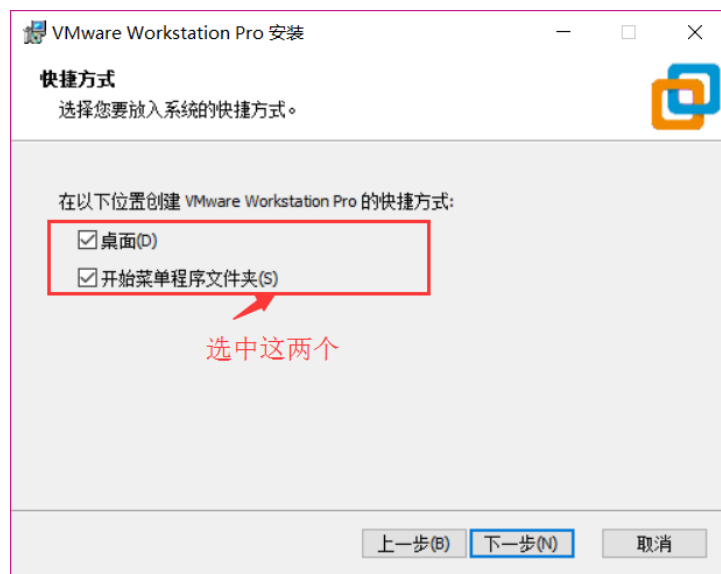


图 1.1.6 快捷方式设置

在图 1.1.6 中有两个选项，我们都选中，这样在安装完成以后就会在开始菜单和桌面上有 VMware 的图标，选中以后点击图 1.1.6 中的“下一步”，进入图 1.1.7 界面：

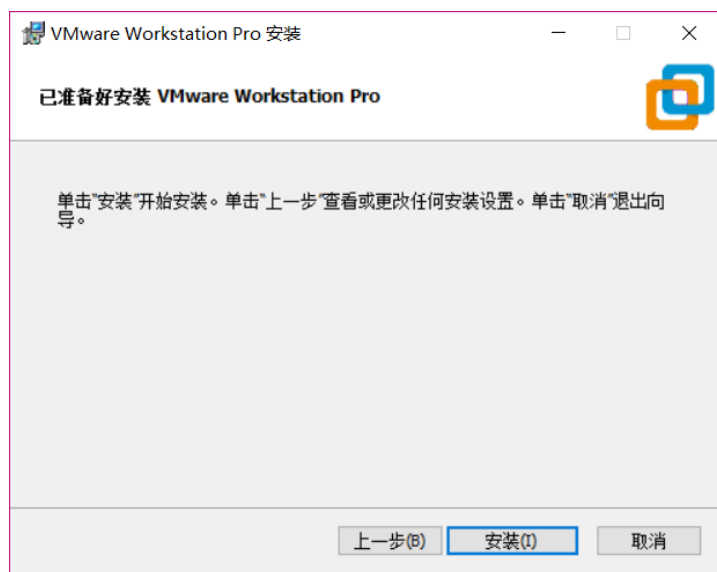


图 1.1.7 安装确定界面

前面几步已经设置好安装参数了，如果还需要修改安装参数的话就点击图 1.1.7 中的“安装”按钮开始安装 VMware，安装过程如图 1.1.8 所示：

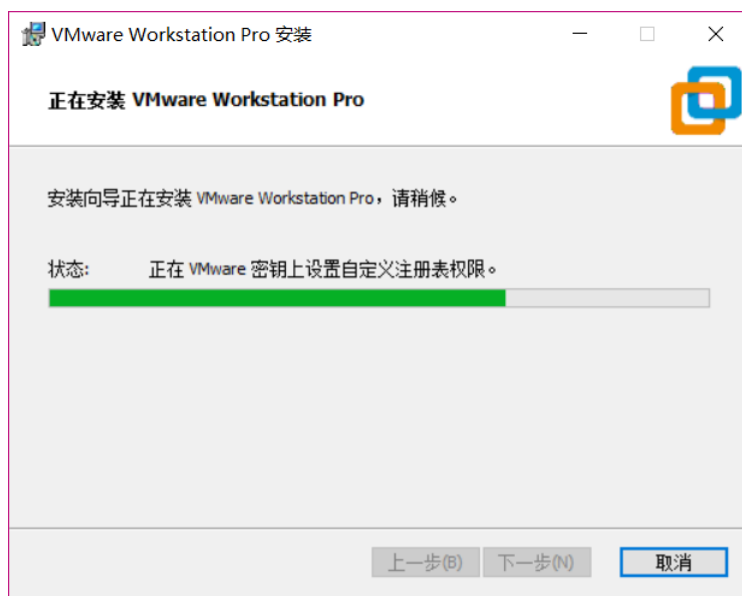


图 1.1.8 安装进行中

图 1.1.8 就是安装过程，耐心等待几分钟，等待安装完成，安装完成以后会有如图 1.1.9 所示提示：

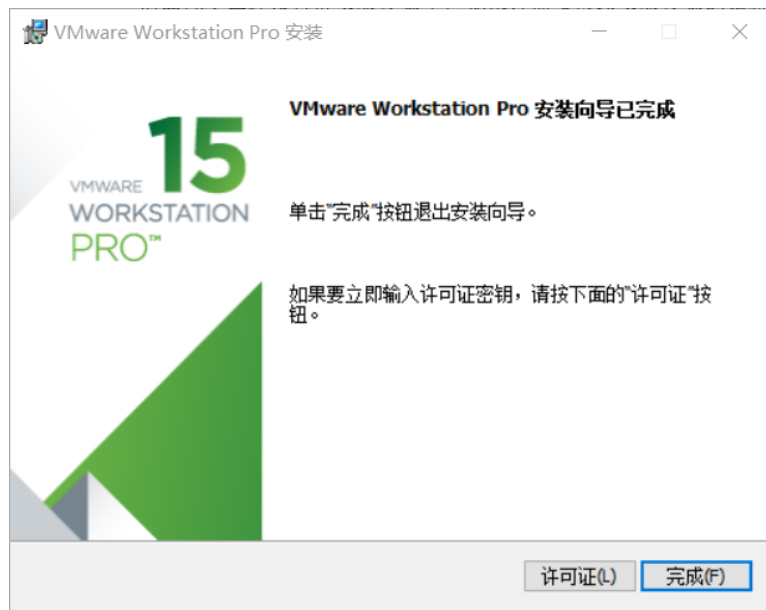


图 1.1.9 安装完成

点击图 1.1.9 中的“完成”按钮，完成 VMware 的安装，安装完成以后就会在桌面上出现 VMware Workstation Pro 的图标，如图 1.1.10 所示：

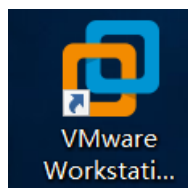


图 1.1.10 VMware 桌面图标

双击图 1.1.10 中的图标打开 VMware 软件，在第一次打开软件的时候会提示你输入许可证密钥，如图 1.1.11 所示：

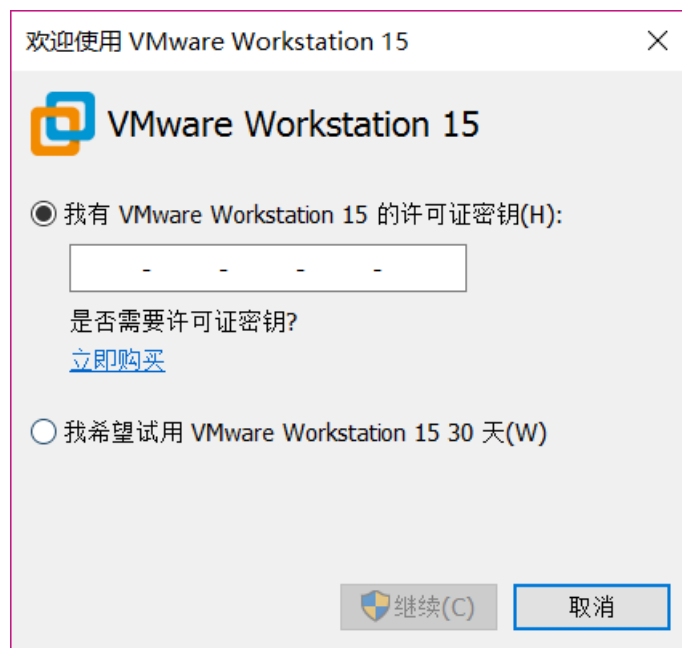


图 1.1.11 输入许可证密钥

前面说了 VMware 是付费软件，是需要购买的，如果你购买了 VMware 的话就会有一串许可密钥，如果没有购买的话就选择“我希望试用 VMware Workstation 15 30 天”选项，这样你就可以体验 30 天 VMware。输入密钥以后点击“继续按钮”，如果你的密钥正确的话就会提示你购买成功，如图 1.1.12 所示：

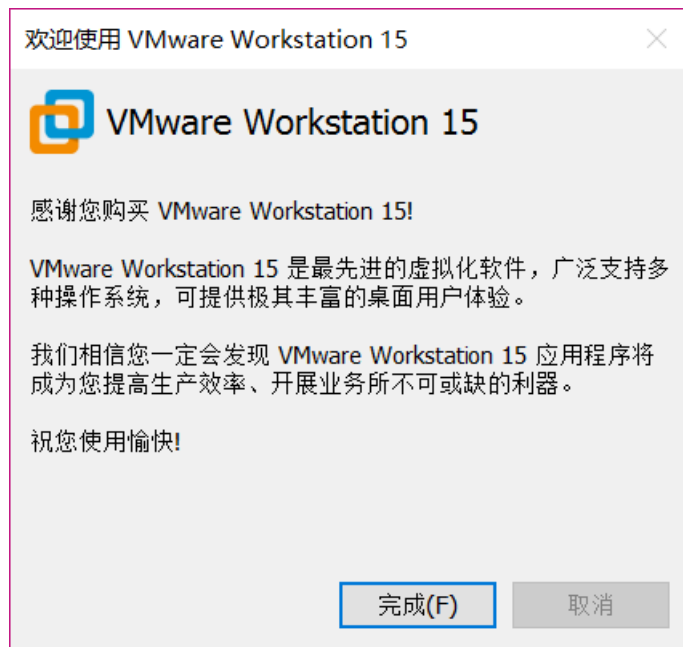


图 1.1.12 购买 VMware 成功

点击图 1.1.12 中的“完成”按钮，VMware 软件正式打开，界面如图 1.1.13 所示：

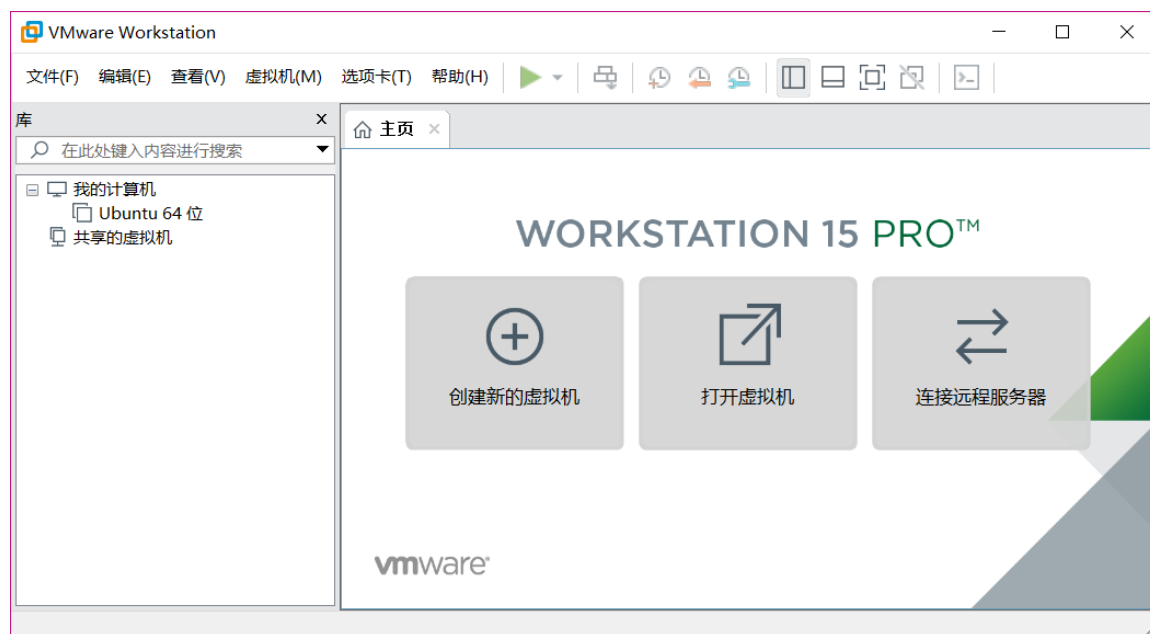


图 1.1.13 VMware Workstation 主界面

至此，虚拟机软件 VMware 安装成功。

1.2 创建虚拟机

安装好 VMware 以后我们就可以在 VMware 上创建一个虚拟机，打开 VMware，选择：文件->新建虚拟机，如图 1.2.1 所示：

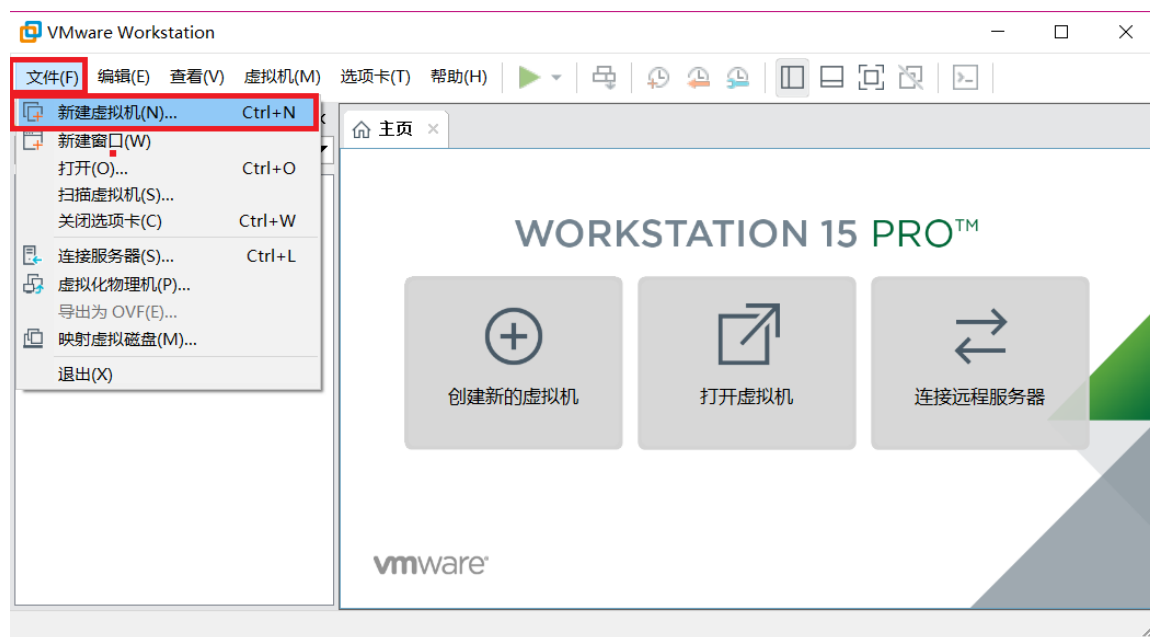


图 1.2.1 新建虚拟机

打开图 1.2.2 所示创建虚拟机向导界面：

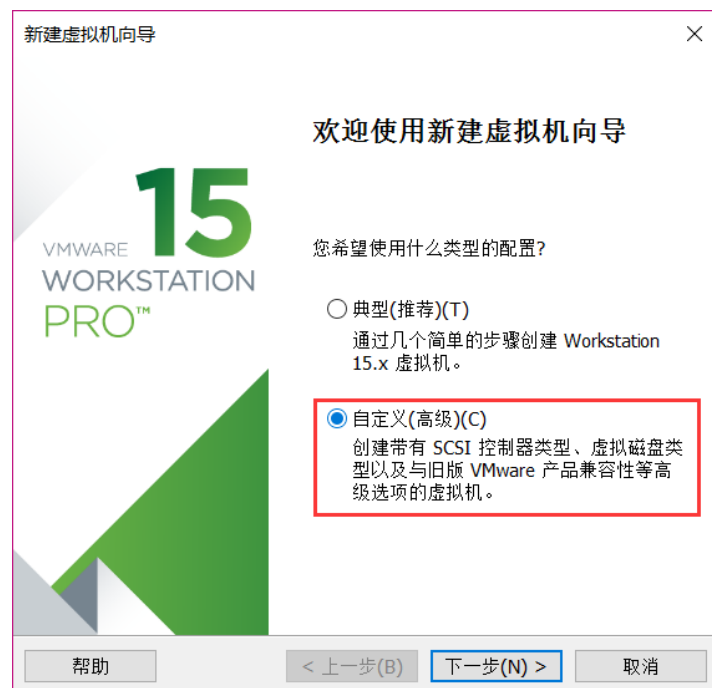


图 1.2.2 创建虚拟机向导

选中图 1.2.2 中的“自定义”选项，然后选择“下一步”，进入图 1.2.3 所示硬件兼容性选择界面：

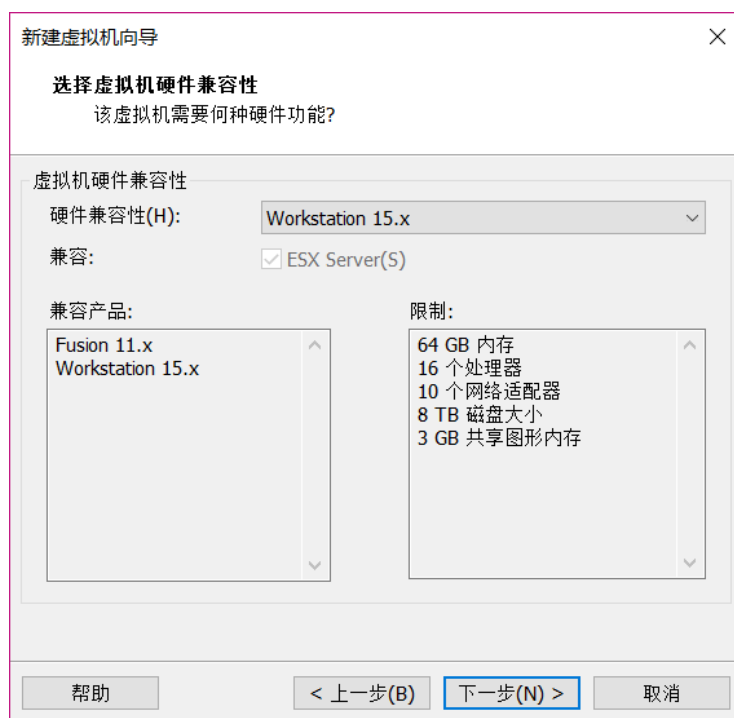


图 1.2.3 硬件兼容性选择

在图 1.2.3 中我们使用默认值就行了，直接点击“下一步”，进入图 1.2.4 所示的操作系统安装界面：

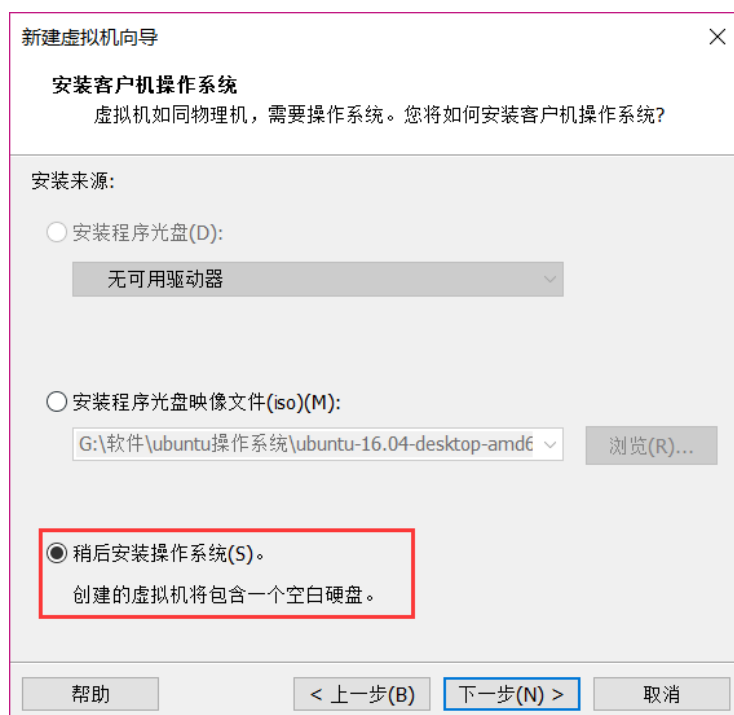


图 1.2.4 安装客户机操作系统

图 1.2.4 就是选择你新创建的虚拟机要安装什么系统？windos 还是 linux，如果你要现在就安装系统的话需要准备好系统文件，一般是.iso 文件。我们现在不安装系统，因此选择“稍后安装操作系统(S)”这个选项，然后选择“下一步”，进入图 1.2.5 所示界面：

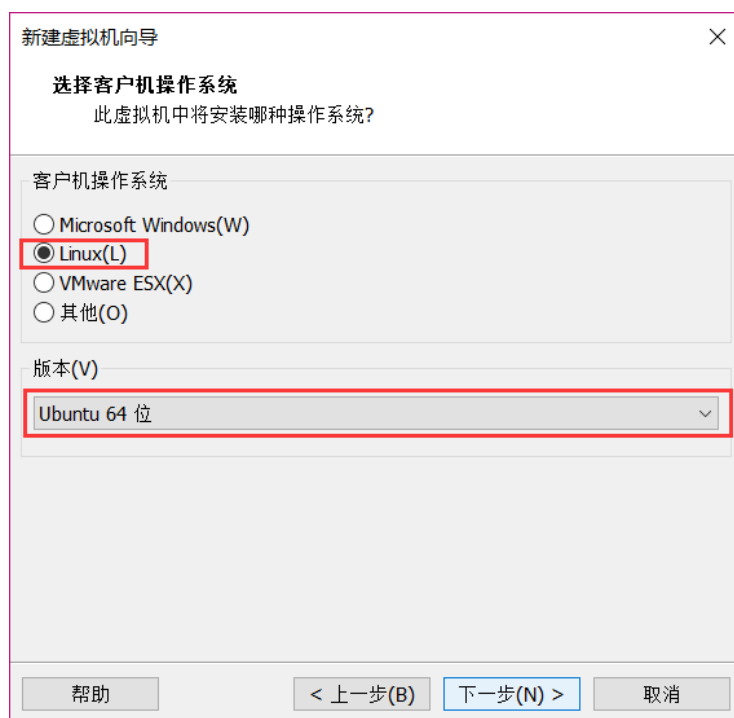


图 1.2.5 客户机操作系统选择

图 1.2.5 中依旧是让你选择你要在虚拟机中装什么系统，图 1.2.5 是和图 1.2.4 配合在一起使用的，在图 1.2.4 中放入系统文件(.iso 文件)，然后在图 1.2.5 中选择你图 1.2.4 中放入的是什么系统，然后 VMware 就会稍后自动安装所设置的系统。在图 1.2.4 中我们没有设置系统文件，因此图 1.2.5 是没用的，不过我们还是在图 1.2.5 中的客户机操作系统一栏选择“Linux”，版本选择 Ubuntu 64 位，然后点击“下一步”，进入图 1.2.6 所示界面：

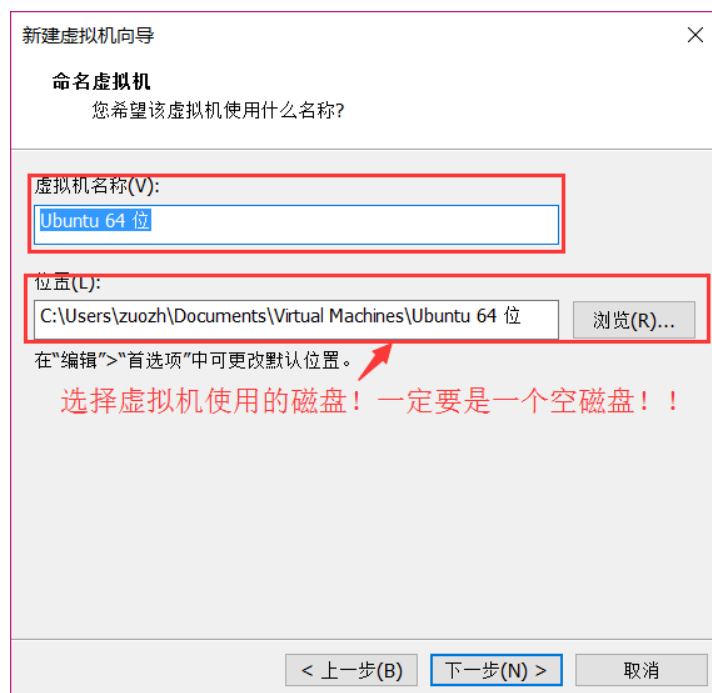


图 1.2.6 命名虚拟机

在图 1.2.6 中上面是命名虚拟机名字，大家可以根据自己的使用习惯给虚拟机命名，重点是

下面的虚拟机位置选择！我们要给虚拟机单独清理出一块磁盘，做嵌入式开发建议这块空磁盘的大小不小于 100GB，比如我清理除了一个 196GB 的 I 盘给虚拟机使用，如图 1.2.7 所示：

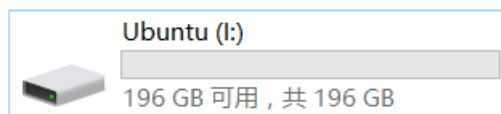


图 1.2.7 虚拟机所使用的磁盘

清理出虚拟机专用的磁盘以后然后就在图 1.2.6 中的位置出选择这个磁盘，比如我的位置选择如图 1.2.8 所示：

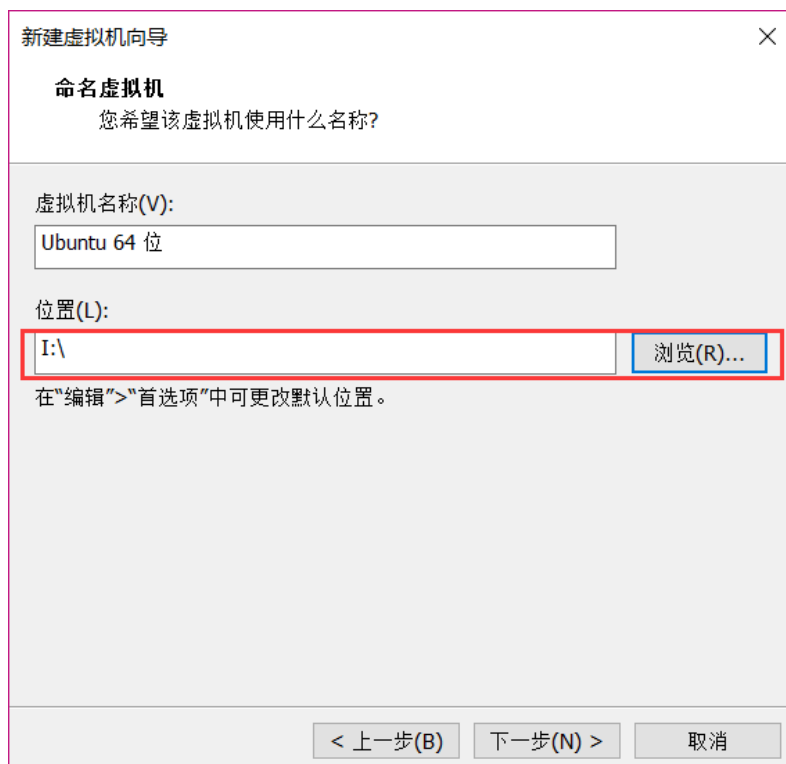


图 1.2.8 选择虚拟机磁盘位置

设置好图 1.2.8 中的虚拟机磁盘位置以后点击“下一步”，进入图 1.2.9 所示的处理器配置选择界面：

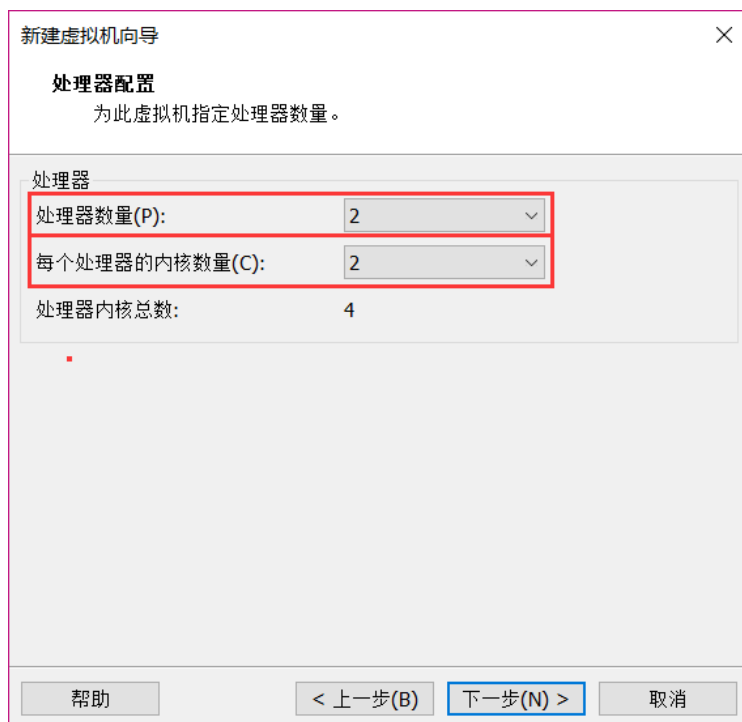


图 1.2.9 处理器配置界面

图 1.2.9 中就是配置你的虚拟机所使用的处理器数量，以及每个处理器的内核数量，这个要根据自己实际使用的电脑 CPU 配置来设置。比如我的电脑 CPU 是 I7-4720HQ，这是个 4 核 8 线程的 CPU，因此我就可以分 2 个核给 VMware，然后 I7-4720HQ 每个物理核有两个逻辑核，因此每个处理器的内核数量就是 2，所以的 VMware 虚拟机配置就如图 1.2.9 所示，大家根据自己的实际电脑 CPU 配置来设置即可，设置好以后点击“下一步”，进入图 1.2.10 所示内存配置界面：

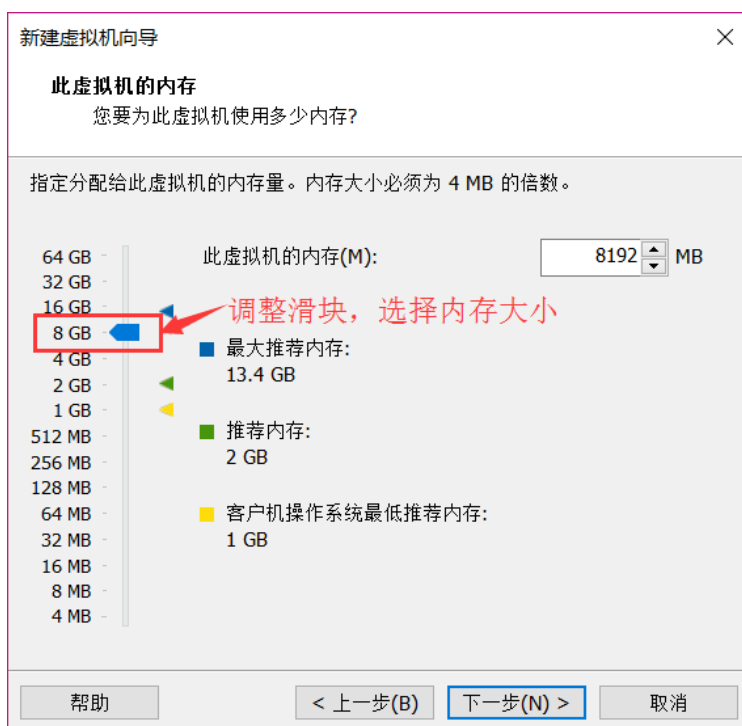


图 1.2.10 内存配置

同样的在图 1.2.10 中根据自己电脑的实际内存配置来设置分给虚拟机的内存大小，比如我的电脑是 16GB 的内存，因此我可以给虚拟机分配 8GB 的内存。配置好虚拟机的内存大小以后点击“下一步”，进入图 1.2.11 所示的网络类型选择界面：

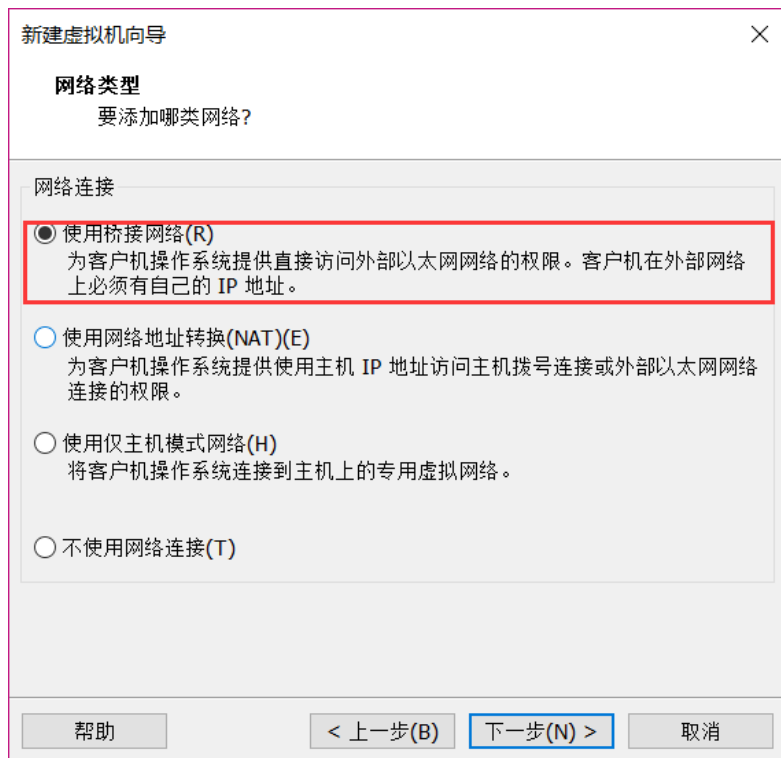


图 1.2.11 网络类型选择界面

在图 1.2.11 中我们选择“使用桥接网络”，然后点击“下一步”，进入图 1.2.12 所示的选择 I/O 控制器类型界面：

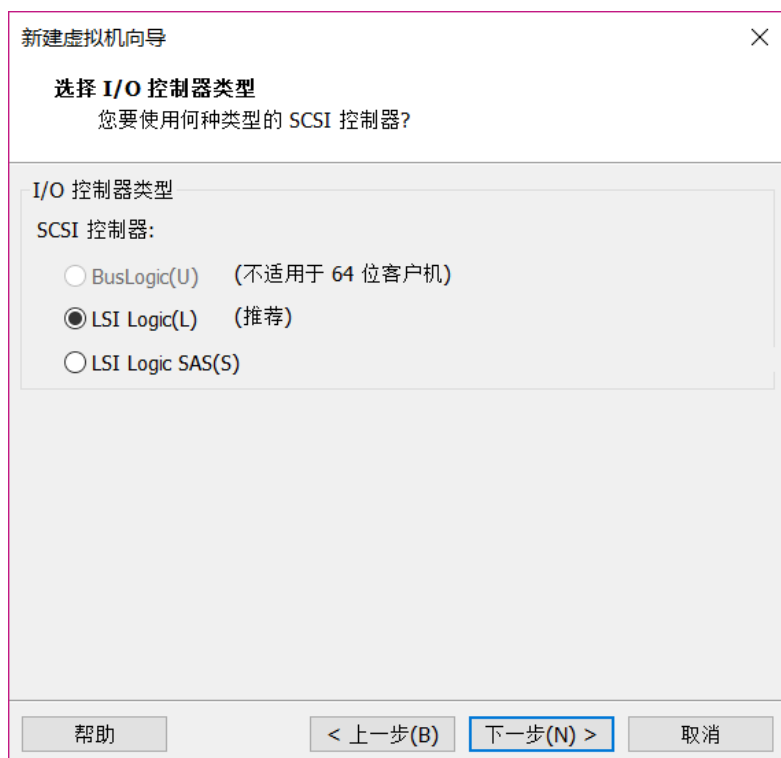


图 1.2.12 I/O 控制器选择

I/O 控制器类型选择默认值就行，也就是“LSILogic”，然后点击“下一步”，进入磁盘类型选择界面，如图 1.2.13 所示：



图 1.2.13

图 1.2.13 中选择磁盘类型，使用默认值“SCSI”即可，然后点击“下一步”，进入选择磁盘

界面，如图 1.2.14 所示：

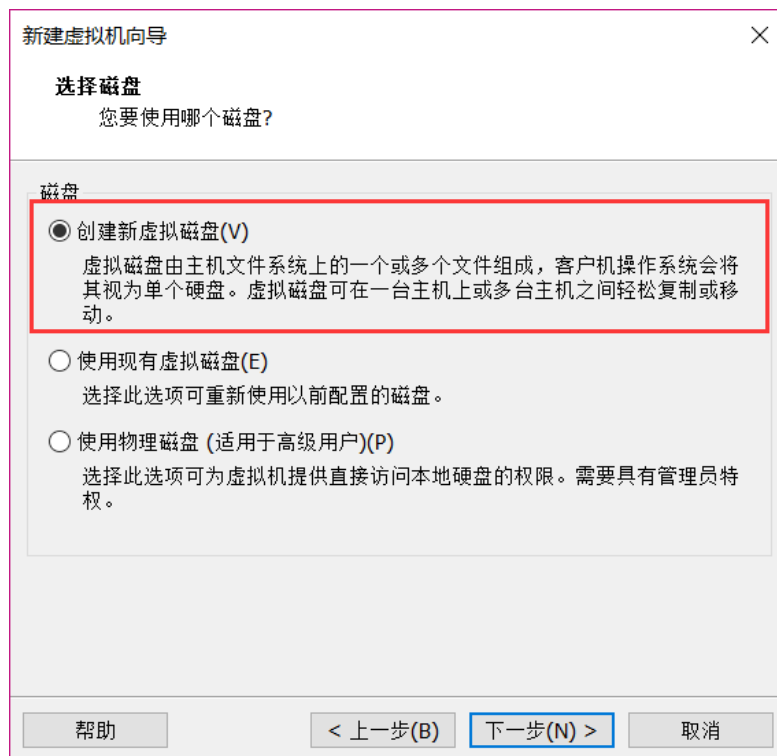


图 1.2.14 磁盘选择

图 1.2.14 中使用默认值，即“创建新虚拟磁盘”，这样我们前面设置好的那个空的磁盘就会被创建为一个新的磁盘，设置要以后点击“下一步”，进入磁盘容量设置界面。**注意，磁盘空间尽量大一点，不要设置成建议的 20GB，最好 50GB 以上，否则开发过程中很容易提示磁盘空间不够，比如我这里设置为 196GB，如图 1.2.15 所示：**

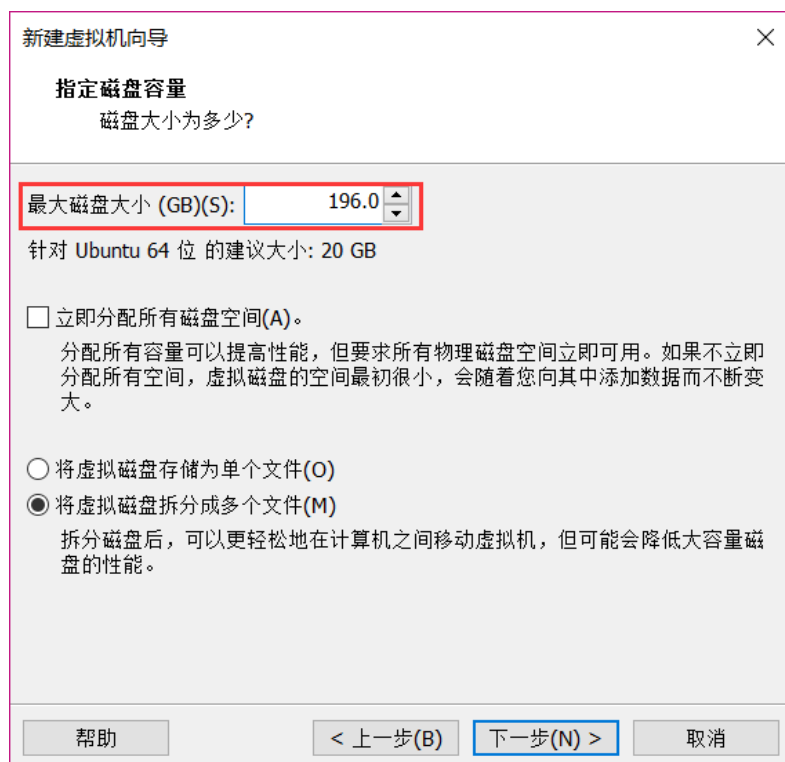


图 1.2.15 磁盘容量设置

图 1.2.15 是用来设置我们清出的空的磁盘多少是给虚拟机用的，我们清出了一个空磁盘肯定是全部给虚拟机用的，因此设置最大磁盘大小为空磁盘的大小，比如图 1.2.7 中我的那个 I 盘是 196GB 的，因此图 1.2.15 中就设置最大磁盘大小为 196GB，然后点击“下一步”，进入图 1.2.16 所示界面指定磁盘文件，

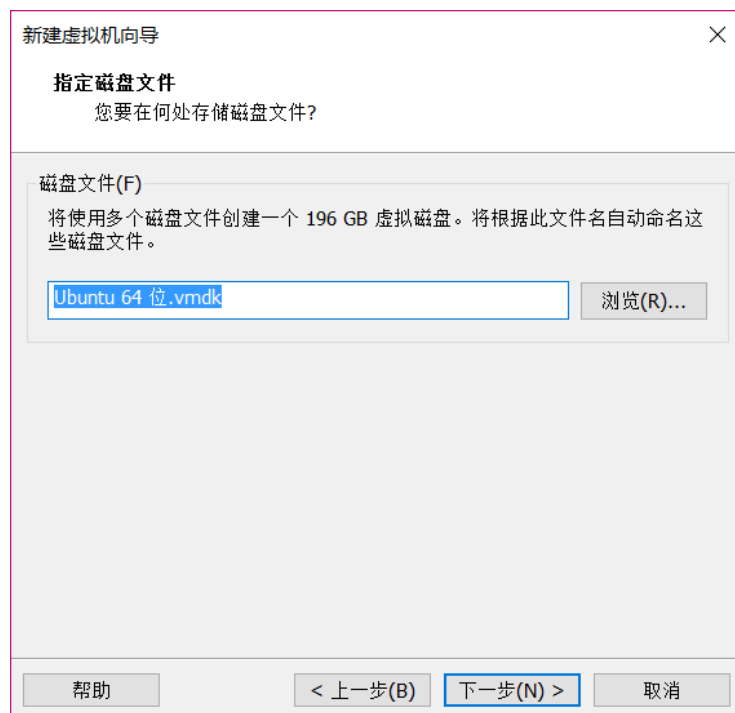


图 1.2.16 指定磁盘文件

图 1.2.16 使用默认设置, 不要做任何修改, 直接点击“下一步”, 进入已准备好创建虚拟机界面, 如图 1.2.17 所示:

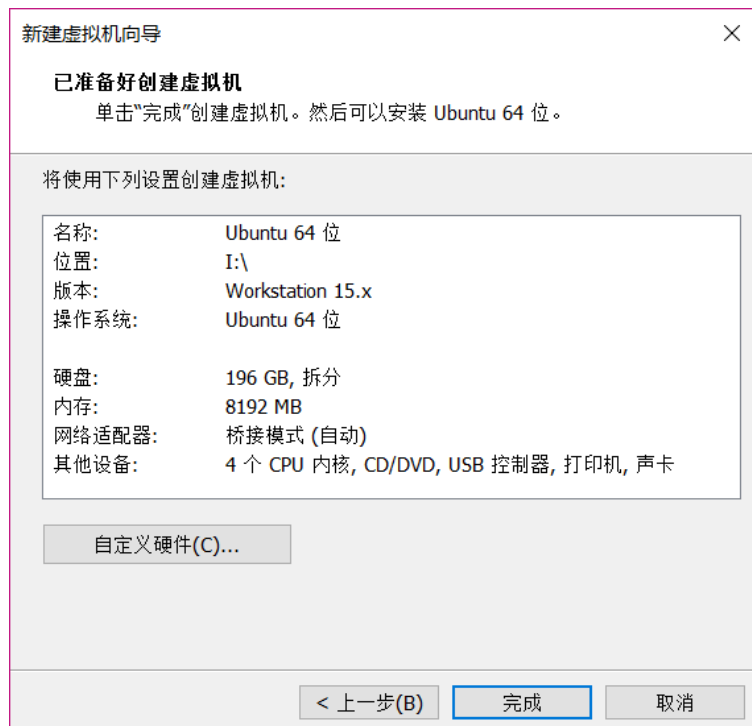


图 1.2.17 准备创建虚拟机

在图 1.2.17 中确认自己的虚拟机配置, 如果确认无误就点击“完成”, 如果有误的话就返回有误的配置界面做修改, 点击“完成”按钮以后就会创建一个虚拟机, 如图 1.2.17 所示:

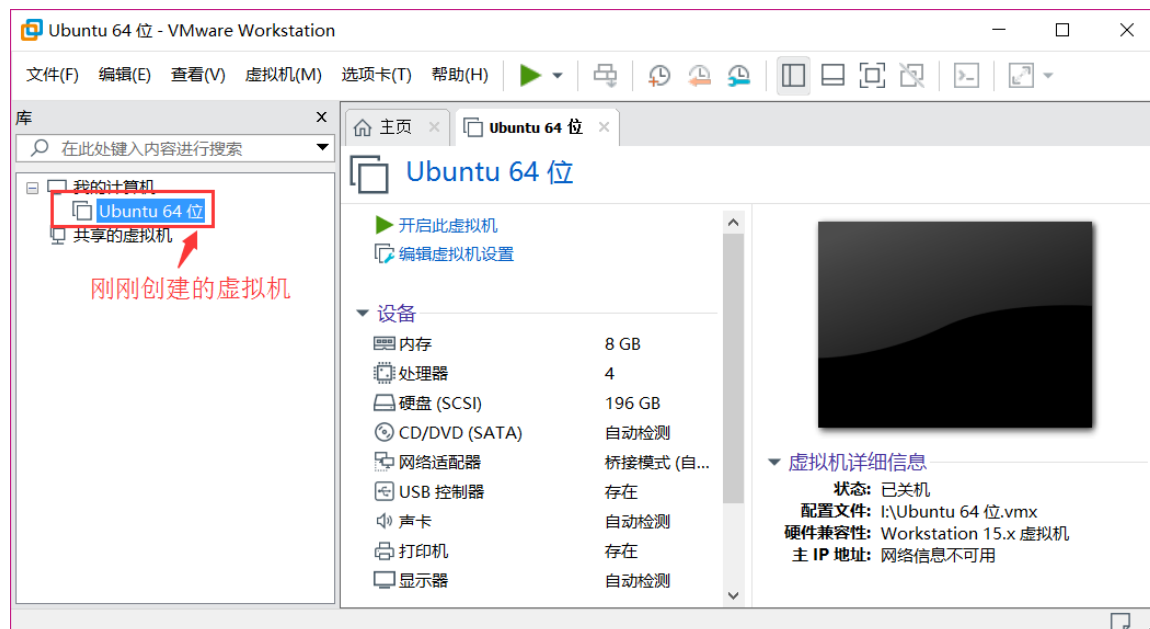


图 1.2.17 新创建的虚拟机

创建虚拟机成功以后就会在右侧的：我的计算机下出现刚刚创建的虚拟机“Ubuntu 64 位”，点击一下就会在右侧打开这个虚拟机的详细信息，如图 1.2.18 所示：



图 1.2.18 新建虚拟机配置信息

在图 1.2.18 中的设备一栏我们可以看到虚拟机详细的配置信息，图 1.2.19 所示的两个按钮就是虚拟机的开关，



图 1.2.19 虚拟机开关

图 1.2.19 中的这两个绿色三角按钮都可以打开虚拟机，但是此时虚拟机没有安装任何操作系统，因此没法打开，接下来我们就是要在刚刚新建的这个虚拟机中安装 Ubuntu 操作系统。

1.3 安装 Ubuntu 操作系统

1.3.1 获取 Ubuntu 系统

前面虚拟机已经创建成功了，相当于硬件已经准备好了，接下来就是要在虚拟机中安装

Ubuntu 系统了, 首先肯定是获取到 Ubuntu 的系统镜像, Ubuntu 系统镜像肯定是在 Ubuntu 官网获取, 下载地址为: <https://www.ubuntu.com/download/desktop>, 如图 1.3.1.1 所示:

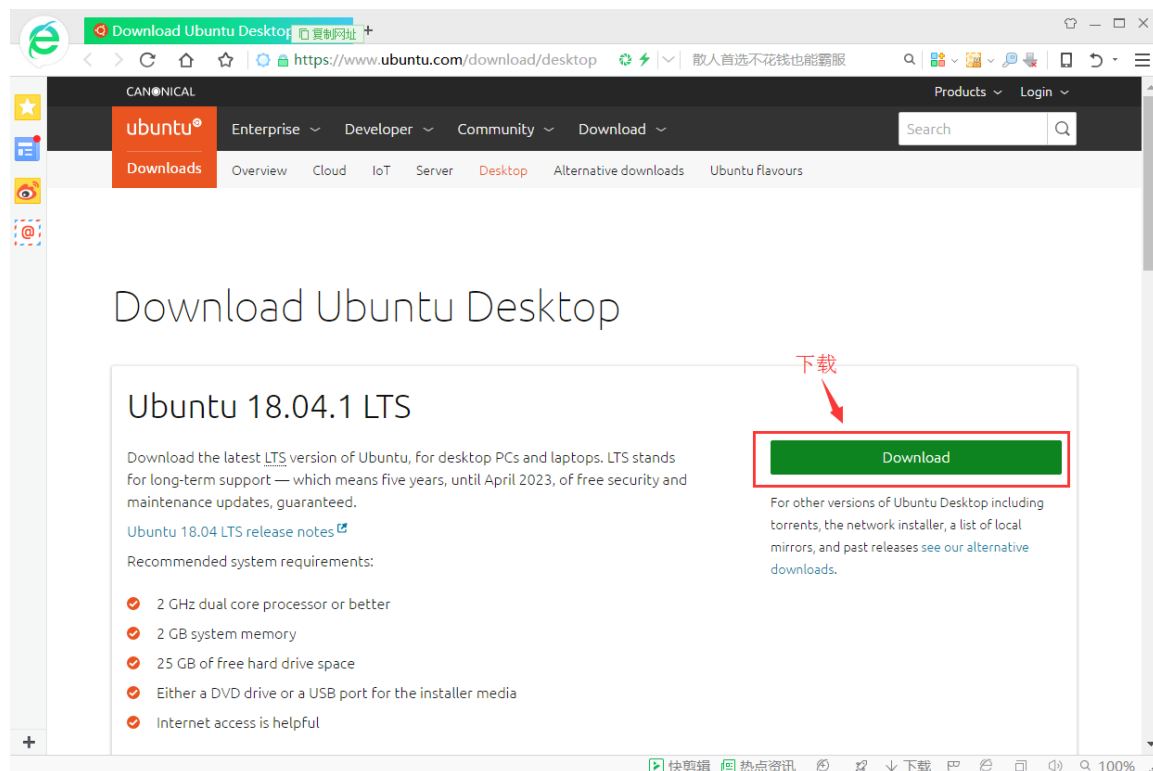


图 1.3.1.1 Ubuntu 最新版系统下载

从图 1.3.1.1 中可以看出, 最新版本的 Ubuntu 系统是 18.04, 但是在笔者编写本教程的时候 18.04 版刚出来没多久, 怕不稳定, 因此笔者实际使用的是 16.04 版本的 Ubuntu, 后面所有的例程和教程均在 16.04 下完成, 包括我们接下来安装的也是 16.04 版本的 Ubuntu。16.04 版本的 Ubuntu 下载地址为: <http://releases.ubuntu.com/16.04/>, 下载“ubuntu-16.04.5-desktop-amd64.iso”这个版本, 我已经下载下来放到了开发板光盘中, 路径为: 3、软件-> ubuntu-16.04.5-desktop-amd64.iso。

1.3.2 安装 Ubuntu 操作系统

Ubuntu 系统获取到以后就可以安装了, 打开 VMware 软件, 选择: 虚拟机->设置, 如图 1.3.2.1 所示:

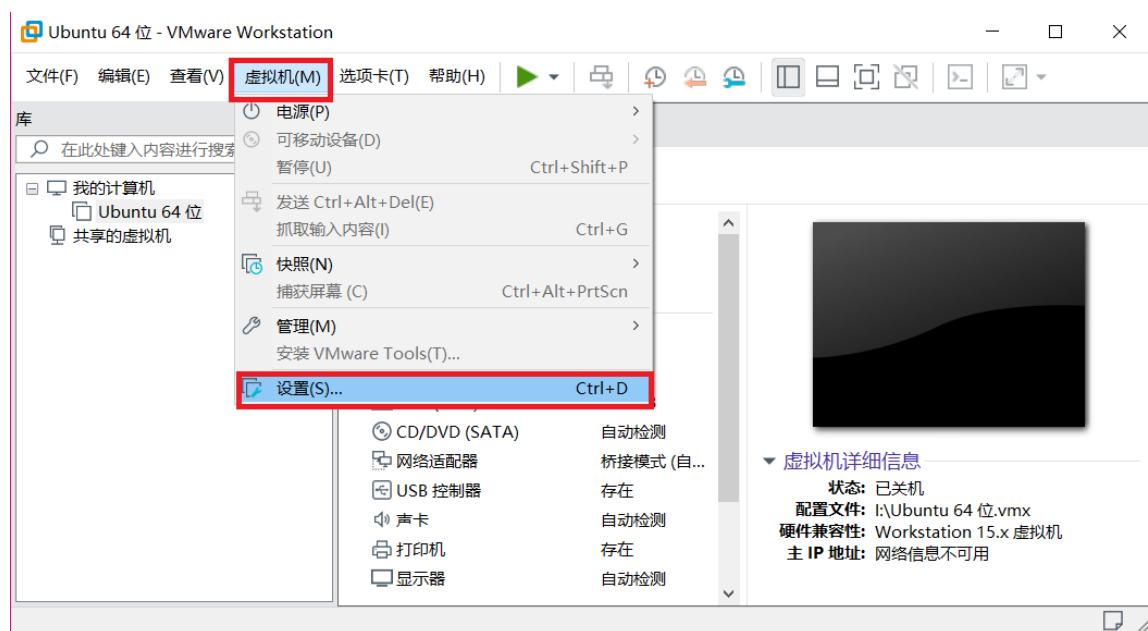


图 1.3.2.1 打开虚拟机设置对话框

打开以后的虚拟机设置对话框如图 1.3.2.2 所示:



图 1.3.2.2 虚拟机对话框

首先设置“USB 控制器”选项，默认 USB 控制器的 USB 兼容性为 USB2.0，这样当你使用 USB3.0 的设备的时候 Ubuntu 可能识别不出来，因此我们需要调整 USB 兼容性为 USB3.0，如图 1.3.2.3 所示：



图 1.3.2.3 USB 兼容性设置

设置要 USB 兼容性以后就开始安装 Ubuntu 系统了，选中虚拟机设置对话框中的“CD/DVD(SATA)”选项，然后在右侧选中“使用 ISO 映像文件”，如图 1.3.2.4 所示：

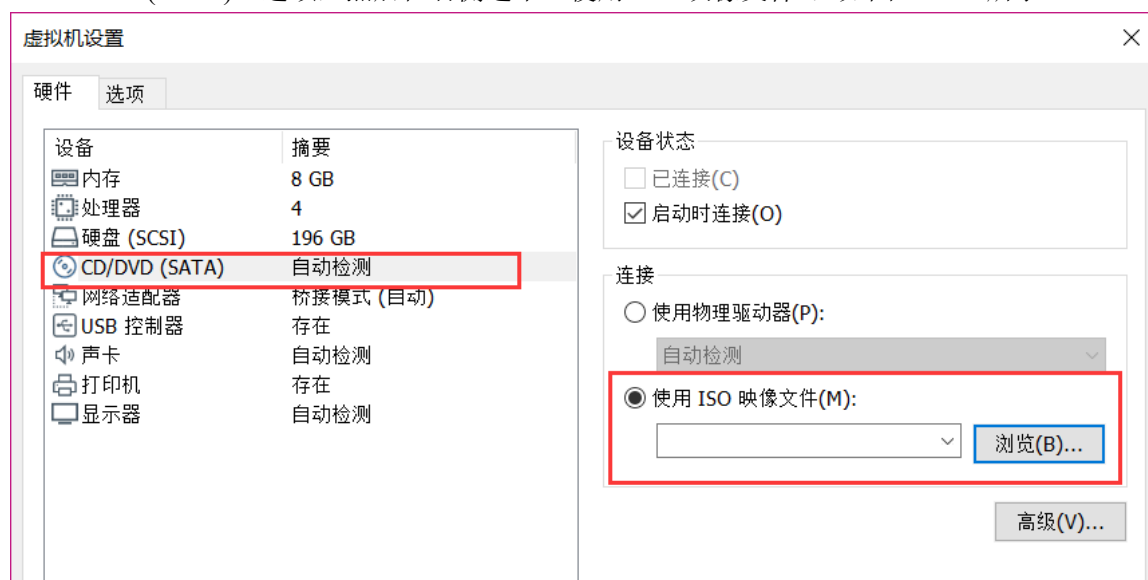


图 1.3.2.4 系统镜像设置

在图 1.3.2.4 中的“使用 ISO 映像文件”里面添加我们刚刚下载到的 Ubuntu 系统镜像，点击“浏览”按钮，选择 Ubuntu 系统镜像，完成以后如图 1.3.2.5 所示：

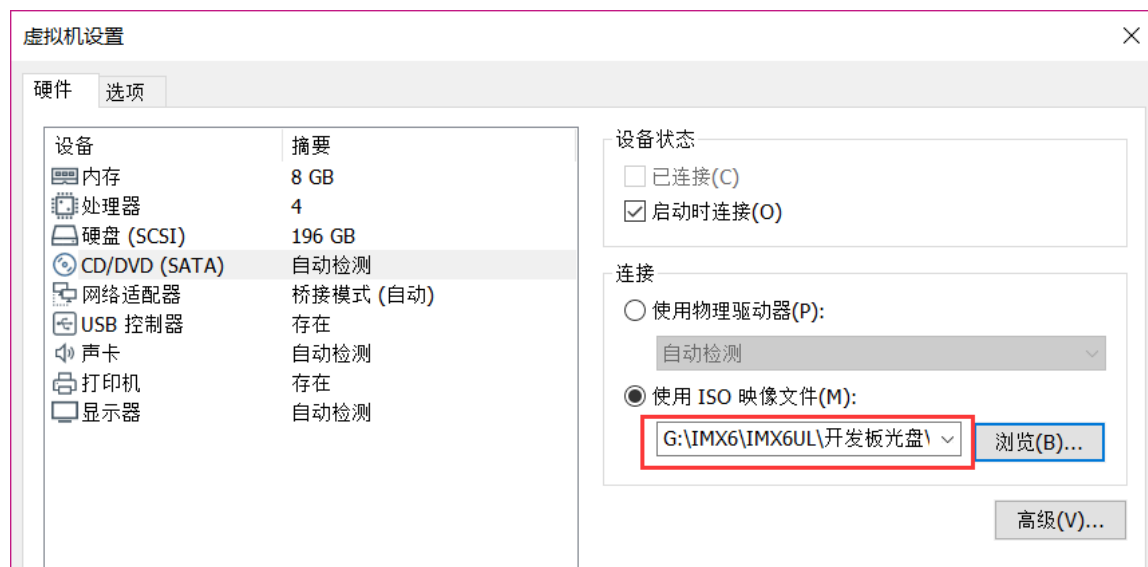


图 1.3.2.5 Ubuntu 镜像选择

设置好以后点击“确定”按钮退出，退出以后就可以打开虚拟机了，虚拟机就会自动的安装 Ubuntu 系统，如图 1.3.2.6 所示：

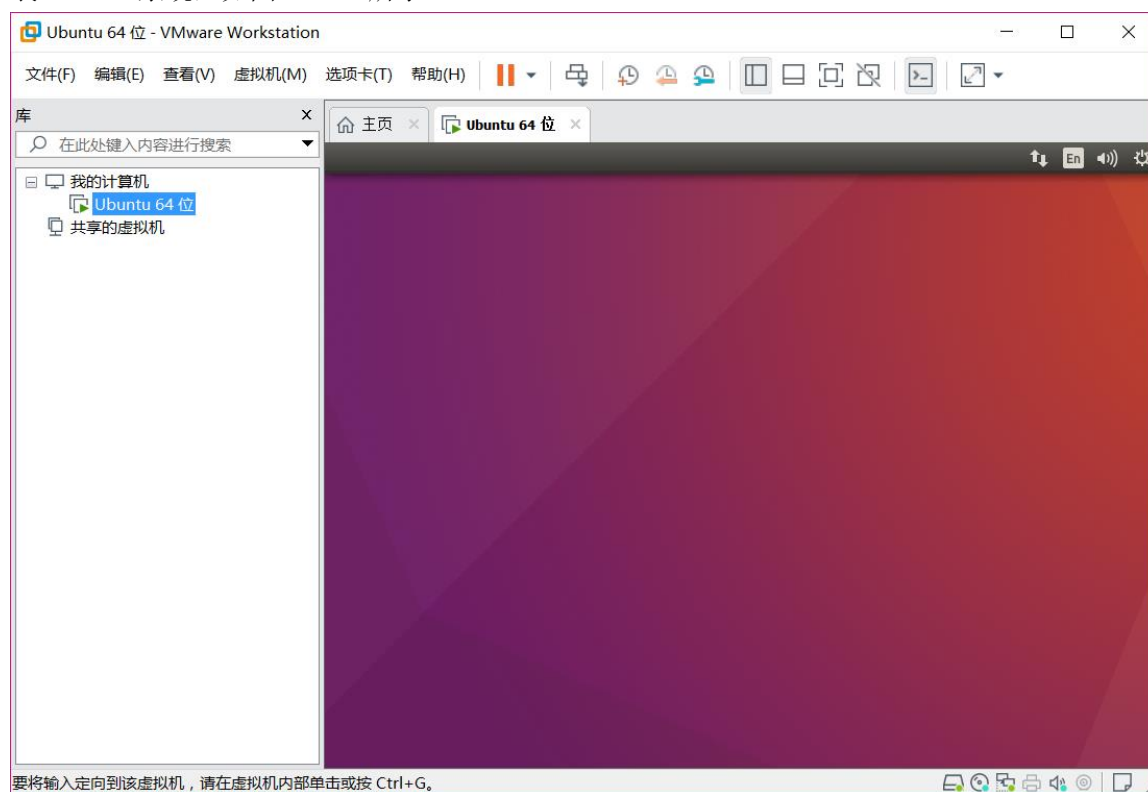


图 1.3.2.6 Ubuntu 安装开始

Ubuntu 开始安装以后首先是语言选择，如图 1.3.2.7 所示：



图 1.3.2.7 语言选择

Ubuntu 默认语言是英文，毫无疑问，我们要选择“中文(简体)”，选择好以后点击右侧的“安装 Ubuntu”按钮，进入安装过程。安装一开始会有 7 个配置步骤，第一配置如图 1.3.2.8 所示，让你选择是否安装 Ubuntu 时下载更新，以及是否为图形或者无线硬件安装其它第三方软件，我们不勾选这两个，否则安装过程很慢。



图 1.3.2.8 是否安装是下载更新

直接点击图 1.3.2.8 中的“继续”按钮，弹出安装类型，使用默认的“清除整个磁盘并安装 Ubuntu”，如图 1.3.2.9 所示：



图 1.3.2.9 安装类型选择

设置好安装类型以后点击“现在安装”按钮，会弹出“将改动写入磁盘吗？”对话框，点击“继续”即可，下一步会让你输入你在哪个位置，输入自己所在的城市即可，比如我在广州就输入广州，如图 1.3.2.10 所示：

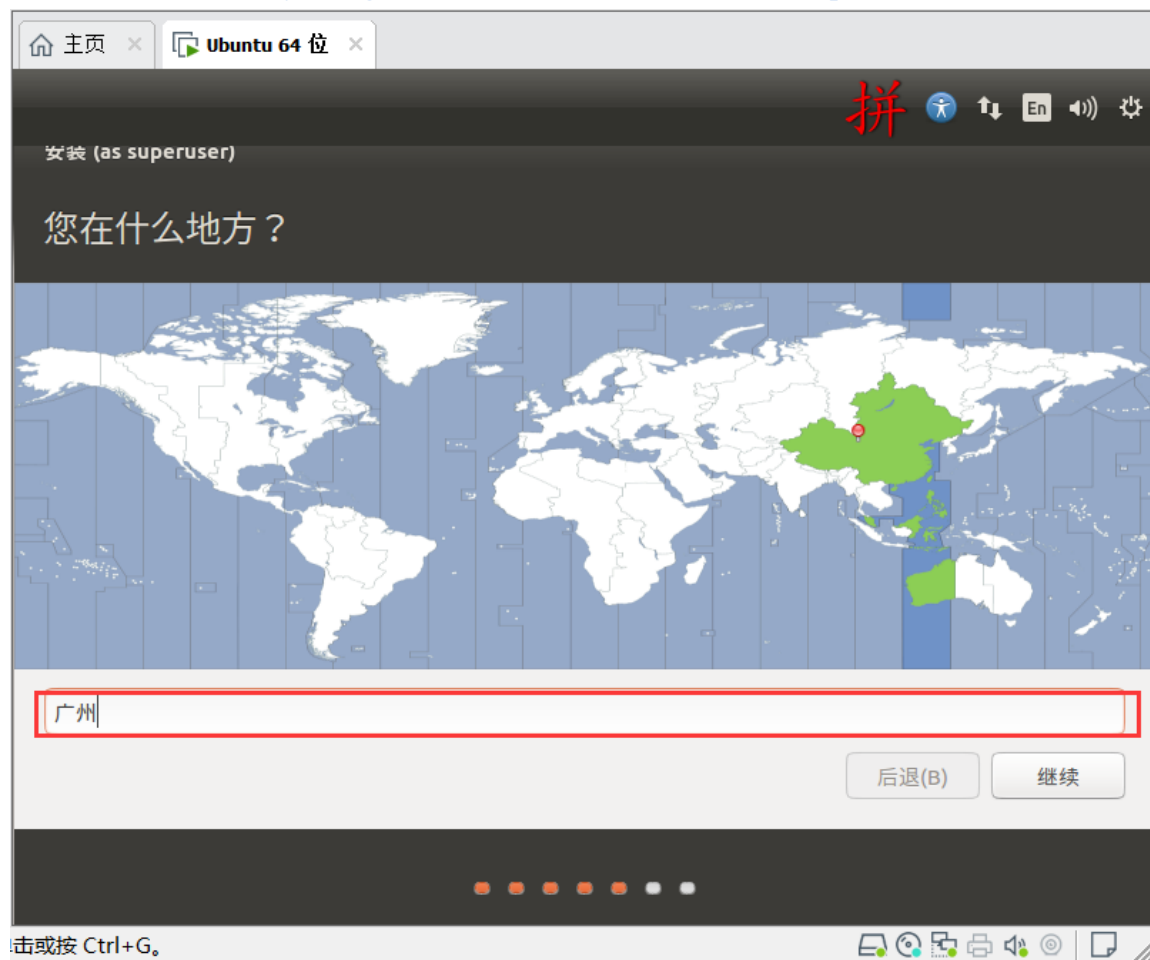


图 1.3.2.10 输入所在位置

Ubuntu 默认带了拼音输入法，切换到拼音输入法的方法如图 1.3.2.11 所示：

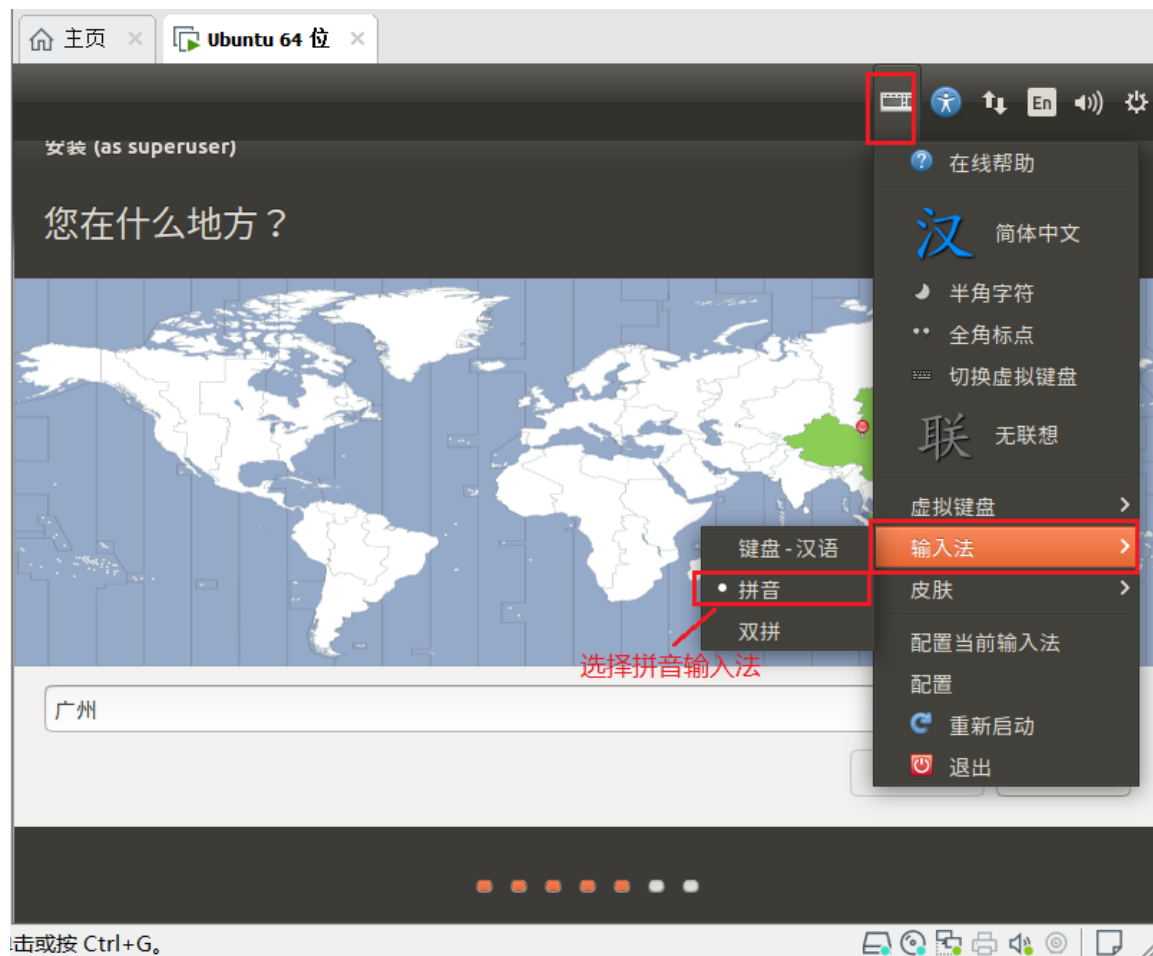


图 1.3.2.11 切换拼音输入法

输入地址以后点击“继续”按钮，会进入键盘布局设置界面，不需要做任何修改，点击“继续”按钮，进入下一步设置用户名和密码，自己设置自己的用户名和密码，比如我的设置如图 1.3.2.12 所示：



图 1.3.2.12 设置用户名和密码

设置好用户名和密码以后点击“继续”按钮，系统就会开始正式安装，如图 1.3.2.13 所示：



图 1.3.2.13 系统安装中

等待系统安装完成，安装过程中会下载一些文件，所以一定要保证电脑能够正常上网，如果不能正常上网的话可以点击右侧的“skip”按钮来跳过下载文件这个步骤，对于系统的安装没有任何影响，安装完成以后提示重启系统，如图 1.3.2.14 所示：

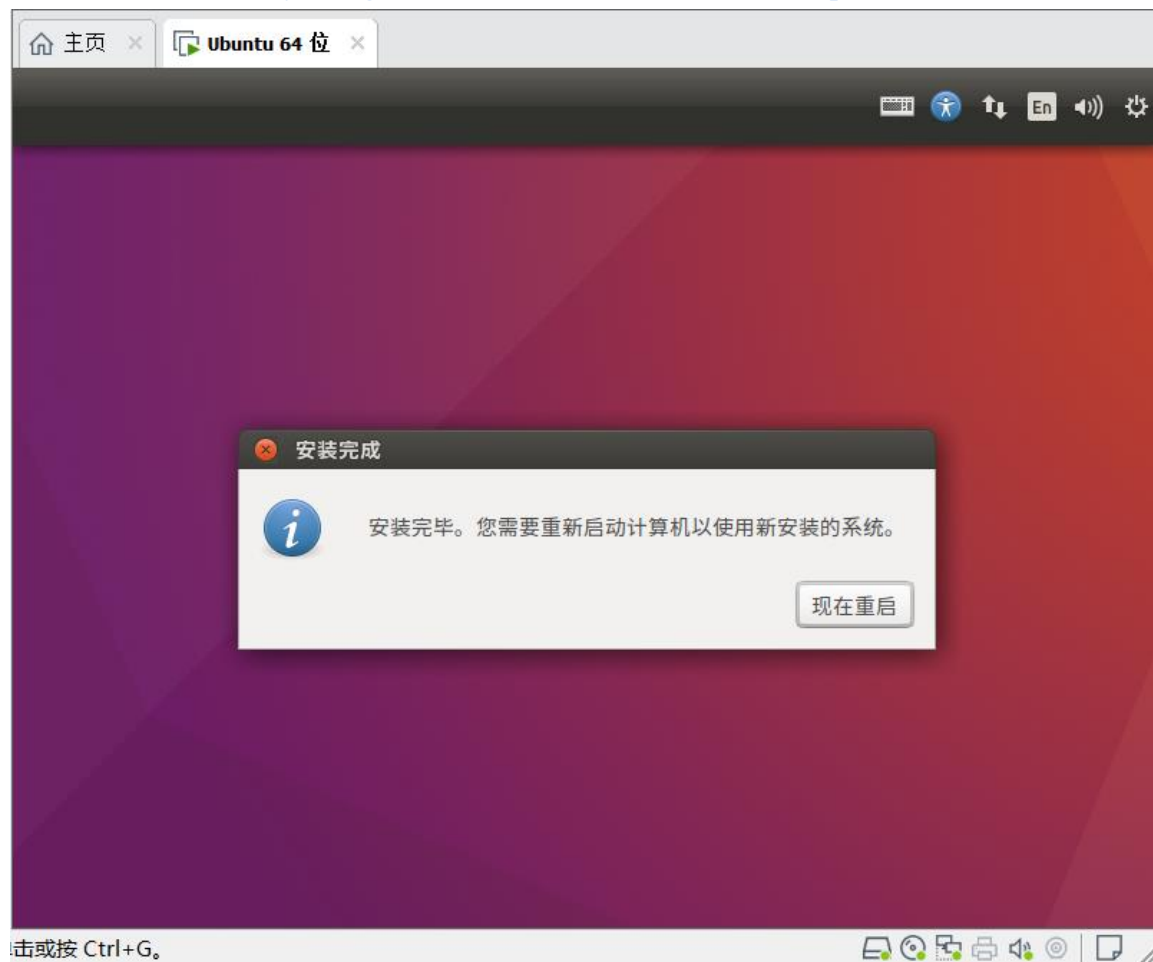


图 1.3.2.14 安装完成，重启系统

重启系统以后就会提示输入密码，如图 1.3.2.15 所示：

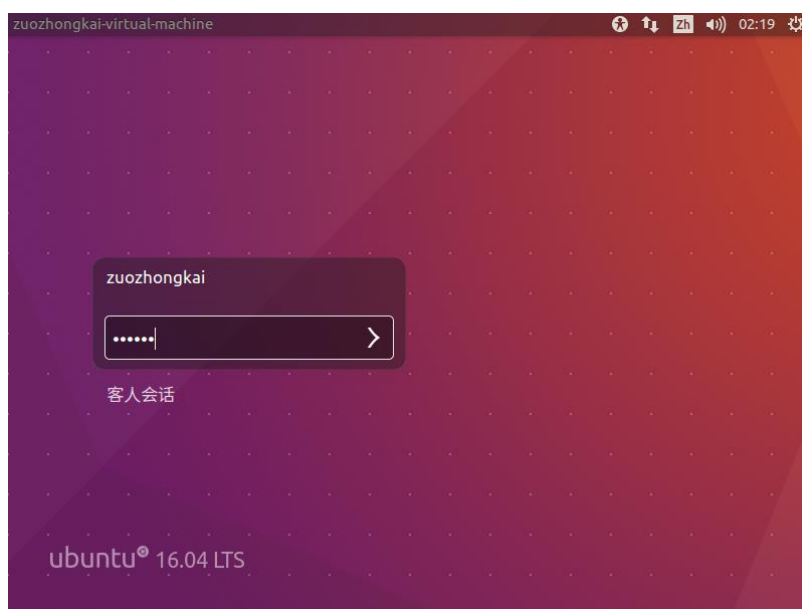


图 1.3.2.15 系统重启，输入密码

在图 1.3.2.15 中输入密码，点击键盘上的回车就会进入系统主界面，系统界面如图 1.3.2.16 所示：

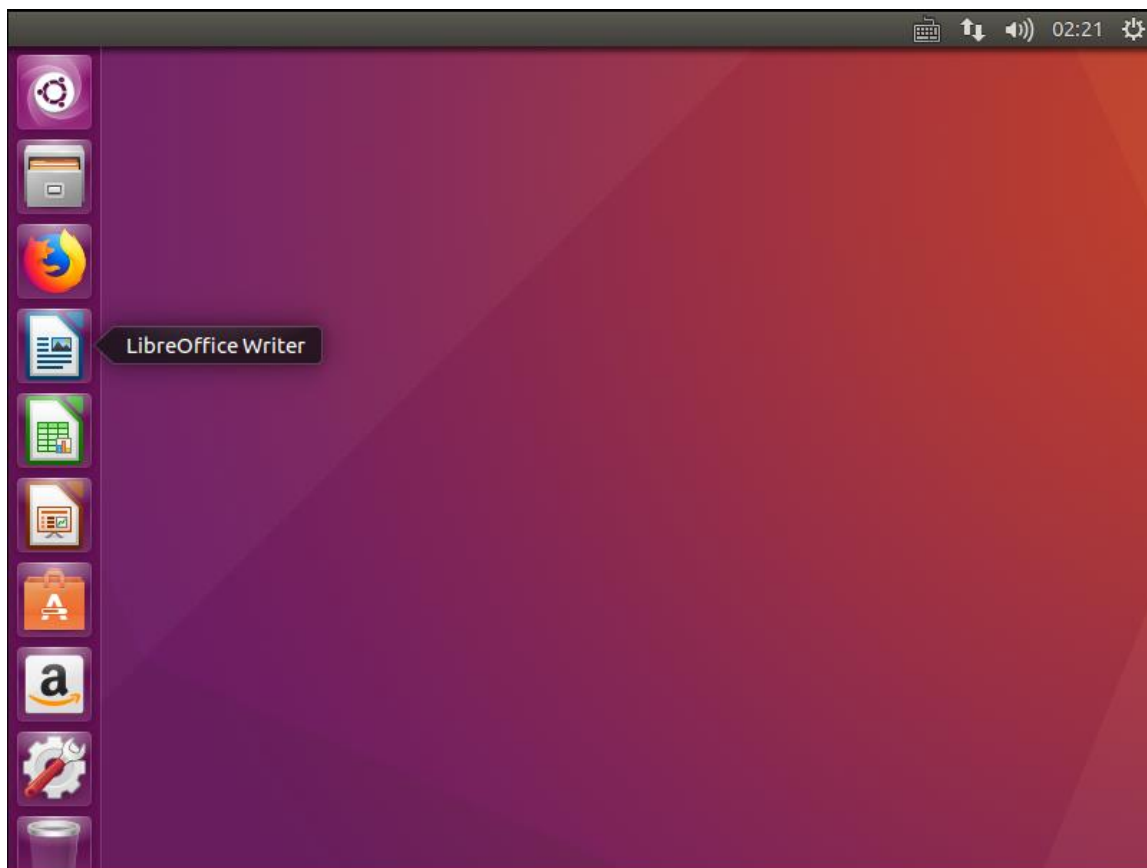


图 1.3.2.16 系统桌面

图 1.3.2.16 就是第一次进入系统的系统桌面，此时我们的系统镜像还在 CD/DVD 里面，我们要将它弹出，先关闭 Ubuntu 系统，点击系统桌面右上角的设置按钮，如图 1.3.2.17 所示：



图 1.3.2.17 关机

按照图 1.3.2.17 所示方式关机。

1.3.3 弹出系统镜像

和我们在真实电脑上安装系统一样，不管我们使用的光盘还是 U 盘安装系统，当系统安装成功以后都要弹出光盘或者拔出 U 盘，然后调整 BIOS 从硬盘启动，否则以后开机的话都会首

先从光盘或者 U 盘启动了，这样会进入系统安装界面。同理，我们在 VMware 中安装 Ubuntu 的时候是在 CD/DVD 中加载了 Ubuntu 系统镜像，现在系统安装成功了，因此也要把这个镜像从 CD/DVD 中弹出。

关闭 Ubuntu 操作系统，重新打开 VMware，不要打开 Ubuntu 系统！打开 VMware 的虚拟机设置界面，然后选中“CD/DVD(SATA)”，右侧的“连接”选择“使用物理驱动器”，如图 1.3.3.1 所示。

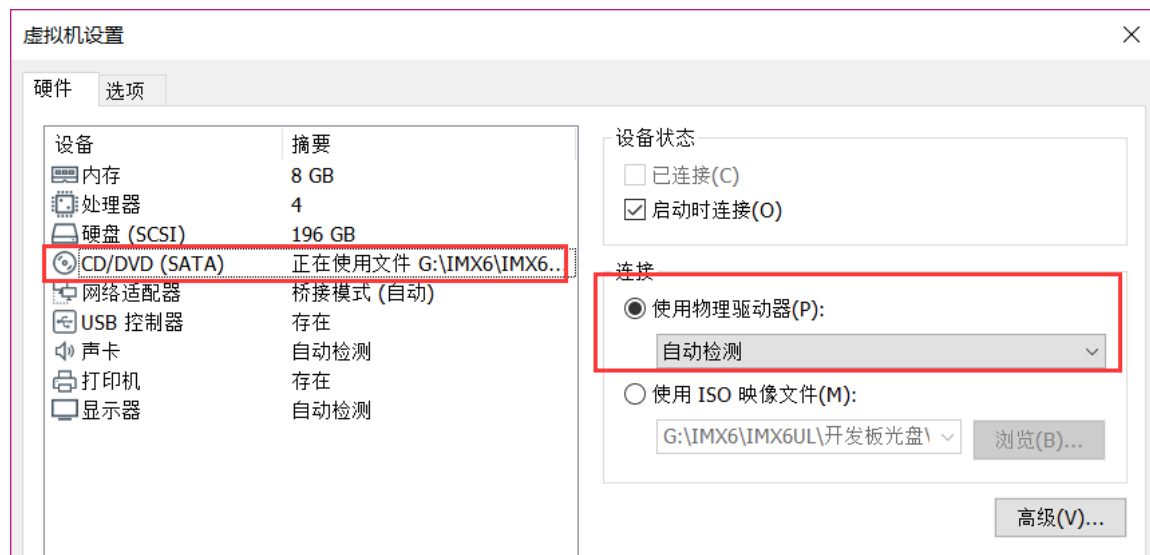


图 1.3.3.1 弹出 Ubuntu 系统镜像

设置好以后点击“确定”按钮，然后重新打开虚拟机，看看是否能够正常启动 Ubuntu，一般肯定能正常打开的。

至此，VMware 虚拟机以及 Ubuntu 系统安装成功，接下来我们就要学习如何是用 Ubuntu 了。

第二章 Ubuntu 系统入门

在上一章我们已经安装好虚拟机，并且在虚拟机中安装好了 Ubuntu 操作系统了，本章我们就来学习 Ubuntu 系统的基本使用，通过本章的学习为我们以后的开发做准备。Ubuntu 系统是和 Windows 一样的大型桌面操作系统，因此功能非常强大，不是一章就能介绍完的，因此本章叫做《Ubuntu 系统入门》。本章的主要目的是教会读者掌握后续嵌入式开发所需的 Ubuntu 基本技能，比如系统的基本设置、常用的 shell 命令、vim 编辑器的基本操作等等，如果想详细的学习 Ubuntu 操作系统请参考其它更为详实的书籍，本章参考了《Ubuntu Linux 从入门到精通》，这本书不厚，很适合用来作 Ubuntu 入门。

2.1 Ubuntu 系统初体验

2.1.1 Hello Ubuntu

上一章我们已经安装好了 Ubuntu 操作系统，我们再回顾一下如何开机：

1、打开 VMware 虚拟机软件，打开以后如图 2.1.1.1 所示：

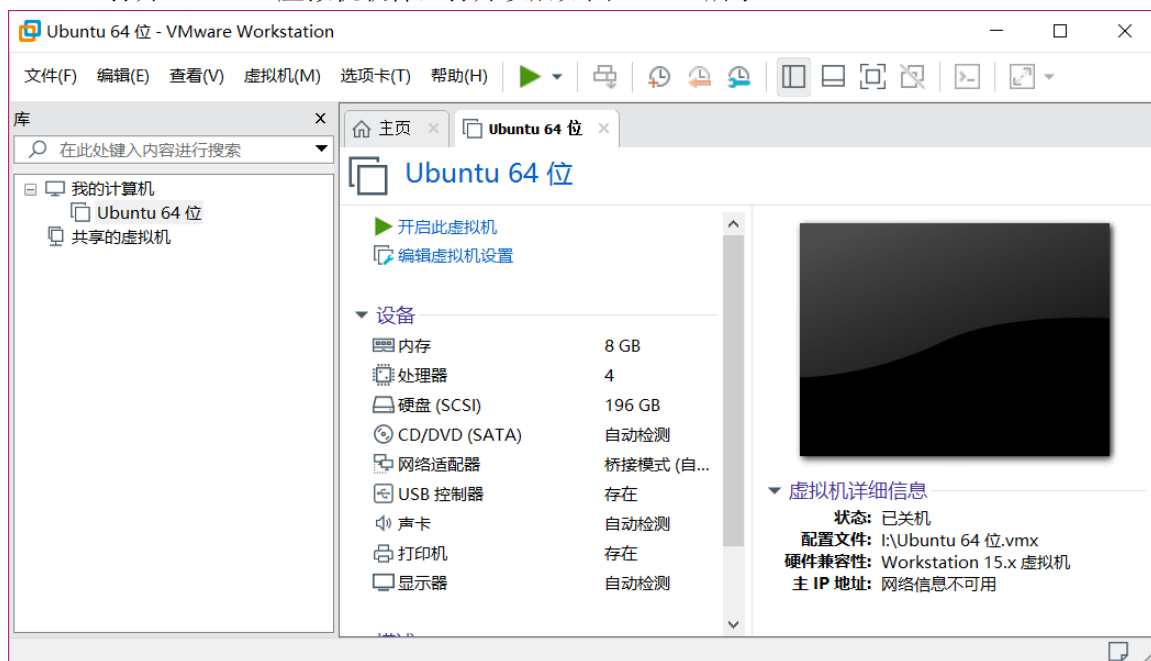


图 2.1.1.1 WMware 主界面

2、打开 VMware 上的开机按钮，打开方式如图 2.1.1.2 所示：



图 2.1.1.2 VMware 开机按钮

3、点击图 2.1.1.2 中两个开机按钮中的任意一个就会打开 Ubuntu 操作系统，首先进入图 2.1.1.3 所示的登陆界面，输入密码即可进入系统。

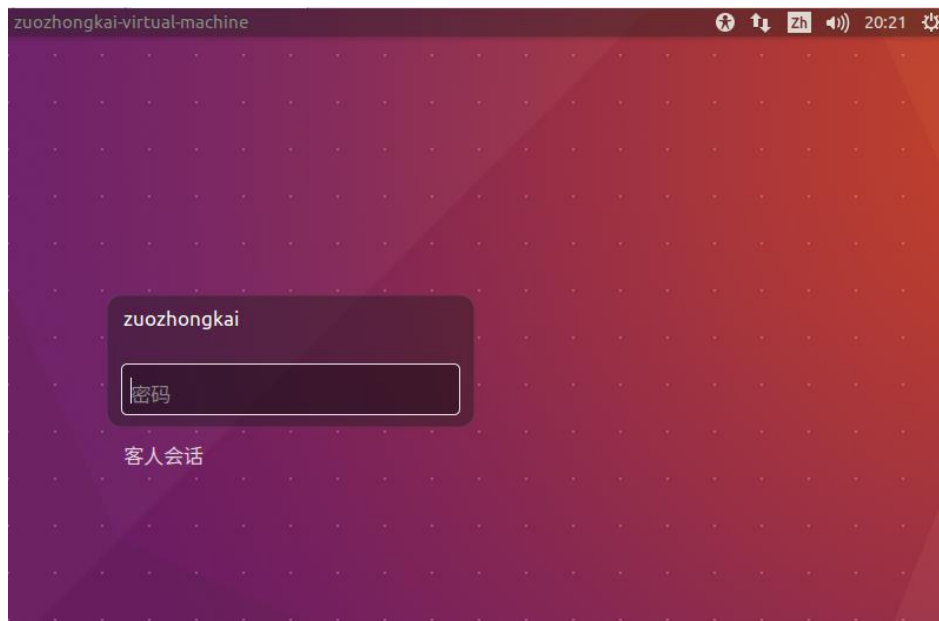


图 2.1.1.3 Ubuntu 登陆界面

在登陆界面输入密码，进入系统主界面，如图 2.1.1.4 所示：

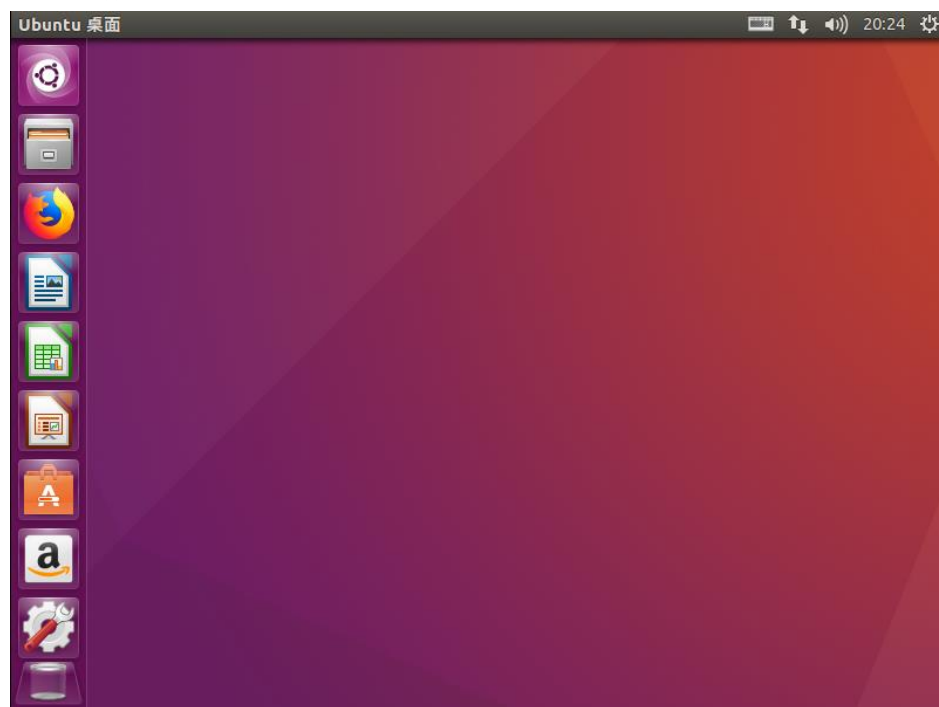


图 2.1.1.4 Ubuntu 主界面

进入主界面以后大家就可以看到和 Windows 基本一样，左侧有一列 APP，第一个是“搜索计算机”，第二个是文件浏览器，打开以后可以浏览 Ubuntu 系统中的文件，打开以后如图 2.1.1.5 所示：

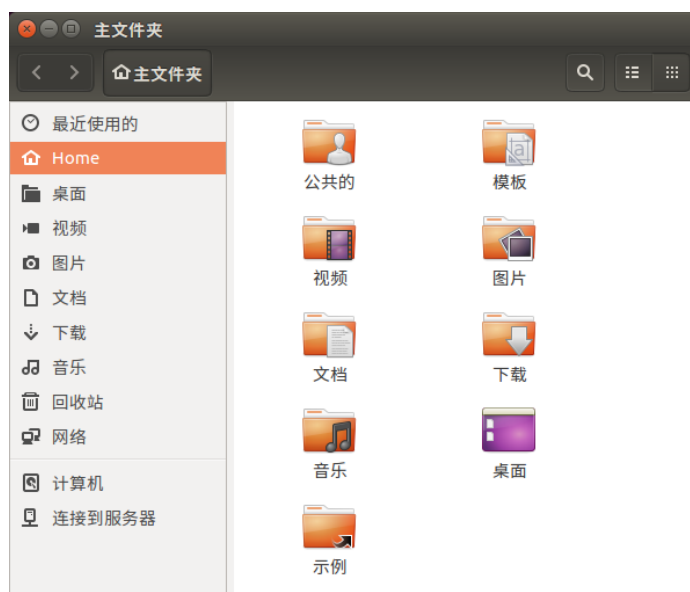


图 2.1.1.5 文件浏览

第三个是 firefox 浏览器，可以用来上网，比如我们登陆百度网站，如图 2.1.1.6 所示：

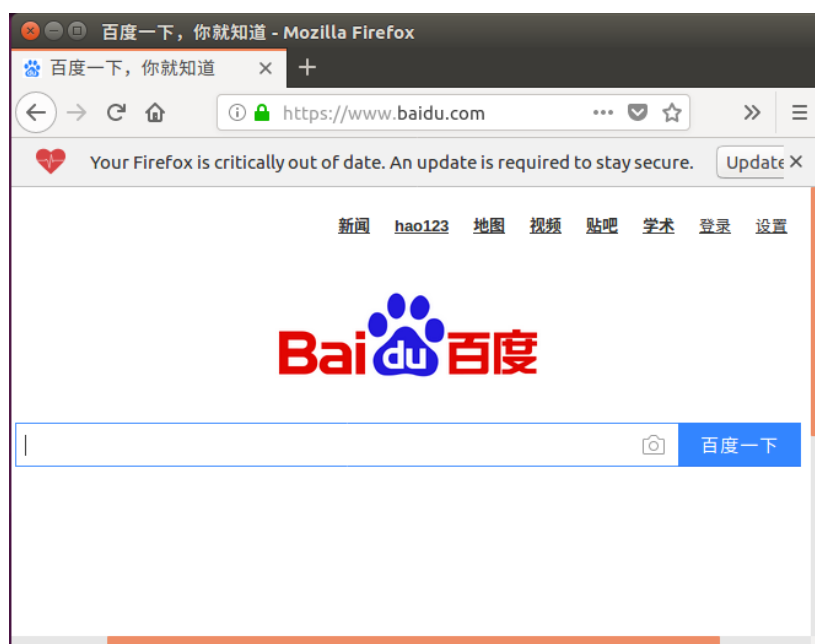


图 2.1.1.6 firefox 浏览器

这里还有其它一些 APP，大家可以自行打开看一下这些 APP 都是干啥的，这里就不一一详细的介绍了。

2.1.2 系统设置

我们会发现，Ubuntu 的默认桌面很小，这是因为 Ubuntu 默认分辨率是 800*600，因此我们首先要设置系统分辨率，调整到合适的大小。打开系统设置界面，打开方式如图 2.1.2.1 所示：

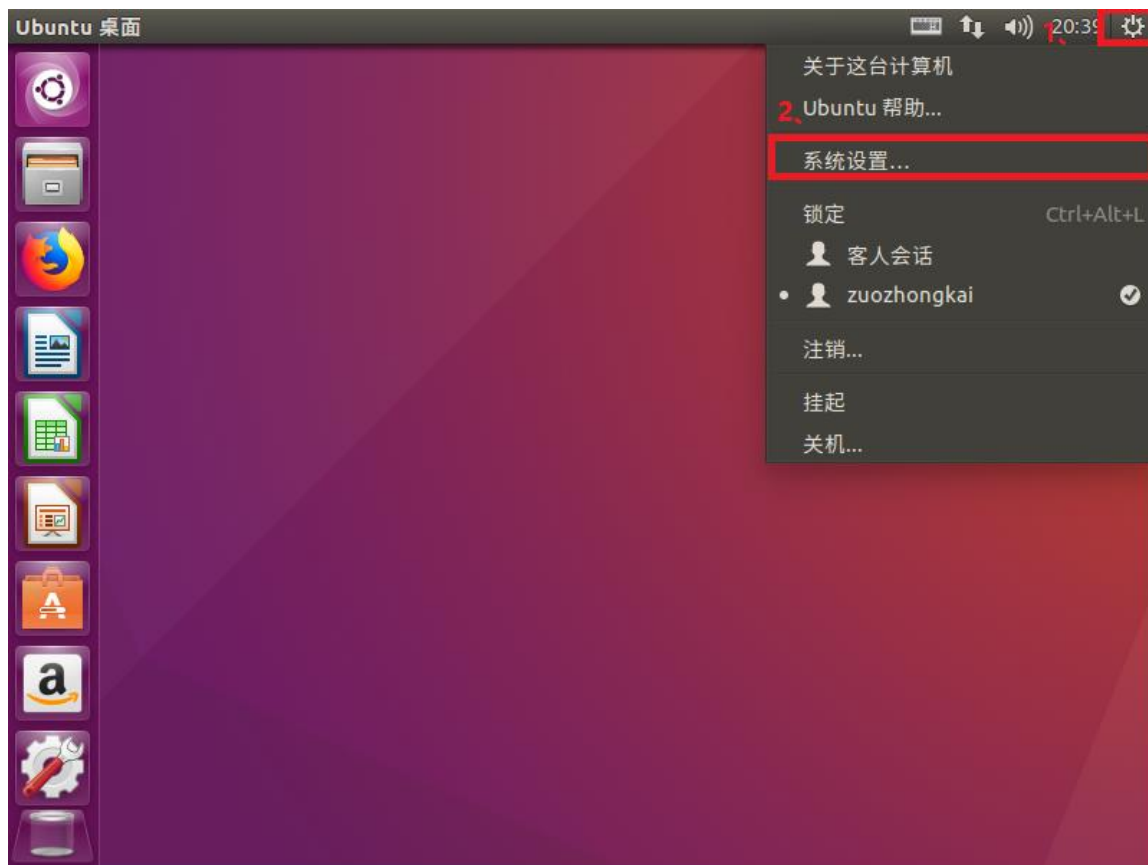


图 2.1.2.1 打开系统设置

打开以后的系统设置界面如图 2.1.2.2 所示:



图 2.1.2.2 系统设置界面

系统设置界面可以完成系统的大部分设置，我们找到“显示”设置并打开，打开以后如图 2.1.2.3 所示：



图 2.1.2.3 显示设置界面

从图 2.1.2.3 中可以看出，系统默认分辨率是 800X600，现在的电脑分辨率最少都是 1920X1080 了，因此我们可以调整这个分辨率至合适的大小，比如我设置为 1440x900 分辨率，设置好以后点击“应用”按钮，这里要注意，由于分辨率太小了，导致“应用”按钮就只露出了很少一部分，如图 2.1.2.4 所示：



图 2.1.2.4 调整屏幕分辨率

设置好分辨率以后 Ubuntu 的主界面就大了，看起来也舒服了。通过设置系统分辨率这个例子，我们就知道了如何设置 Ubuntu 系统，如果有需要设置其它东西的话都可以到系统设置里面去进行，这里就不一一详细的介绍了。

2.1.3 系统注销与关机

当我们不使用 Ubuntu 系统以后就需要将其关机，就和我们使用 Windows 系统一样，千万不要通过直接退出 VMware 软件来关机!! 关机很简单，在主界面，点击右上角的齿轮图标，然后选择关机选项，如图 2.1.3.1 所示：



图 2.1.3.1 关机

在图 2.1.3.1 中可以看到有三个选项：注销，挂起和关机，这个和 Windows 下是一样的，你如果想要注销就点击“注销”按钮，想要关机就点击“关机”按钮，以关机为例，点击关机以后会弹出图 2.1.3.2 所示关机确认界面，在确认界面上可以选择是“重启”还是“关机”。



图 2.1.3.2 关机确认界面

在图 2.1.3.2 中，左边的按钮为重启图标，点击以后系统重启，右边的按钮为关机按钮，点击以后就会关闭 Ubuntu 系统。

2.1.4 中文输入测试

我们是中国人，平时用的做多的肯定是中文，那么 Ubuntu 下中文输入是否和 Windows 一样呢？如何在 Ubuntu 下使用中文输入法。我们在安装 Ubuntu 系统的时候就已经使用过中文输

入法了，就是选择我们所在地的时候。本节我们就以创建一个文本为例，介绍如何在 Ubuntu 中使用中文输入法。

在桌面上点击鼠标右键，然后选择新建文档->空白文档，如图 2.1.4.1 所示：



图 2.1.4.1 新建空白文档

文档名字使用默认名字：无标题文档，如图 2.1.4.2 所示：

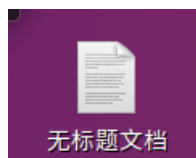


图 2.1.4.2 新建的无标题文档

双击打开文档，打开以后如图 2.1.4.3 所示：



图 2.1.4.3 打开文档

打开文档以后，我们可以尝试在里面输入一些英文和数字，输入英文和数字是没有任何问题的，输入中文的话需要切换到 Ubuntu 自带的拼音输入法，有两种方式切换，一种是使用快捷键:Windows+空格键，一种是使用鼠标点击设置输入法，如图 2.1.4.4 所示：



图 2.1.4.4 切换拼音输入法

这两种方法都可以切换输入法，切换到拼音输入法以后就可以输入中文了，如图 2.1.4.5 所示：



图 2.1.4.5 输入中文文本

大家会发现 Ubuntu 下的拼音输入法使用起来跟 Windows 下的输入法差距太大了，没有 Windows 下的输入法好用，没办法，谁让桌面端 Linux 用的少呢，所以也就没有啥公司开发 Linux 下的输入法。

通过上面几个小节中对 Ubuntu 的基本操作来看，基本和 Windows 下的操作差不多，我们真正要使用 Ubuntu 的不是通过图形界面操作，而是通过命令行操作的。这也是我们接下来着重要讲的：Ubuntu(Linux)终端操作，会涉及到很多命令，但是常用的命令就那几十个，不需要刻

意的去背，使用习惯了就自然记住了。不要看到要记命令就觉得可怕。根据 2080 原则，80% 情况下只使用那 20% 的命令，实际情况会更少，常用的可能就那 5%~10% 的命令。

2.2 Ubuntu 终端操作

本节就是我们学习 Ubuntu 操作系统的重点了，终端操作，也就是俗称的“敲命令”，不管是哪个版本的 Linux 发行版系统，它都会提供终端操作，Linux 下的终端操作类似于 Windows 下的 DOS 操作。要使用终端首先肯定是要打开终端，在主界面上点击鼠标右键，然后选择打开终端，如图 2.2.1 所示：

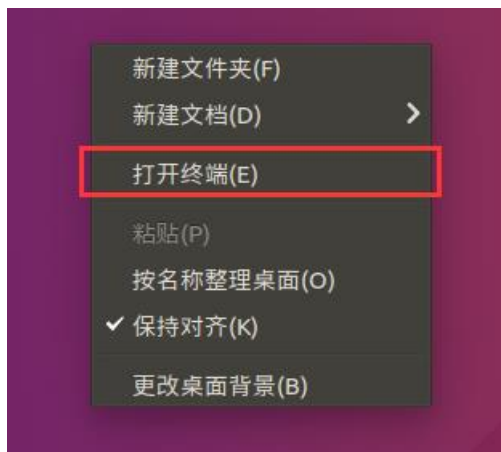


图 2.2.1 打开终端

打开终端以后如图 2.2.2 所示：

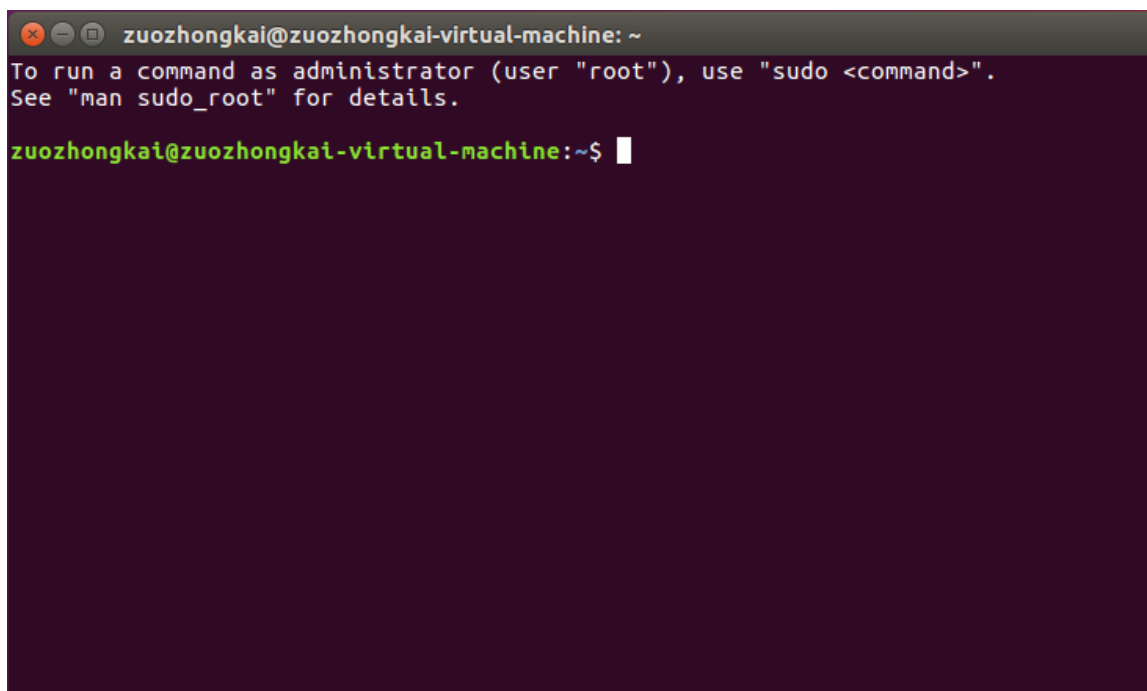


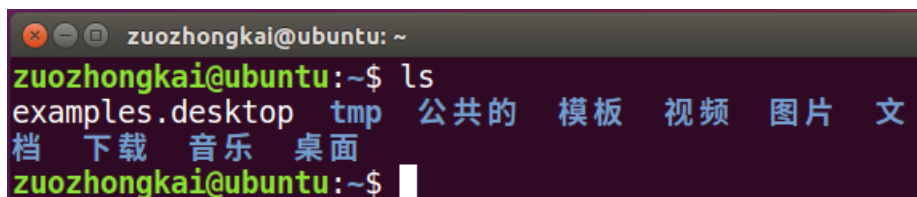
图 2.2.2 终端界面

我们就是在图 2.2.2 所示界面上输入命令的，终端默认会有类似下面一行所示的一串提示符：

```
zuozhongkai@zuozhongkai-virtual-machine:~$
```

上述字符串中，@前面的“zuozhongkai”是当前的用户名字，@后面的 zuozhongkai-virtual-

machine 是我的机器名字。最后面的符号“\$”表示当前用户是普通用户，我们可以在提示符后面输入命令，比如输入命令“ls”，命令“ls”是打印出当前所在目录中所有文件和文件夹，如图 2.2.3 所示：



```
zuozhongkai@ubuntu: ~  
zuozhongkai@ubuntu:~$ ls  
examples.desktop tmp 公共的 模板 视频 图片 文  
档 下载 音乐 桌面  
zuozhongkai@ubuntu:~$
```

图 2.2.3 ls 命令

在图 2.2.3 中我们输入了“ls”这个命令，然后打印出了当前目录下的所有文件和文件夹，后面我们学习命令的时候就是在终端中输入相应命令的。

2.3 Shell 操作

2.3.1 Shell 简介

学习 linux 的时候会频繁的看到 Shell 这个词语？那么什么是 Shell 呢？网上搜索一下，各种专业的解释一堆，但是对于第一次接触 Linux 的人来说这些专业的词语只会让人更晕。简单的说 Shell 就是敲命令。国内把 Linux 下通过命令行输入命令叫做“敲命令”，外国人玩的比较洋气，人家叫做“Shell”。因此以后看到 Shell 这个词语第一反应就是在终端中敲命令，将多个 Shell 命令按照一定的格式放到一个文本中，那么这个文本就叫做 Shell 脚本。

严格意义上来讲，Shell 是一个应用程序，它负责接收用户输入的命令，然后根据命令做出相应的动作，Shell 负责将应用层或者用户输入的命令传递给系统内核，由操作系统内核来完成相应的工作，然后将结果反馈给应用层或者用户。

2.3.2 Shell 基本操作

前面我们说 Shell 就是“敲命令”，那么既然是命令，那肯定是有格式的，Shell 命令的格式如下：

command	-options	[argument]
---------	----------	------------

command: Shell 命令名称。

options: 选项，同一种命令可能有不同的选项，不同的选项其实现的功能不同。

argument: Shell 命令是可以带参数的，也可以不带参数运行。

同样以命令“ls”为例，下面“ls”命令的三种不同格式其结果也不同：

ls
ls -l
ls /usr

这三种命令的运行结果如图 2.3.2.1 所示：

```

zuozhongkai@ubuntu: ~
zuozhongkai@ubuntu:~$ ls
examples.desktop tmp 公共的 模板 视频 图片 文档 下载 音乐 桌面
zuozhongkai@ubuntu:~$ ls -l
总用量 48
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 12月 18 02:08 examples.desktop
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月 19 00:29 tmp
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 19 00:25 桌面
zuozhongkai@ubuntu:~$ ls /usr
bin games include lib local locale sbin share src
zuozhongkai@ubuntu:~$

```

图 2.3.2.1 ls 命令

在图 2.3.2.1 中“ls”命令用来打印出当前目录下的所有文件和文件夹，而“ls -l”同样是打印出当前目录下的所有文件和文件夹，但是此命令会列出所有文件和文件夹的详细信息，比如文件大小、拥有者、创建日期等等。最有一个“ls /usr”是用来打印出目录“/usr”下的所有文件和文件夹。

Shell 命令是支持自动补全功能的，因为 Shell 命令非常多，如果不作自动补全的话就需要用户去记忆这些命令的全部字母。使用自动补全功能以后我们只需要输入命令的前面一部分字母，然后按下 TAB 键，如果只有一个命令匹配的话就会自动补全这个命令剩下的字母。如果有多个命令匹配的话系统就会发出报警声音，此时在按下一次 TAB 键就会列出所有匹配的命令，比如我们输入字母“if”，然后按下 TAB 键，结果如图 2.3.2.2 所示：

```

zuozhongkai@ubuntu:~$ if
if      ifconfig ifdown   ifquery ifup

```

图 2.3.2.2 “if” 开始的命令

从图 2.3.2.2 可以看出，以“if”开头的命令有 5 个，我们以“ifconfig”为例，此命令是用来查看网卡信息的，我们重新输入“ifc”然后在按一下 TAB 键，就会自动补全出“ifconfig”命令，因为以“ifc”开头的命令只有一个，结果如图 2.3.2.3 所示：

```

zuozhongkai@ubuntu:~$ ifconfig
ens33  Link encap:以太网 硬件地址 00:0c:29:96:89:d6
       inet 地址:192.168.31.235 广播:192.168.31.255 掩码:255.255.255.0
       inet6 地址: fe80::e92a:d882:e1b:7402/64 Scope:Link
       UP BROADCAST RUNNING MULTICAST MTU:1500 跃点数:1
       接收数据包:14268 错误:0 丢弃:0 过载:0 帧数:0
       发送数据包:1571 错误:0 丢弃:0 过载:0 载波:0
       碰撞:0 发送队列长度:1000
       接收字节:5861098 (5.8 MB) 发送字节:109785 (109.7 KB)

lo      Link encap:本地环回
       inet 地址:127.0.0.1 掩码:255.0.0.0
       inet6 地址: ::1/128 Scope:Host
       UP LOOPBACK RUNNING MTU:65536 跃点数:1
       接收数据包:247 错误:0 丢弃:0 过载:0 帧数:0
       发送数据包:247 错误:0 丢弃:0 过载:0 载波:0
       碰撞:0 发送队列长度:1000
       接收字节:19425 (19.4 KB) 发送字节:19425 (19.4 KB)

```

图 2.3.2.3 ifconfig 命令结果

2.2.4 常用 Shell 命令

我们做嵌入式开发用的最多就是 Shell 命令，Shell 命令是所有的 Linux 系统发行版所通用的，并不是说我在 Ubuntu 下学会了 Shell 命令，换另外一个 Linux 发行版操作系统以后就没用了(不同的发行版 Linux 系统可能会自定义一些命令)。本节我们先来介绍一些 Shell 下常用的命令：

1、目录信息查看命令 ls

文件浏览是最基本的操作了，Shell 下文件浏览命令为 ls，格式如下：

```
ls [选项] [路径]
```

ls 命令主要用于显示指定目录下的内容，列出指定目录下包含的所有的文件以及子目录，它的主要参数有：

- a 显示所有的文件以及子目录，包括以“.”开头的隐藏文件。
- l 显示文件的详细信息，比如文件的形态、权限、所有者、大小等信息。
- t 将文件按照创建时间排序列出。
- A 和-a 一样，但是不列出“.”(当前目录)和“..”(父目录)。
- R 递归列出所有文件，包括子目录中的文件。

Shell 命令里面的参数是可以组合在一起用的，比如组合“-al”就是显示所有文件的详细信息，包括以“.”开头的隐藏文件，ls 命令使用如图 2.2.4.1 所示：

```
zuozhongkai@ubuntu:~/tmp$ ls
a b c
zuozhongkai@ubuntu:~/tmp$ ls -a
. .. a b c
zuozhongkai@ubuntu:~/tmp$ ls -al
总用量 8
drwxrwxr-x  2 zuozhongkai zuozhongkai 4096 12月 19 21:41 .
drwxr-xr-x 20 zuozhongkai zuozhongkai 4096 12月 19 20:31 ..
-rw-rw-r--  1 zuozhongkai zuozhongkai   0 12月 19 21:41 a
-rw-rw-r--  1 zuozhongkai zuozhongkai   0 12月 19 21:41 b
-rw-rw-r--  1 zuozhongkai zuozhongkai   0 12月 19 21:41 c
```

图 2.2.4.1 ls 命令演示

注意图 2.2.4.1 中 tmp 文件夹是我为了演示方便，自己创建的，里面的文件 a，b 和 c 也是我创建的，关于文件夹和文件的创建后面会详细的讲解。

2、目录切换命令 cd

要想在 Shell 中切换到其它的目录，使用的命令是 cd，命令格式如下：

```
cd [路径]
```

路径就是我们要进入的目录路径，比如下面所示操作：

```
cd /          //进入到根目录“/”下，Linux 系统的根目录为“/”，
cd /usr       //进入到目录“/usr”里面。
cd ..         //进入到上一级目录。
cd ~          //切换到当前用户主目录
```

比如我们要进入到目录“/usr”下去，并且查看“/usr”下有什么文件，操作如图 2.2.4.2 所示：


```
zuozhongkai@ubuntu:~$ cd /usr
zuozhongkai@ubuntu:/usr$ ls
bin  games  include  lib  local  locale  sbin  share  src
```

图 2.2.4.2 cd 命令演示

在图 2.2.4.2 中,我们先使用命令“cd /usr”进入到“/usr”目录下,然后使用“ls”命令显示“/usr”目录下的所有文件。仔细观察图 2.2.4.2 可以看到,当我们切换到其它目录以后在符号“\$”前面就会以蓝色的字体显示出当前目录名字,如图 2.2.4.3 所示:

```
zuozhongkai@ubuntu:/usr$ ls
bin  games  include  lib  local  locale  sbin  share  src
```

图 2.2.4.3 目录路径显示

3、当前路径显示命令 pwd

pwd 命令用来显示当前工作目录的绝对路径,不需要任何的参数,使用如图 2.2.4.4 所示:

```
zuozhongkai@ubuntu:~$ pwd
/home/zuozhongkai
```

图 2.2.4.4 pwd 命令

4、系统信息查看命令 uname

要查看当前系统信息,可以使用命令 uname,命令格式如下:

uname [选项]

可选的选项参数如下:

- r 列出当前系统的具体内核版本号。
- s 列出系统内核名称。
- o 列出系统信息。

使用如图 2.2.4.5 所示:

```
zuozhongkai@ubuntu:~$ uname
Linux
zuozhongkai@ubuntu:~$ uname -r
4.15.0-29-generic
zuozhongkai@ubuntu:~$ uname -s
Linux
zuozhongkai@ubuntu:~$ uname -o
GNU/Linux
```

图 2.2.4.5 uanme 命令操作

5、清屏命令 clear

clear 命令用于清除终端上的所有内容,只留下一行提示符。

6、切换用户执行身份命令 sudo

Ubuntu(Linux)是一个允许多用户的操作系统,其中权限最大的就是超级用户 root,有时候我们执行一些操作的时候是需要用 root 用户身份才能执行,比如安装软件。通过 sudo 命令可以使我们暂时将身份切换到 root 用户。当使用 sudo 命令的时候是需要输入密码的,这里要注意输入密码的时候是没有任何提示的!命令格式如下:

sudo [选项] [命令]

选项主要参数如下:

- h 显示帮助信息。

- l 列出当前用户可执行与不可执行的命令
- p 改变询问密码的提示符。

假如我们现在要创建一个新的用户 test，创建新用户的命令为“adduser”，创建新用户的权限只有 root 用户才有，我们在装系统的时候创建的那个用户是没有这个权限的，比如我的“zuozhongkai”用户。所以创建新用户的话需要使用“sudo”命令以 root 用户执行“adduser”这个命令，如图 2.2.4.6 所示：

```

zuozhongkai@ubuntu:~$ adduser test
adduser: 只有 root 才能将用户或组添加到系统。
zuozhongkai@ubuntu:~$ sudo adduser test
正在添加用户 "test"...
正在添加新组 "test" (1001)...
正在添加新用户 "test" (1001) 到组 "test"...
创建主目录 "/home/test"...
正在从 "/etc/skel" 复制文件...
输入新的 UNIX 密码:
重新输入新的 UNIX 密码:
passwd: 已成功更新密码
正在改变 test 的用户信息
请输入新值，或直接敲回车键以使用默认值
全名 []:
房间号码 []:
工作电话 []:
家庭电话 []:
其它 []:
这些信息是否正确? [Y/n] y

```

图 2.2.4.6 sudo 命令演示

在图 2.2.4.6 中，我们一开始直接使用“adduser test”命令添加用户的时候提示我们“adduser: 只有 root 才能将用户或组添加到系统。”所以我们要在前面加上“sudo”命令，表示以 root 用户执行 adduser 操作。

7、添加用户命令 adduser

在讲解 sudo 命令的时候我们已经用过命令“adduser”，此命令需要 root 身份去运行。命令格式如下：

```
adduser [参数] [用户名]
```

常用的参数如下：

- system 添加一个系统用户
- home DIR DIR 表示用户的主目录路径
- uid ID ID 表示用户的 uid。
- ingroup GRP 表示用户所属的组名。

adduser 的使用我们前面已经演示过了，大家可以试着再添加一个用户。

8、删除用户命令 deluser

前面讲了添加用户的命令，那肯定也有删除用户的命令，删除用户使用命令“deluser”，命令参数如下：

```
deluser [参数] [用户名]
```

主要参数有：

- system 当用户是一个系统用户的时候才能删除。
- remove-home 删除用户的主目录
- remove-all-files 删除与用户有关的所有文件。

-backup 备份用户信息

同样的，命令“deluser”也要使用“sudo”来以 root 用户运行，以删除我们前面创建的用户 test 为例，deluser 使用如图 2.2.4.7 所示：

```
zuozhongkai@ubuntu:~$ sudo deluser --remove-all-files test
正在寻找要备份或删除的文件...
/usr/sbin/deluser: 无法处理特殊文件 /sys/kernel/security/apparmor/.null
/usr/sbin/deluser: 无法处理特殊文件 /run/udev/static_node-tags/uaccess/snd\x2fseq
/usr/sbin/deluser: 无法处理特殊文件 /run/udev/static_node-tags/uaccess/snd\x2ftimer
/usr/sbin/deluser: 无法处理特殊文件 /dev/vcsa7
/usr/sbin/deluser: 无法处理特殊文件 /dev/vcs7
.....
/usr/sbin/deluser: 无法处理特殊文件 /lib/systemd/system/single.service
/usr/sbin/deluser: 无法处理特殊文件 /lib/systemd/system/cryptdisks.service
/usr/sbin/deluser: 无法处理特殊文件 /lib/systemd/system/cryptdisks-early.service
正在删除文件...
正在删除用户 'test'...
警告：组 "test"没有其他成员了。
完成。
```

图 2.2.4.7 命令 deluser 演示

9、切换用户命令 su

前面在讲解命令“sudo”的时候说过，“sudo”是以 root 用户身份执行一个命令，并没有更改当前的用户身份，所有需要 root 身份执行的命令都必须在前面加上“sudo”。命令“su”可以直接将当前用户切换为 root 用户，切换到 root 用户以后就可以尽情的进行任何操作了！因为你已经获得了系统最高权限，在 root 用户下，所有的命令都可以无障碍执行，不需要在前面加上“sudo”，“su”命令格式如下：

su [选项] [用户名]

常用选项参数如下：

- c --command 执行指定的命令，执行完毕以后恢复原用户身份。
- login 改变用户身份，同时改变工作目录和 PATH 环境变量。
- m 改变用户身份的时候不改变环境变量
- h 显示帮助信息

以切换到 root 用户为例，使用如图 2.2.4.8 所示：

```
zuozhongkai@ubuntu:~$ sudo su
[sudo] zuozhongkai 的密码：
root@ubuntu:/home/zuozhongkai#
```

图 2.2.4.8 su 命令演示

在图 2.2.4.8 中，先使用命令“sudo su”切换到 root 用户，su 命令不写明用户名的话默认切换到 root 用户。然后输入密码，密码正确的话就会切换到 root 用户，可以看到切换到 root 用户以后提示符的“@”符号前面的用户名变成了“root”，表示当前的用户是 root 用户。并且以“#”结束。

注意！！由于 root 用户权限太大，稍微不注意就可能删除掉系统文件，导致系统奔溃，因此强烈建议大家，不要以 root 用户运行 Ubuntu。当要用到 root 身份执行某些命令的时候使用“sudo”命令即可。

要切换回原来的用户，使用命令“sudo su 用户名”即可，比如我要从 root 切换回 zuozhongkai 这个用户，操作如图 2.2.4.9 所示：

```
root@ubuntu:/home/zuozhongkai# sudo su zuozhongkai
zuozhongkai@ubuntu:~$
```

图 2.2.4.9 切换回原来用户

10、显示文件内容命令 cat

查看文件内容是最常见的操作了，在 windows 下可以直接使用记事本查看一个文本文件内容，linux 下也有类似记事本的软件，叫做 gedit，找到一个文本文件，双击打开，默认使用的就是 gedit，如图 2.2.4.10 所示：

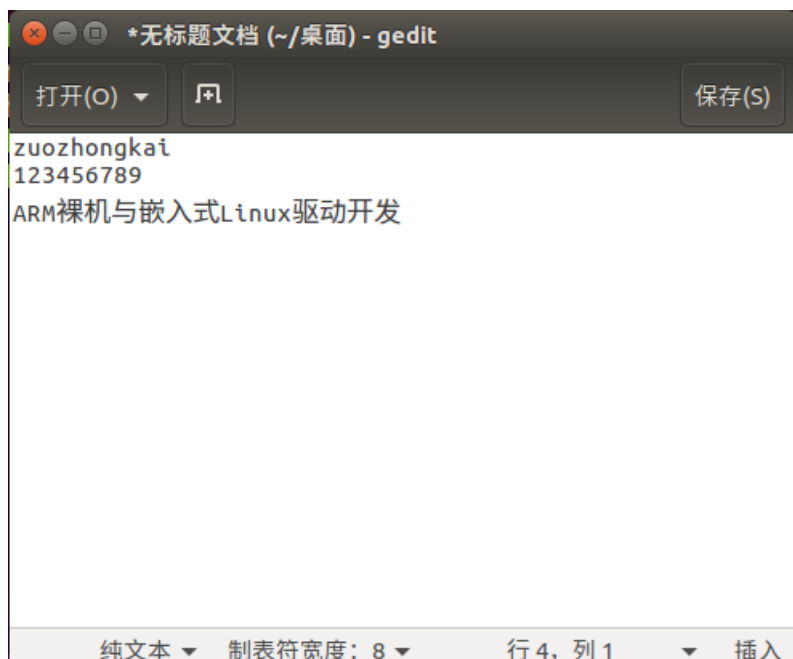


图 2.2.4.10 gedit 打开文档

我们现在讲解的是 Shell 命令，那么 Shell 下有没有办法读取文件的内容呢？肯定有的，那就是命令“cat”，命令格式如下：

```
cat [选项] [文件]
```

选项主要参数如下：

- n 由 1 开始对所有输出的行进行编号。
- b 和 -n 类似，但是不对空白行编号。
- s 当遇到连续两个行以上空白行的话就合并为一个行空白行。

比如我们以查看文件“/etc/environment”的内容为例，结果如图 2.2.4.11 所示：

```
zuozhongkai@ubuntu:~$ cat /etc/environment
PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games"
zuozhongkai@ubuntu:~$ cat /etc/environment -n
1 PATH="/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games"
```

图 2.2.4.11 命令 cat 演示

11、显示和配置网络属性命令 ifconfig

ifconfig 是一个跟网络属性配置和显示密切相关的命令，通过此命令我们可以查看当前网络属性，也可以通过此命令配置网络属性，比如设置网络 IP 地址等等，此命令格式如下：

```
ifconfig interface options | address
```

主要参数如下：

- interface** 网络接口名称，比如 eth0 等。
- up** 开启网络设备。
- down** 关闭网络设备。
- add** IP 地址，设置网络 IP 地址。

netmask add 子网掩码。

命令 ifconfig 的使用如图 2.2.4.12 所示:

```
zuozhongkai@ubuntu:~$ ifconfig
ens33    Link encap:以太网  硬件地址 00:0c:29:96:89:d6
          inet 地址:192.168.31.235 广播:192.168.31.255 掩码:255.255.255.0
          inet6 地址: fe80::e92a:d882:e1b:7402/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  跃点数:1
          接收数据包:13404 错误:0 丢弃:0 过载:0 帧数:0
          发送数据包:967 错误:0 丢弃:0 过载:0 载波:0
          碰撞:0 发送队列长度:1000
          接收字节:810508 (810.5 KB)  发送字节:75728 (75.7 KB)

lo       Link encap:本地环回
          inet 地址:127.0.0.1 掩码:255.0.0.0
          inet6 地址: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  跃点数:1
          接收数据包:248 错误:0 丢弃:0 过载:0 帧数:0
          发送数据包:248 错误:0 丢弃:0 过载:0 载波:0
          碰撞:0 发送队列长度:1000
          接收字节:19004 (19.0 KB)  发送字节:19004 (19.0 KB)

zuozhongkai@ubuntu:~$ ifconfig ens33
ens33    Link encap:以太网  硬件地址 00:0c:29:96:89:d6
          inet 地址:192.168.31.235 广播:192.168.31.255 掩码:255.255.255.0
          inet6 地址: fe80::e92a:d882:e1b:7402/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  跃点数:1
          接收数据包:13406 错误:0 丢弃:0 过载:0 帧数:0
          发送数据包:971 错误:0 丢弃:0 过载:0 载波:0
          碰撞:0 发送队列长度:1000
          接收字节:810658 (810.6 KB)  发送字节:76218 (76.2 KB)
```

图 2.2.4.12 ifconfig 命令演示

在图 2.2.4.12 中有两个网卡: ens33 和 lo, ens33 是我的电脑实际使用的网卡, lo 是回测网卡。可以看出网卡 ens33 的 IP 地址为 192.168.31.235, 我们使用命令“ifconfig”将网卡 ens33 的 IP 地址改为 192.168.31.20, 操作如图 2.2.4.13 所示:

```
zuozhongkai@ubuntu:~$ sudo ifconfig ens33 192.168.31.20
zuozhongkai@ubuntu:~$ ifconfig ens33
ens33    Link encap:以太网  硬件地址 00:0c:29:96:89:d6
          inet 地址:192.168.31.20 广播:192.168.31.255 掩码:255.255.255.0
          inet6 地址: fe80::e92a:d882:e1b:7402/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  跃点数:1
          接收数据包:15188 错误:0 丢弃:0 过载:0 帧数:0
          发送数据包:1040 错误:0 丢弃:0 过载:0 载波:0
          碰撞:0 发送队列长度:1000
          接收字节:917930 (917.9 KB)  发送字节:84590 (84.5 KB)
```

图 2.2.4.13 修改网卡 IP 地址

从图 2.2.4.13 可以看出, 我在使用命令“ifconfig”修改网卡 ens33 的 IP 地址的时候使用了“sudo”, 说明在 Ubuntu 下修改网卡 IP 地址是需要 root 用户权限的。当修改完以后使用命令“ifconfig ens33”再次查看网卡 ens33, 发现网卡 ens33 的 IP 地址变成了 192.168.31.20

12、系统帮助命令 man

Ubuntu 系统中有很多命令, 这些命令都有不同的格式, 不同的格式对应不同的功能, 要完全记住这些命令和格式几乎是不可能的, 必须有一个帮助手册, 当我们需要了解一个命令的详细信息的时候查阅这个帮助手册就行了。Ubuntu 提供了一个命令来帮助用户完成这个功能, 那就是“man”命令, 通过“man”命令可以查看其它命令的语法格式、主要功能、主要参数说明

等, “man” 命令格式如下:

```
man [命令名]
```

比如我们要查看命令 “ifconfig” 的说明, 输入 “man ifconfig” 即可, 如图 2.2.4.14 所示:

```
zuozhongkai@ubuntu:~$ man ifconfig
```

图 2.2.4.14 man 命令演示

在终端中输入图 2.2.4.14 所示的命令, 然后点击回车键就会打开 “ifconfig” 这个命令的详细说明, 如图 2.2.4.15 所示:

```
IFCONFIG(8)                                Linux Programmer's Manual                                IFCONFIG(8)

NAME
    ifconfig - configure a network interface

SYNOPSIS
    ifconfig [-v] [-a] [-s] [interface]
    ifconfig [-v] interface [aftype] options | address ...

DESCRIPTION
    Ifconfig is used to configure the kernel-resident network interfaces.
    It is used at boot time to set up interfaces as necessary. After that,
    it is usually only needed when debugging or when system tuning is
    needed.

    If no arguments are given, ifconfig displays the status of the cur-
    rently active interfaces. If a single interface argument is given, it
    displays the status of the given interface only; if a single -a argu-
    ment is given, it displays the status of all interfaces, even those
    that are down. Otherwise, it configures an interface.

Address Families
    If the first argument after the interface name is recognized as the

Manual page ifconfig(8) line 1 (press h for help or q to quit)
```

图 2.2.4.15 命令 “ifconfig” 详细介绍信息

图 2.2.4.15 就是命令 “ifconfig” 的详细介绍信息, 按 “q” 键退出到终端。

13、系统重启命令 reboot

通过点击 Ubuntu 主界面右上角的齿轮按钮来选择关机或者重启系统, 同样的我们也可以使用 Shell 命令 “reboot” 来重启系统, 直接输入命令 “reboot” 然后点击回车键即可, 如图 2.2.4.16 所示:

```
zuozhongkai@ubuntu:~$ reboot
```

图 2.2.4.16 reboot 命令演示

14、系统关闭命令 poweroff

使用命令 “reboot” 可以重启系统, 使用命令 “poweroff” 就可以关闭系统, 在终端中输入命令 “poweroff” 然后按下回车键即可关闭 Ubuntu 系统, 如图 2.2.4.17 所示:

```
zuozhongkai@ubuntu:~$ poweroff
```

图 2.2.4.17 poweroff 命令演示

15、软件安装命令 install

截至目前, 我们都没有讲过 Ubuntu 下如何安装软件, 因为 Ubuntu 安装软件不像 Windows 下那样, 直接双击.exe 文件就开始安装了。Ubuntu 下很多软件是需要先自行下载源码, 下载源

码以后自行编译, 编译完成以后使用命令“`install`”来安装。当然 Ubuntu 下也有其它的软件安装方法, 但是用的最多的就是自行编译源码然后安装, 尤其是嵌入式 Linux 开发。命令“`install`”格式如下:

```
install [选项]... [-T] 源文件      目标文件
或: install [选项]...      源文件...  目录
或: install [选项]... -t 目录      源文件...
或: install [选项]... -d 目录...
```

“`install`”命令是将文件(通常是编译后的文件)复制到目的位置, 在前三种形式中, 将源文件复制到目标文件或将多个源文件复制到一个已存在的目录中同时设置其所有权和权限模式。在第四种形式会创建指定的目录。命令“`install`”通常和命令“`apt-get`”组合在一起使用的, 关于“`apt-get`”命令我们稍后会讲解。

以上就是 Shell 最基本一些命令, 还有一些其它的命令我们在后面在讲解, 循序渐进嘛。

2.4 APT 下载工具

对于长时间使用 Windows 的我们, 下载安装软件非常容易, Windows 下有很多的下载软件, Ubuntu 同样有不少的下载软件, 本节我们讲解 Ubuntu 下我们用的最多的下载工具: APT 下载工具, APT 下载工具可以实现软件自动下载、配置、安装二进制或者源码的功能。APT 下载工具和我们前面讲解的“`install`”命令结合在一起构成了 Ubuntu 下最常用的下载和安装软件方法。它解决了 Linux 平台下安装软件的一个缺陷, 即软件之间相互依赖。

APT 采用的 C/S 模式, 也就是客户端/服务器模式, 我们的 PC 机作为客户端, 当需要下载软件的时候就向服务器请求, 因此我们需要知道服务器的地址, 也叫做安装源或者更新源。打开系统设置, 打开“软件和更新”设置, 打开以后如图 2.4.1.1 所示:



图 2.4.1.1 软件和更新设置

在图 2.4.1.1 中的“Ubuntu 软件”选项卡下面的“下载自”就是 APT 工具的安装源, 因为

我们是在中国，所以需要选择中国的服务器，否则的话可能会导致下载失败！这个也就是网上说的 Ubuntu 安装成功以后要更新源。

在我们使用 APT 工具下载安装或者更新软件的时候，首先会在下载列表中与本机软件对比，看一下需要下载哪些软件，或者升级哪些软件，默认情况下 APT 会下载最新的软件包，被安装的软件包所依赖的其它软件也会被下载安装。说了这么多，APT 下载工具究竟怎么用呢？APT 工具常用的命令如下：

1、更新本地数据库

如果想查看本地哪些软件可以更新的话可以使用如下命令：

```
sudo apt-get update
```

这个命令会访问源地址，并且获取软件列表并保存在本电脑上，过程如图 2.4.1.2 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get update
[sudo] zuozhongkai 的密码：
命中:1 http://security.ubuntu.com/ubuntu xenial-security InRelease
命中:2 http://cn.archive.ubuntu.com/ubuntu xenial InRelease
获取:3 http://cn.archive.ubuntu.com/ubuntu xenial-updates InRelease [109 kB]
获取:4 http://cn.archive.ubuntu.com/ubuntu xenial-backports InRelease [107 kB]
已下载 216 kB，耗时 13秒 (16.5 kB/s)
正在读取软件包列表... 完成
```

图 2.4.1.2 更新本地数据库

2、检查依赖关系

有时候本地某些软件可能存在依赖关系，所谓依赖关系就是 A 软件依赖于 B 软件。通过如下命令可以查看依赖关系，如果存在依赖关系的话 APT 会提出解决方案：

```
sudo apt-get check
```

上述命令的执行结果如图 2.4.1.3 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get check
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
```

图 2.4.1.3 检查依赖关系

3、软件安装

这个是重点了，安装软件，使用如下命令：

```
sudo apt-get install package-name
```

可以看出上述命令是由“apt-get”和“install”组合在一起的，“package-name”就是要安装的软件名字，“apt-get”负责下载软件，“install”负责安装软件。比如我们要安装软件 Ubuntu 下的串口工具“minicom”，我们就可以使用如下命令：

```
sudo apt-get install minicom
```

执行上述命令以后就会自动下载和安装 minicom 软件，如图 2.4.1.3 所示：

```

zuozhongkai@ubuntu:~$ sudo apt-get install minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
将会同时安装下列软件：
  lrzsz
下列【新】软件包将被安装：
  lrzsz minicom
升级了 0 个软件包，新安装了 2 个软件包，要卸载 0 个软件包，有 92 个软件包未被升级。
需要下载 306 kB 的归档。
解压缩后会消耗 1,193 kB 的额外空间。
您希望继续执行吗？ [Y/n] y
获取:1 http://cn.archive.ubuntu.com/ubuntu xenial/universe amd64 lrzsz amd64 0.12.21-8 [73.8 kB]
获取:2 http://cn.archive.ubuntu.com/ubuntu xenial-updates/universe amd64 minicom amd64 2.7-1+deb8u1build0.16.04.1 [232 kB]
已下载 306 kB，耗时 3秒 (80.8 kB/s)
正在选中未选择的软件包 lrzsz。
(正在读取数据库 ... 系统当前共安装有 217399 个文件和目录。)
正准备解包 .../lrzsz_0.12.21-8_amd64.deb ...
正在解包 lrzsz (0.12.21-8) ...
正在选中未选择的软件包 minicom。
正准备解包 .../minicom_2.7-1+deb8u1build0.16.04.1_amd64.deb ...
正在解包 minicom (2.7-1+deb8u1build0.16.04.1) ...
正在处理用于 man-db (2.7.5-1) 的触发器 ...
正在设置 lrzsz (0.12.21-8) ...
正在设置 minicom (2.7-1+deb8u1build0.16.04.1) ...
    
```

图 2.4.1.3 安装 minicom 软件

图 2.4.1.3 就是安装 minicom 这个软件的过程，在图 2.4.1.3 中的安装过程中，会有如下所示询问：

您希望继续执行吗？ [Y/n]

如果希望继续执行的话就输入 y，如果不希望继续执行的话就输入 n。安装完成以后我们直接在终端输入如下命令打开 minicom 这个串口软件：

```
minicom -s
```

打开以后的 minicom 软件如图 2.4.1.4 所示：

```

+-----[configuration]-----+
| Filenames and paths          |
| File transfer protocols      |
| Serial port setup            |
| Modem and dialing            |
| Screen and keyboard          |
| Save setup as dfl             |
| Save setup as..              |
| Exit                         |
| Exit from Minicom            |
    
```

图 2.4.1.4 minicom 软件

关于 minicom 的使用大家可以上网搜索一下，这里就不详细讲解了，要退出 minicom 可以直接按下 ESC 键

4、软件更新

有时候我们需要更新软件，更新软件的话使用命令：

```
sudo apt-get upgrade package-name
```

其中 package-name 为要升级的软件名字，比如我们升级刚刚安装的 minicom 这个软件，如图 2.4.1.5 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get upgrade minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
minicom 已经是最新版 (2.7-1+deb8u1build0.16.04.1)。
正在计算更新... 完成
下列软件包的版本将保持不变：
ubuntu-minimal
```

图 2.4.1.5 更新 minicom 软件

从图 2.4.1.5 可以看出，minicom 已经是最新的了，不用更新，不过有其它软件需要更新，因此会自动更新其它的软件。

5、卸载软件

如果要卸载某个软件的话使用如下命令：

```
sudo apt-get remove package-name
```

其中 package-name 是要卸载的软件，比如卸载前面安装的 minicom 这个软件，操作如图 2.4.1.6 所示：

```
zuozhongkai@ubuntu:~$ sudo apt-get remove minicom
正在读取软件包列表... 完成
正在分析软件包的依赖关系树
正在读取状态信息... 完成
下列软件包是自动安装的并且现在不需要了：
lrzsz
使用 'sudo apt autoremove' 来卸载它(它们)。
下列软件包将被【卸载】：
minicom
升级了 0 个软件包，新安装了 0 个软件包，要卸载 1 个软件包，有 1 个软件包未被升级。
解压缩后将会空出 928 kB 的空间。
您希望继续执行吗？ [Y/n] y
(正在读取数据库 ... 系统当前共安装有 217494 个文件和目录。)
正在卸载 minicom (2.7-1+deb8u1build0.16.04.1) ...
正在处理用于 man-db (2.7.5-1) 的触发器 ...
```

图 2.4.1.6 卸载软件

从图 2.4.1.6 中可以看出软件 minicom 被卸载掉了。关于 APT 下载工具就讲解到这里，我们用的最多的就是“sudo apt-get install package-name”来下载和安装软件。有关 Ubuntu 其它的安装软件的方法打开可以自行上网查阅学习，这里就不一一详解了。

2.5 Ubuntu 下文本编辑

2.5.1 Gedit 编辑器

进行文本编辑是最常用的操作，Windows 下我们会使用记事本来完成，或者其它一些优秀的文本编辑器，比如 notepad++，Ubuntu 下有一个自带的文本编辑器，那就是 Gedit。Gedit 是一个窗口式的编辑器，关于 Gedit 的使用前面我们已经讲解了。本节我们重点讲解的是另外一

个编辑器：VI/VIM 编辑器。

2.5.2 VI/VIM 编辑器

我们如果要在终端模式下进行文本编辑或者修改文件就可以使用 VI/VIM 编辑器，Ubuntu 自带了 VI 编辑器，但是 VI 编辑器对于习惯了 Windows 下进行开发的人来说不方便，比如竟然不能使用键盘上的上下左右键调整光标位置。因此我推荐大家使用 VIM 编辑器，VIM 编辑器是 VI 编辑器升级版本，VI/VIM 编辑器都是一种基于指令式的编辑器，不需要鼠标，也没有菜单，仅仅使用键盘来完成所有的编辑工作。

我们需要先安装 VIM 编辑器，命令如下：

```
sudo apt-get install vim
```

安装完成以后就可以使用 VIM 编辑器了，VIM 编辑器有 3 种工作模式：输入模式、指令模式和底行模式，通过切换不同的模式可以完成不同的功能，我们就以编辑一个文本文档为例讲解 VIM 编辑器的使用。打开终端，输入命令：vi test.txt，如图 2.5.2.1 所示：

```
zuozhongkai@ubuntu:~$ vim test.txt
```

图 2.5.2.1 新建 test.txt 文档

在终端中输入图 2.5.2.1 中所示的命令以后就会创建一个 test.txt 文档，并且用 VIM 打开了，如图 2.5.2.2 所示：

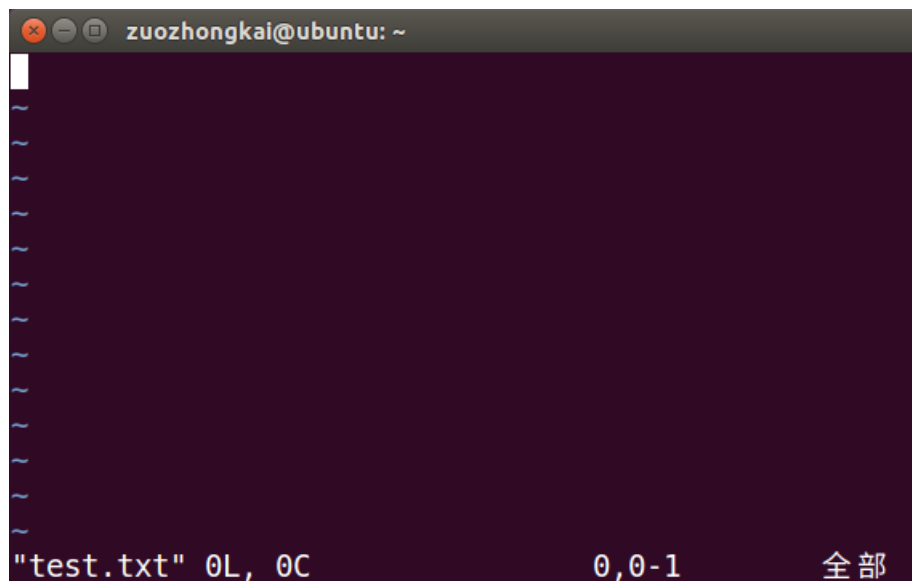


图 2.5.2.2 VIM 打开的 test.txt 文档

我们试着在图 2.5.2.2 中输入数字，发现根本没法输入，这不是因为你的键盘坏了。因为 VIM 默认是以只读模式打开的文档，因此我们要切换到输入模式，切换到输入模式的命令如下：

- i** 在当前光标所在字符的前面，转为输入模式。
- I** 在当前光标所在行的行首转换为输入模式。
- a** 在当前光标所在字符的后面，转为输入模式。
- A** 在光标所在行的行尾，转换为输入模式。
- o** 在当前光标所在行的下方，新建一行，并转为输入模式。
- O** 在当前光标所在行的上方，新建一行，并转为输入模式。
- s** 删除光标所在字符。
- r** 替换光标处字符。

最常用的就是“a”，我们在图 2.5.2.2 中按下键盘上的“a”键，这时候终端左下角会提示“插入”字样，表示我们进入到了输入模式，如图 2.5.2.3 所示：

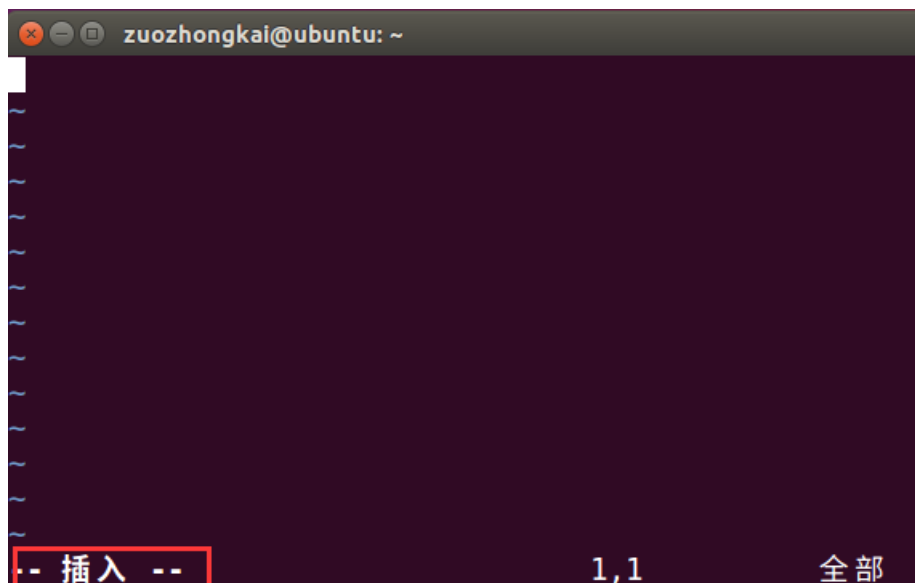


图 2.5.2.3 切换到插入模式

图 2.5.2.3 表明我们可以正常输入文本了，我们可以输入图 2.5.2.4 所示文本：

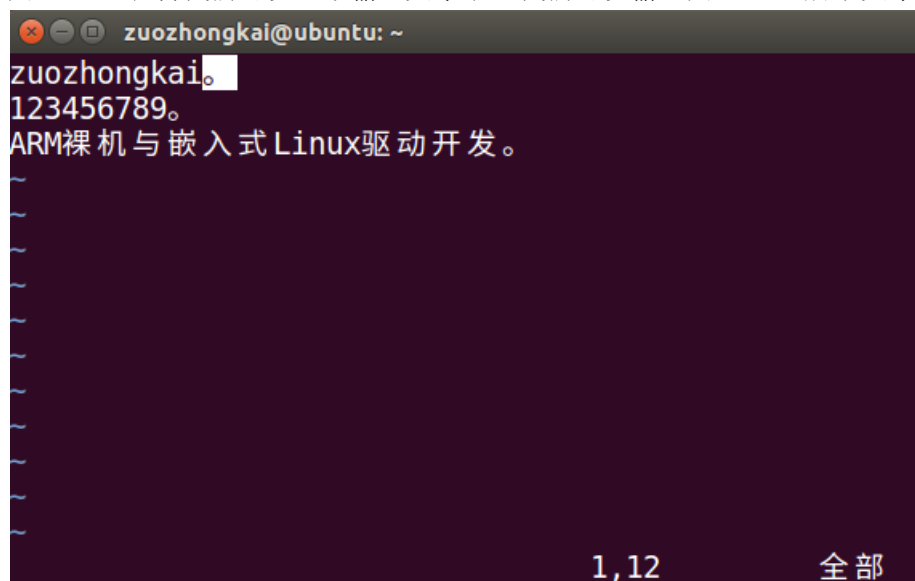


图 2.5.2.4 输入文本

在图 2.5.2.4 中我们在 test.txt 中输入了字母、数字和中文，我们输入完成以后需要保存文本啊，Windows 下的记事本可以使用快捷键 Ctrl+S 来保存，VIM 是否也可以使用 Ctrl+S 来保存呢？你会发现当你按下 Ctrl+S 键以后你的终端不能操作了!!! 这是因为在 Ubuntu 下 Ctrl+S 快捷键不是用来完成保存的功能的，而是暂停该终端！所以你一旦在使用终端的时候按下 Ctrl+S 快捷键，那么你的终端肯定不会再有任何反应，如果你按下 Ctrl+S 关闭了当前终端的话可以按下 Ctrl+Q 来重新打开终端。

既然 Ctrl+S 不能保存文本文档，那么有没有其它方法保存文本文档呢？肯定是有的，我们需要从 VIM 现在的输入模式切换到指令模式，方式就是按下键盘的 ESC 键，按下 ESC 键以后终端左下角的“插入”字样就会消失，此时你就不能在输入任何文本了，如果想再次输入文本的话就按下“a”键重新进入到输入模式。指令模式顾名思义就是输入指令的模式，这些指令是

控制文本的指令，我们将这些指令进行分类，如下所示：

1、移动光标指令：

h(或左方向键)	光标左移一个字符。
l(或右方向键)	光标右移一个字符。
j(或下方向键)	光标下移一行。
k(或上方向键)	光标上移一行。
nG	光标移动到第 n 行首。
n+	光标下移 n 行。
n-	光标上移 n 行。

2、屏幕翻滚指令

Ctrl+f	屏幕向下翻一页，相当于下一页。
Ctrl+b	屏幕向上翻一页，相当于上一页。

3、复制、删除和粘贴指令

cc	删除整行，并且修改整行内容。
dd	删除该行，不提供修改功能。
ndd	删除当前行向下 n 行。
x	删除光标所在的字符。
X	删除光标前面的一个字符。
nyy	复制当前行及其下面 n 行。
p	粘贴最近复制的内容。

上面就是 VI/VIM 的命令模式下最常用的一些命令，还有一些不常用的我没有列出来，感兴趣的可以自行上网查阅。从上面的命令可以看出，并没有保存文本的命令，那是因为保存文档的命令是在底行模式中，我们要先进入到指令模式，进入底行模式的方式是先进入指令模式下，然后在指令模式下输入“:”进入底行模式，如图 2.5.2.5 所示：

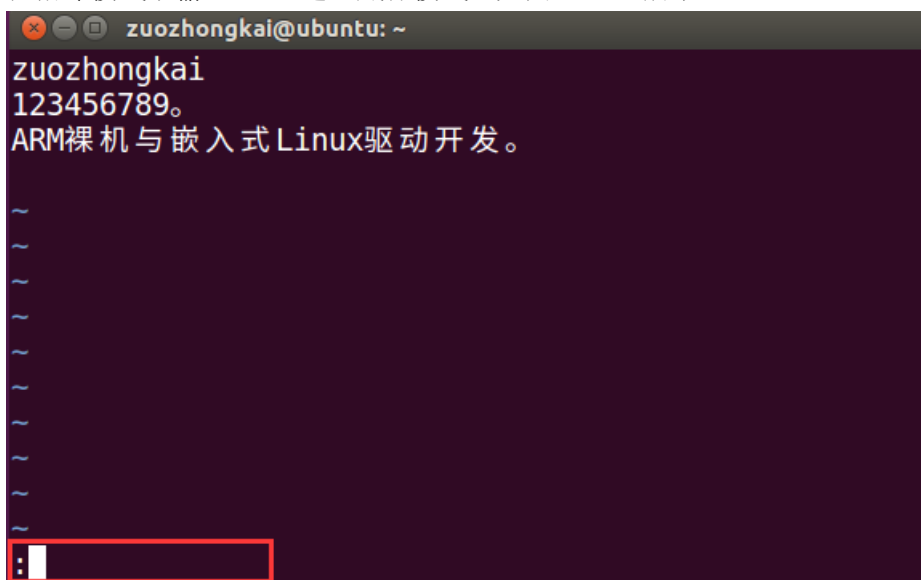


图 2.5.2.5 “:”底行模式

在图 2.5.2.5 中当进入底行模式以后会在终端的左下角就会出现符号“:”，我们可以在“:”后面输入命令，常用的命令如下：

- x** 保存当前文档并且退出。

- q 退出。
- w 保存文档。
- q! 退出 VI/VIM，不保存文档。

如果我们要退出并保存文本的话需要在“:”底行模式下输入“wq”，如图 2.5.2.6 所示：

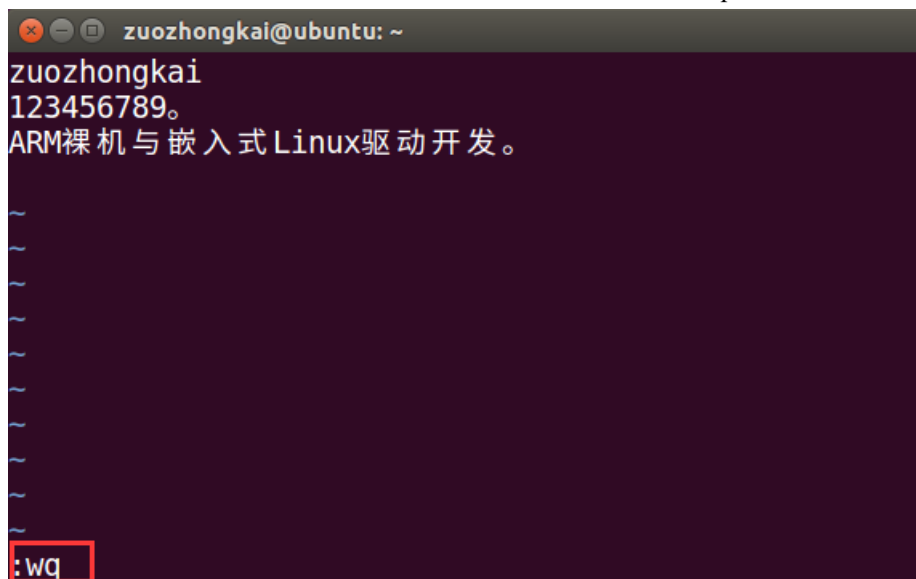


图 2.5.2.6 保存并退出 VIM

在“:”底行模式下输入“wq”以后按下回车键就保存 test.txt 并退出 VI/VIM 编辑器，退出以后我们可以使用命令“cat”来查看刚刚新建的 test.txt 文档的内容，如图 2.5.2.7 所示：

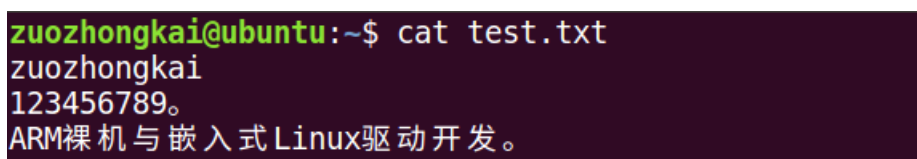


图 2.5.2.7 查看文档内容

从图 2.5.2.7 中可以看出，test.txt 中的内容就是我们用 VIM 输入的内容，至此我们就完整的进行了一遍 VI/VIM 创建文档、编辑文档和保存文档。

在上面讲解进入 VIM 的底行模式的时候之说了在指令模式下输入“:”的方法，还可以在指令模式下输入“/”进入底行模式，输入“/”以后如图 2.5.2.8 所示。

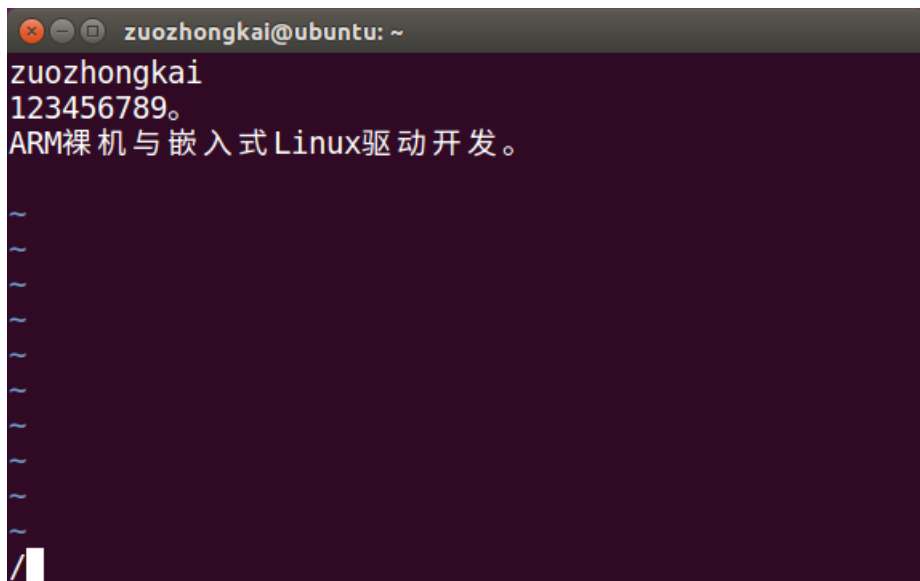


图 2.5.2.8 “/” 底行模式

在“/”底行模式下我们可以在文本中搜索指定的内容，比如搜索 test.txt 文件中“嵌入式”三个字，使用方法如图 2.5.2.9 所示：

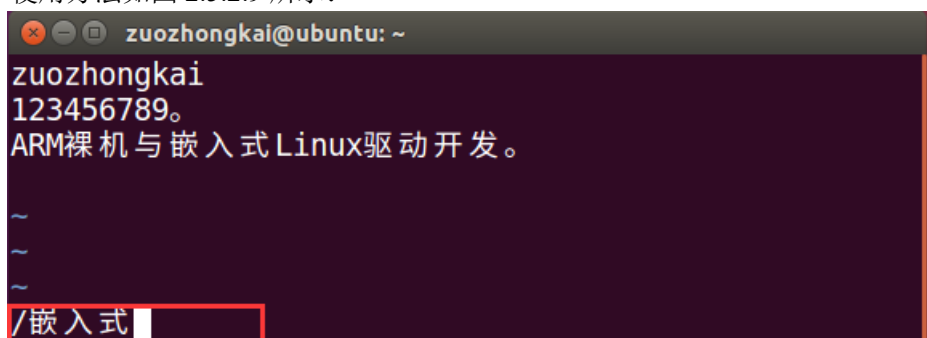


图 2.5.2.9 搜索文本

在“/”后面输入要搜索的内容，然后按下回车键就会在 test.txt 中找到与字符串“嵌入式”匹配的部分，如图 2.5.2.10 所示：

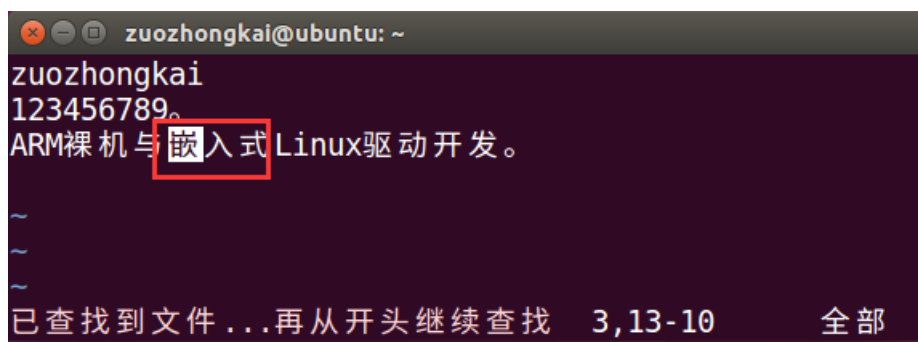


图 2.5.2.10 查找到指定内容

图 2.5.2.10 中可以看出，在 test.txt 中找到了“嵌入式”这个词，并且标记出来位置。我们以后要在一个文档中搜索是否存在某个字符串的时候就可以使用这种方法。有关 VI/VIM 编辑器的讲解就到这里，我们完整的练习了一遍如何使用 VIM 创建文档、编辑文档、保存文档和在文档中搜索字符串。有关更多更详细的 VIM 编辑器的操作大家自行上网查阅相关文档和博客。

2.6 Linux 文件系统

操作系统的基本功能之一就是文件管理，而文件的管理是由文件系统来完成的。Linux 支持多种文件系统，本节我们就来讲解 Linux 下的文件系统、文件系统类型、文件系统结构和文件系统相关 Shell 命令。

2.6.1 Linux 文件系统简介以及类型

1、Linux 文件系统简介

操作系统就是处理各种数据的，这些数据在硬盘上就是二进制，人类肯定不能直接看懂这些二进制数据，要有一个翻译器，将这些二进制的的数据还原为人类能看懂的文件形式，这个工作就是由文件系统来完成的，文件系统的目的就是实现数据的查询和存储，由于使用场合、使用环境的不同，Linux 有多种文件系统，不同的文件系统支持不同的体系。文件系统是管理数据的，而可以存储数据的物理设备有硬盘、U 盘、SD 卡、NAND FLASH、NOR FLASH、网络存储设备等。不同的存储设备其物理结构不同，不同的物理结构就需要不同的文件系统去管理，比如管理 NAND FLASH 的话使用 YAFFS 文件系统，管理硬盘、SD 卡的话就是 ext 文件系统等等。

我们在使用 Windows 的时候新买一个硬盘回来一般肯定是将这个硬盘分为好几个盘，比如 C 盘、D 盘等等。这个叫磁盘的分割，Linux 下也支持磁盘分割，Linux 下常用的磁盘分割工具为：fdisk，fdisk 这个工具我们后面会详细讲解怎么用，因为我们移植 Linux 的时候需要将 SD 卡分为三个分区来存储不同的东西。在 Windows 下我们创建一个新的盘符以后都要做格式化处理，格式化其实就是给这个盘符创建文件系统的过程，我们在 Windows 格式化某个盘的时候都会让你选择文件系统，如图 2.6.1.1 所示：

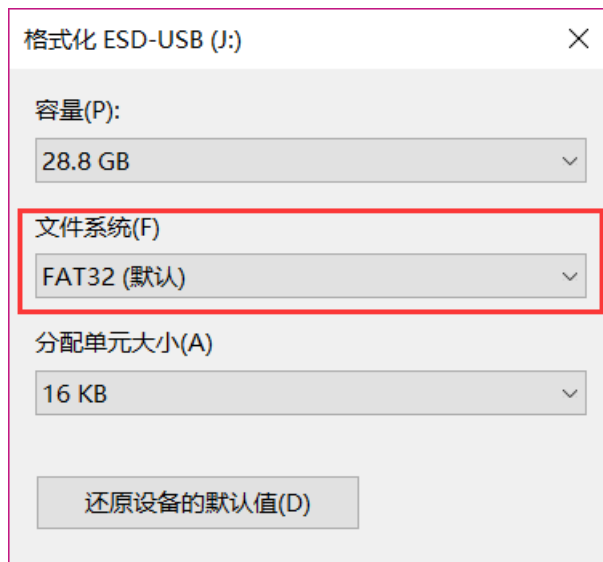


图 2.6.1.1 格式化磁盘

图 2.6.1.1 就是格式化磁盘的时候选择文件系统，Windows 下一般有 FAT、NTFS 和 exFAT 这些文件系统。同样的，在 Linux 下我们使用 fdisk 创建好分区以后也是要先在创建好的分区上面创建文件系统，也就是格式化。

在 Windows 下有磁盘分区概念，比如 C，D，E 盘等，在 Linux 下没有这个概念，因此 Linux 下你找不到像 C、D、E 盘这样的东西。前面我们说了 Linux 下可以给磁盘分割，但是没

有 C、D、E 盘那怎么访问这些分区呢？在 Linux 下创建一个分区并且格式化好以后我们要将其“挂载”到一个目录下才能访问这个分区。Windows 的文件系统挂载过程是其内部完成的，用户是看不到的，Linux 下我们使用 mount 命令来挂载磁盘。挂载磁盘的时候是需要确定挂载点的，也就是你的这个磁盘要挂载到哪个目录下。

2、Linux 文件系统类型

前面我们说了，在 Windows 下有 FAT、NTFS 和 exFAT 这样的文件系统，在 Linux 下又有哪些文件系统呢，Linux 下的文件系统主要有 ext2、ext3、ext4 等文件系统。Linux 还支持其他的 UNIX 文件系统，比如 XFS、JFS、UFS 等，也支持 Windows 的 FAT 文件系统 and 网络文件系统 NFS 等。这里我们主要讲一下 Linux 自带的 ext2、ext3 和 ext4 文件系统。

ext2 文件系统：

ext2 是 Linux 早期的文件系统，但是随着技术的发展 ext2 文件系统已经不推荐使用了，ext2 是一个非日志文件系统，大多数的 Linux 发行版都不支持 ext2 文件系统了。

ext3 文件系统：

ext3 是在 ext2 的基础上发展起来的文件系统，完全兼容 ext2 文件系统，ext3 是一个日志文件系统，ext3 支持大文件，ext3 文件系统的特点有如下：

高可靠性：使用 ext3 文件系统的话，即使系统非正常关机、发生死机等情况，恢复 ext3 文件系统也只需要数十秒。

数据完整性：ext3 提高了文件系统的完整性，避免意外死机或者关机对文件系统的伤害。

文件系统速度：ext3 的日志功能对磁盘驱动器读写头进行了优化，文件系统速度相对与 ext2 来说没有降低。

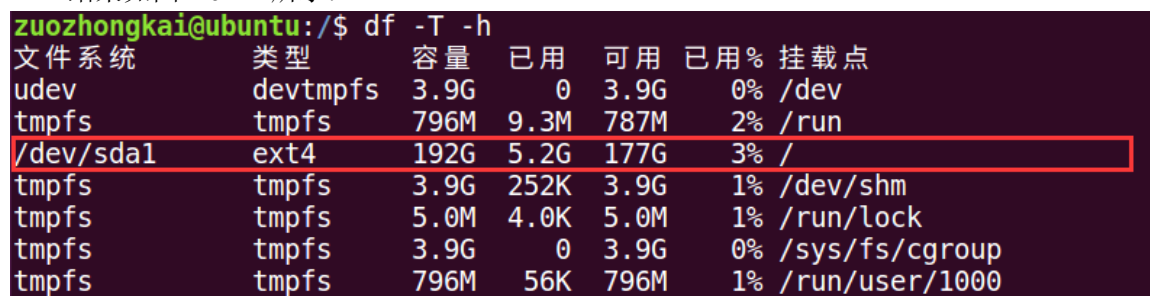
数据转换：从 ext2 转换到 ext3 非常容易，只需要两条指令就可以完成转换。用户不需要花时间去备份、恢复、格式化分区等，用 ext3 文件系统提供的工具 tune2fs 即可轻松的将 ext2 文件系统转换为 ext3 日志文件系统。ext3 文件系统不需要经过任何修改，可以直接挂载成 ext2 文件系统。

ext4 文件系统：

ext4 文件系统是在 ext3 上发展起来的，ext4 相比与 ext3 提供了更佳的性能和可靠性，并且功能更丰富，ext4 向下兼容 ext3 和 ext2，因此可以将 ext2 和 ext3 挂载为 ext4。那么我们安装的 Ubuntu 使用的哪个版本的文件系统呢？在终端中输入如下命令来查询当前磁盘挂载的啥文件系统：

```
df -T -h
```

结果如图 2.6.1.2 所示：



文件系统	类型	容量	已用	可用	已用%	挂载点
udev	devtmpfs	3.9G	0	3.9G	0%	/dev
tmpfs	tmpfs	796M	9.3M	787M	2%	/run
/dev/sda1	ext4	192G	5.2G	177G	3%	/
tmpfs	tmpfs	3.9G	252K	3.9G	1%	/dev/shm
tmpfs	tmpfs	5.0M	4.0K	5.0M	1%	/run/lock
tmpfs	tmpfs	3.9G	0	3.9G	0%	/sys/fs/cgroup
tmpfs	tmpfs	796M	56K	796M	1%	/run/user/1000

图 2.6.1.2 Ubuntu 使用的文件系统

在图 2.6.1.2 中，框起来的就是我们安装 Ubuntu 的这个磁盘，在 Linux 下一切皆为文件，“/dev/sda1”就是我们的磁盘分区，可以看出这个磁盘分区类型是 ext4，它的挂载点是“/”，也

就是根目录。

2.6.2 Linux 文件系统结构

在 Windows 下直接打开 C 盘，我们进入的就是 C 盘的根目录，打开 D 盘进入的就是 D 盘的根目录，比如 C 盘根目录如下：

本地磁盘 (C:)				
名称	修改日期	类型	大小	
\$WINDOWS.~BT	2017-01-18 10:25	文件夹		
\$Windows.~WS	2017-12-17 11:34	文件夹		
AppData	2018-08-27 14:47	文件夹		
DRMsoft	2017-09-07 11:24	文件夹		
ESD	2017-12-17 12:11	文件夹		
Intel	2016-09-07 15:13	文件夹		
M1530_MFP_Series_Basic_Solution	2016-12-29 23:30	文件夹		
PerfLogs	2016-07-16 19:47	文件夹		
Program Files	2017-12-13 15:08	文件夹		
Program Files (x86)	2018-12-18 0:04	文件夹		
ProgramData	2018-12-17 21:57	文件夹		
touchgfx-env	2017-12-05 11:37	文件夹		
TouchGFXProjects	2018-01-07 11:22	文件夹		
uacdump	2016-11-28 12:39	文件夹		
Users	2016-09-08 13:00	文件夹		
Windows	2018-10-31 23:49	文件夹		
InstallConfig.ini	2017-10-13 23:12	配置设置	1 KB	

图 2.6.2.1 C 盘根目录

在 Linux 下因为没有 C、D 盘之说，因此 Linux 只有一个根目录，没有 C 盘根目录、D 盘根目录之类的。其实如果你的 Windows 只有一个 C 盘的话那么整个系统也就只有一个根目录。

Windows 下的 C 盘根目录就是“C:”，在 Linux 下的根目录就是“/”，你没有看错，Linux 根目录就是用“/”来表示的，打开 Ubuntu 的文件浏览器，文件浏览器在左侧的导航栏，图标如图 2.6.2.2 所示：



图 2.6.2.2 文件浏览器

打开以后的文件浏览器如图 2.6.2.3 所示：

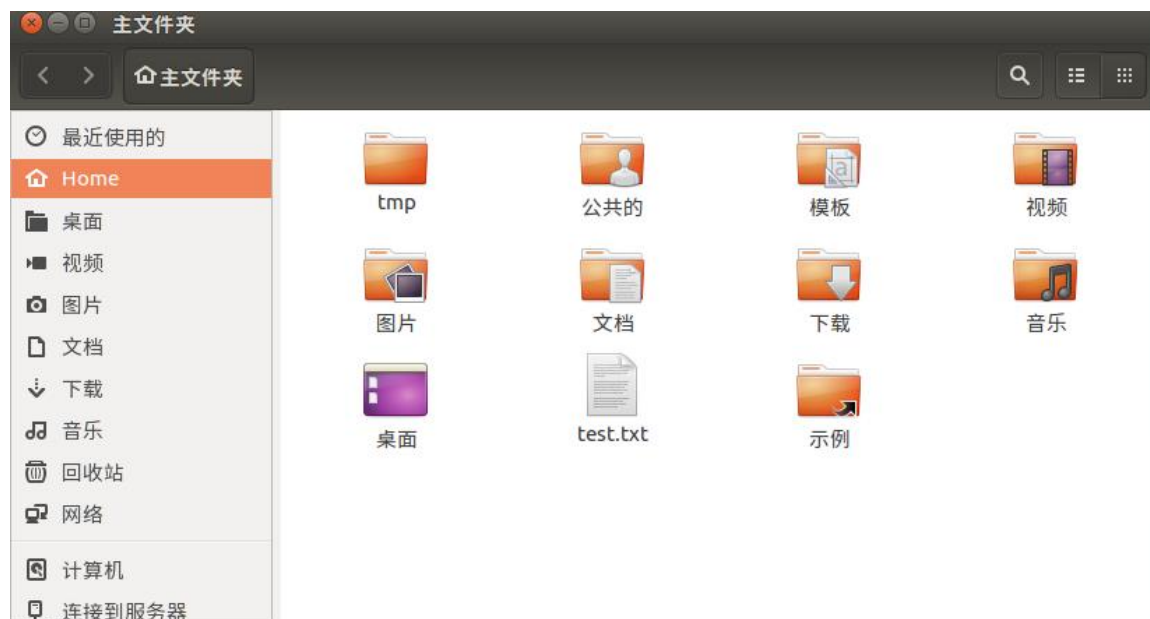


图 2.6.2.3 文件浏览器

直接打开文件浏览器以后，我们默认不是处于根目录中的，不像 Windows，我们直接打开 C 盘就处于 C 盘根目录下。Ubuntu 是支持多用户的，Ubuntu 为每个用户创建了一个根目录，比如我电脑现在登陆的是“zuozhongkai”这个用户，因此默认进入的是“zuozhongkai”这个用户的根目录。我们点击图 2.6.2.3 中左侧的“计算机”，打开以后如图 2.6.2.4 所示：



图 2.6.2.4 根目录“/”

图 2.6.2.4 就是 Ubuntu 的根目录“/”，这时候肯定就有人有疑问，刚刚说 Ubuntu 会给每个

用户创建一个根目录，那这些用户的根目录在哪里？是不是和根目录“/”是一个地位的？其实所谓的给每个用户创建一个根目录只是方便说而已，这个所谓的用户根目录其实就是“/”下的一个文件夹，以我的“zuozhongkai”这个用户为例，其用户根目录就是：/home/zuozhongkai。只要你创建了一个用户，那么系统就会在/home 这个目录下创建一个以这个用户名命名的文件夹，这个文件夹就是这个用户的根目录。

用户可以对自己的用户根目录下的文件进行随意的读写操作，但是如果修改根目录“/”下的文件就会提示没有权限。打开终端以后默认进入的是当前用户根目录，比如我们打开终端以后输入“ls”命令查看当前目录下有什么文件，结果如图 2.6.2.5 所示：

```
zuozhongkai@ubuntu:~$ ls
examples.desktop  tmp      模板  图片  下载  桌面
test.txt          公共的  视频  文档  音乐
```

图 2.6.2.5 目录查看

可以看出图 2.6.2.5 中的文件和图 2.6.2.3 中的一模一样，都是“zuozhongkai”这个账户的根目录。我们来看一下根目录“/”下都有哪些文件，在终端中输入如下命令：

```
cd / //进入到根目录“/”
ls //查看根目录“/”下的文件以及文件夹
```

执行上述两行命令以后，终端如图 2.6.2.6 所示：

```
zuozhongkai@ubuntu:~$ cd /
zuozhongkai@ubuntu:/$ ls
bin      dev      initrd.img  lib64      mnt      root     snap      tmp      vmlinuz
boot     etc      initrd.img.old  lost+found  opt      run      srv      usr      vmlinuz.old
cdrom    home     lib         media      proc     sbin     sys      var
```

图 2.6.2.6 查看根目录“/”

图 2.6.2.6 中列举出了根目录“/”下面的所有文件夹，这里我们仔细观察一下，当我们进入到根目录“/”里面以后终端提示符“\$”前面的符号“~”变成了“/”，这是因为当我们在终端中切换了目录以后“\$”前面就会显示切换以后的目录路径。我们来看一下根目录“/”中的一些重要的文件夹：

- /bin** 存储一些二进制可执行命令文件，/usr/bin 也存放了一些基于用户的命令文件。
- /sbin** 存储了很多系统命令，/usr/sbin 也存储了许多系统命令。
- /root** 超级用户 root 的根目录文件。
- /home** 普通用户默认目录，在该目录下，每个用户都有一个以本用户名命名的文件夹。
- /boot** 存放 Ubuntu 系统内核和系统启动文件。
- /mnt** 通常包括系统引导后被挂载的文件系统的挂载点。
- /dev** 存放设备文件，我们后面学习 Linux 驱动主要是跟这个文件夹打交道的。
- /etc** 保存系统管理所需的配置文件和目录。
- /lib** 保存系统程序运行所需的库文件，/usr/lib 下存放了一些用于普通用户的库文件。
- /lost+found** 一般为空，当系统非正常关机以后，此文件夹会保存一些零散文件。
- /var** 存储一些不断变化的文件，比如日志文件
- /usr** 包括与系统用户直接有关的文件和目录，比如应用程序和所需的库文件。
- /media** 存放 Ubuntu 系统自动挂载的设备文件。
- /proc** 虚拟目录，不实际存储在磁盘上，通常用来保存系统信息和进程信息。
- /tmp** 存储系统和用户的临时文件，该文件夹对所有的用户都提供读写权限。
- /opt** 可选文件和程序的存放目录。
- /sys** 系统设备和文件层次结构，并向用户程序提供详细的内核数据信息。

2.6.2 文件操作命令

本节我们来学习一下在终端进行文件操作的一些常用命令:

1、创建新文件命令—touch

在前面学习 VIM 的时候我们知道可以用 vi 指令来创建一个文本文档, 本节我们就学习一个功能更全面的文件创建命令—touch。touch 不仅仅可以用来创建文本文档, 其它类型的文档也可以创建, 命令格式如下:

```
touch [参数] [文件名]
```

使用 touch 创建文件的时候, 如果[文件名]的文件不存在, 那就直接创建一个以[文件名]命名的文件, 如果[文件名]文件存在的话就仅仅修改一下此文件的最后修改日期, 常用的命令参数如下:

- a 只更改存取时间。
- c 不建立任何文件。
- d<日期> 使用指定的日期, 而并非现在日期。
- t<时间> 使用指定的时间, 而并非现在时间。

进入到用户根目录下, 直接使用命令“cd ~”即可快速进入用户根目录, 进入用户根目录以后使用 touch 命令创建一个名为 test 的文件, 创建过程如图 2.6.2.1 所示:

```
zuozhongkai@ubuntu:~$ cd ~ //进入用户根目录
zuozhongkai@ubuntu:~$ ls //查看当前目录下的文件
examples.desktop tmp 模板 图片 下载 桌面
test.txt 公共的 视频 文档 音乐
zuozhongkai@ubuntu:~$ touch test //创建test文件
zuozhongkai@ubuntu:~$ ls test //查看创建的test文件
test
zuozhongkai@ubuntu:~$ ls test -l //查看创建的test文件详细信息
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 22 01:24 test
```

图 2.6.2.1 touch 命令操作

2、文件夹创建命令—mkdir

既然可以创建文件, 那么肯定也可以创建文件夹, 创建文件夹使用命令“mkdir”, 命令格式如下:

```
mkdir [参数] [文件夹名目录名]
```

主要参数如下:

- p 如所要创建的目录其上层目录目前还未创建, 那么会一起创建上层目录。

我们在用户根目录下创建两个分别名为“testdir1”和“testdir2”的文件夹, 操作如图 2.6.2.2 所示:

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的文件
examples.desktop test.txt 公共的 视频 文档 音乐
test tmp 模板 图片 下载 桌面
zuozhongkai@ubuntu:~$ mkdir testdir1 //创建testdir1文件夹
zuozhongkai@ubuntu:~$ mkdir testdir2 //创建testdir2文件夹
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件, 看看文件夹创建是否成功
examples.desktop testdir1 test.txt 公共的 视频 文档 音乐
test testdir2 tmp 模板 图片 下载 桌面
```

图 2.6.2.2 创建文件夹

在图 2.6.2.2 中, 我们使用命令“mkdir”创建了“testdir1”和“testdir2”这两个文件夹。

3、文件及目录删除命令—rm

既然有创建文件的命令, 那肯定有删除文件的命令, 要删除一个文件或者文件夹可以使用命令“rm”, 此命令可以完成删除一个文件或者多个文件及文件夹, 它可以实现递归删除。对于链接文件, 只删除链接, 原文件保持不变, 所谓的链接文件, 其实就是 Windows 下的快捷方式文件, 此命令格式如下:

```
rm [参数] [目的文件或文件夹目录名]
```

命令主要参数如下:

- d 直接把要删除的目录的硬连接数据删成 0, 删除该目录。
- f 强制删除文件和文件夹(目录)。
- i 删除文件或者文件夹(目录)之前先询问用户。
- r 递归删除, 指定文件夹(目录)下的所有文件和子文件夹全部删除掉。
- v 显示删除过程。

我们使用 rm 命令来删除前面使用命令“touch”创建的 test 文件, 操作过程如图 2.6.2.3 所示:

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop testdir1 test.txt 公共的 视频 文档 音乐
test             testdir2 tmp      模板 图片 下载 桌面
zuozhongkai@ubuntu:~$ rm test //删除文件test
zuozhongkai@ubuntu:~$ ls //查看当前目录, 看看test文件是否删除
examples.desktop testdir2 tmp      模板 图片 下载 桌面
testdir1         test.txt 公共的 视频 文档 音乐
```

图 2.6.2.3 删除文件

命令“rm”也可以直接删除文件夹, 我们可以试一下删除前面创建的 testdir1 文件夹, 先直接使用命令“rm testdir1”测试一下是否可以删除, 结果如图 2.6.2.4 所示:

```
zuozhongkai@ubuntu:~$ rm testdir1
rm: 无法删除 'testdir1': 是一个目录
```

图 2.6.2.4 删除文件夹

在图 2.6.2.4 中可以看出, 直接使用命令“rm”是无法删除文件夹(目录)的, 我们需要加上参数“-rf”, 也就是强制递归删除文件夹(目录), 操作结果如图 2.6.2.5 所示:

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop testdir2 tmp      模板 图片 下载 桌面
testdir1         test.txt 公共的 视频 文档 音乐
zuozhongkai@ubuntu:~$ rm testdir1 -rf //使用参数“-rf”强制递归删除文件夹
zuozhongkai@ubuntu:~$ ls //查看删除文件夹以后的当前目录
examples.desktop test.txt 公共的 视频 文档 音乐
testdir2         tmp      模板 图片 下载 桌面
```

图 2.6.2.5 带参数删除文件夹

从图 2.6.2.5 可以看出, 当在命令“rm”中加入参数“-rf”以后就可以删除掉文件夹“testdir1”了。

4、文件夹(目录)删除命令—rmdir

上面我们讲解了如何使用命令“rm”删除文件夹, 那就是要加上参数“-rf”, 其实 Linux 提供了直接删除文件夹(目录)的命令—rmdir, 它可以不加任何参数的删除掉指定的文件夹(目录), 命令格式如下:

`rmdir` [参数] [文件夹(目录)]

命令主要参数如下:

-p 删除指定的文件夹(目录)以后,若上层文件夹(目录)为空文件夹(目录)的话就将其一起删除。

我们使用命令“`rmdir`”删除掉前面创建的“`testdir2`”文件夹,操作过程如图 2.6.2.6 所示:

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop  test.txt  公共的  视频  文档  音乐
testdir2          tmp      模板  图片  下载  桌面
zuozhongkai@ubuntu:~$ rmdir testdir2 //删除文件夹testdir2
zuozhongkai@ubuntu:~$ ls //查看删除文件夹以后的目录文件
examples.desktop  tmp      模板  图片  下载  桌面
test.txt          公共的  视频  文档  音乐
```

图 2.6.2.6 命令 `rmdir` 删除文件夹

5、文件复制命令—`cp`

在 Windows 下我们可以通过在文件上点击鼠标右键来进行文件的复制和粘贴,在 Ubuntu 下我们也可以通过点击文件右键进行文件的复制和粘贴。但是本节我们来讲解如何在终端下使用命令来进行文件的复制, Linux 下的复制命令为“`cp`”,命令描述如下:

`cp` [参数] [源地址] [目的地址]

主要参数描述如下:

- a** 此参数和同时指定“`-dpR`”参数相同
- d** 在复制有符号连接的文件时,保留原始的连接。
- f** 强行复制文件,不管要复制的文件是否已经存在于目标目录。
- I** 覆盖现有文件之前询问用户。
- p** 保留源文件或者目录的属性。
- r 或 -R** 递归处理,将指定目录下的文件及子目录一并处理

我们在用户根目录下,使用前面讲解的命令“`mkdir`”创建两个文件夹: `test1` 和 `test2`,过程如图 2.6.2.7 所示。

```
zuozhongkai@ubuntu:~$ ls //查看当前目录下的文件
examples.desktop  tmp      模板  图片  下载  桌面
test.txt          公共的  视频  文档  音乐
zuozhongkai@ubuntu:~$ mkdir test1 test2 //创建test1和test2两个文件夹
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop  test2    tmp      模板  图片  下载  桌面
test1            test.txt 公共的  视频  文档  音乐
```

图 2.6.2.7 创建 `test1` 和 `test2` 两个文件夹

进入上面创建的 `test1` 文件夹,然后在 `test1` 文件夹里面创建一个 `a.c` 文件,操作过程如图 2.6.2.8 所示:

```
zuozhongkai@ubuntu:~$ cd test1 //进入test1文件夹
zuozhongkai@ubuntu:~/test1$ touch a.c //创建a.c文件
zuozhongkai@ubuntu:~/test1$ ls //查看当前目录下所有文件
a.c
```

图 2.6.2.8 创建 `a.c` 文件

我们先将图 2.6.2.8 中的 `a.c` 这个文件做个备份,也就是复制到同文件夹 `test1` 里面,新的文件命名为 `b.c`。然后在将 `test1` 文件夹中的 `a.c` 和 `b.c` 这两个文件都复制到文件夹 `test2` 中,操作

如图 2.6.2.9 所示

```

zuozhongkai@ubuntu:~/test1$ cp a.c b.c //拷贝a.c到本文件夹中，并重命名为b.c
zuozhongkai@ubuntu:~/test1$ ls //查看当前目录下的所有文件
a.c b.c
zuozhongkai@ubuntu:~/test1$ cp *.c ../test2 //拷贝a.c和b.c到文件夹test2中
zuozhongkai@ubuntu:~/test1$ cd ../test2 //进入上级目录中的test2文件夹
zuozhongkai@ubuntu:~/test2$ ls //查看test2文件夹下的文件
a.c b.c

```

图 2.6.2.9 拷贝文件

在图 2.6.2.9 中，我们添加了一些高级使用技巧，首先是拷贝 a.c 和 b.c 文件到 test2 文件夹中，我们使用了通配符“*”，“*.c”就表示 test1 下的所有以“.c”结尾的文件，也就是 a.c 和 b.c。

“../test2”中的“../”表示上级目录，因此“../test2”就是上级目录下的 test2 文件夹。

上面都是文件复制，我们接下来学习一下文件夹复制，我们将 test2 文件夹复制到同目录下，新拷贝的文件夹命名为 test3，操作如图 2.6.2.10 所示：

```

zuozhongkai@ubuntu:~$ cp -rf test2/ test3/ //复制test2文件夹为test3文件夹
zuozhongkai@ubuntu:~$ ls //查看当前目录下的所有文件
examples.desktop test2 test.txt 公共的 视频 文档 音乐
test1 test3 tmp 模板 图片 下载 桌面
zuozhongkai@ubuntu:~$ cd test3 //进入test3文件夹
zuozhongkai@ubuntu:~/test3$ ls //查看test3中的所有文件
a.c b.c

```

图 2.6.2.10 拷贝文件夹

6、文件移动命令—mv

有时候我们需要将一个文件或者文件夹移动到另外一个地方去，或者给一个文件或者文件夹进行重命名，这个时候我们就可以使用命令“mv”了，此命令格式如下：

```
mv [参数] [源地址] [目的地址]
```

主要参数描述如下：

- b 如果要覆盖文件的话覆盖前先进行备份。
- f 若目标文件或目录与现在的文件重复，直接覆盖目的文件或目录。
- I 在覆盖之前询问用户。

使用上面讲解“cp”命令的时候创建了三个文件夹，在上面创建的 test1 文件夹里面创建一个 c.c 文件，然后将 c.c 这个文件重命名为 d.c。最后将 d.c 这个文件移动到 test2 文件夹里面，操作如图 2.6.2.11 所示：

```

zuozhongkai@ubuntu:~$ cd test1 //进入test1文件夹
zuozhongkai@ubuntu:~/test1$ touch c.c //创建c.c文件
zuozhongkai@ubuntu:~/test1$ ls //查看当前目录下的所有文件
a.c b.c c.c
zuozhongkai@ubuntu:~/test1$ mv c.c d.c //移动c.c到当前目录下，并重命名为d.c
zuozhongkai@ubuntu:~/test1$ ls //查看当前目录下的所有文件
a.c b.c d.c

```

图 2.6.2.11 移动文件操作

我们再将 test1 中的 d.c 文件移动到 test2 文件夹里面，操作如图 2.6.2.12 所示：

```
zuozhongkai@ubuntu:~/test2$ ls //查看test2下所有文件
a.c b.c
zuozhongkai@ubuntu:~/test2$ cd ../test1 //进入test1文件夹中
zuozhongkai@ubuntu:~/test1$ ls //查看test1文件夹所有文件
a.c b.c d.c
zuozhongkai@ubuntu:~/test1$ mv d.c ../test2 //将d.c移动到test2中
zuozhongkai@ubuntu:~/test1$ ls ../test2 //查看test2下的所有文件
a.c b.c d.c
```

图 2.6.2.12 移动文件操作

2.6.3 文件压缩和解压缩

文件的压缩和解压缩是非常常见的操作，在 Windows 下我们有很多压缩和解压缩的工具，比如 zip、360 压缩等等。在 Ubuntu 下也有压缩工具，本节我们学习 Ubuntu 下图形化以及命令行这两种压缩和解压缩操作。

1、图形化压缩和解压缩

图形化压缩和解压缩和 Windows 下基本一样，在要压缩或者解压的文件上点击鼠标右键，然后选择要进行的操作，我们先讲解一下如何进行文件的压缩。首先找到要压缩的文件，然后在要压缩的文件上点击鼠标右键，选择“压缩”选项，如图 2.6.3.1 所示：



图 2.6.3.1 文件压缩

在图 2.6.3.1 中我们要对 test2 这个文件夹进行压缩，点击“压缩”以后会弹出图 2.6.3.2 所

示界面让选择压缩后的文件名和压缩格式。

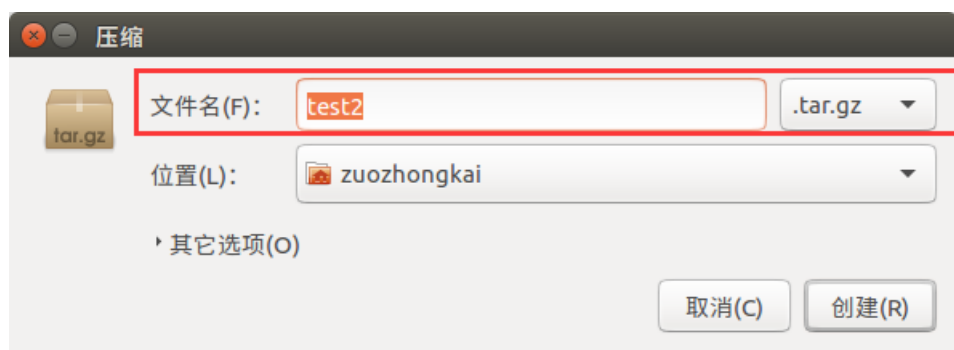


图 2.6.3.2 压缩命名和格式选择

在图 2.6.3.2 中, 设置好压缩以后的文件名, 然后选择压缩格式, 可选的压缩格式如图 2.6.3.3 所示:

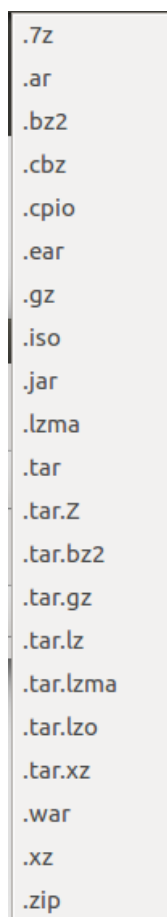


图 2.6.3.3 可选压缩格式

从图 2.6.3.3 中可以看出, 可以选择的压缩格式还是有很多的, 挑选一个格式进行压缩, 比如我选择的“.zip”这个格式, 压缩完成以后如图 2.6.3.4 所示:

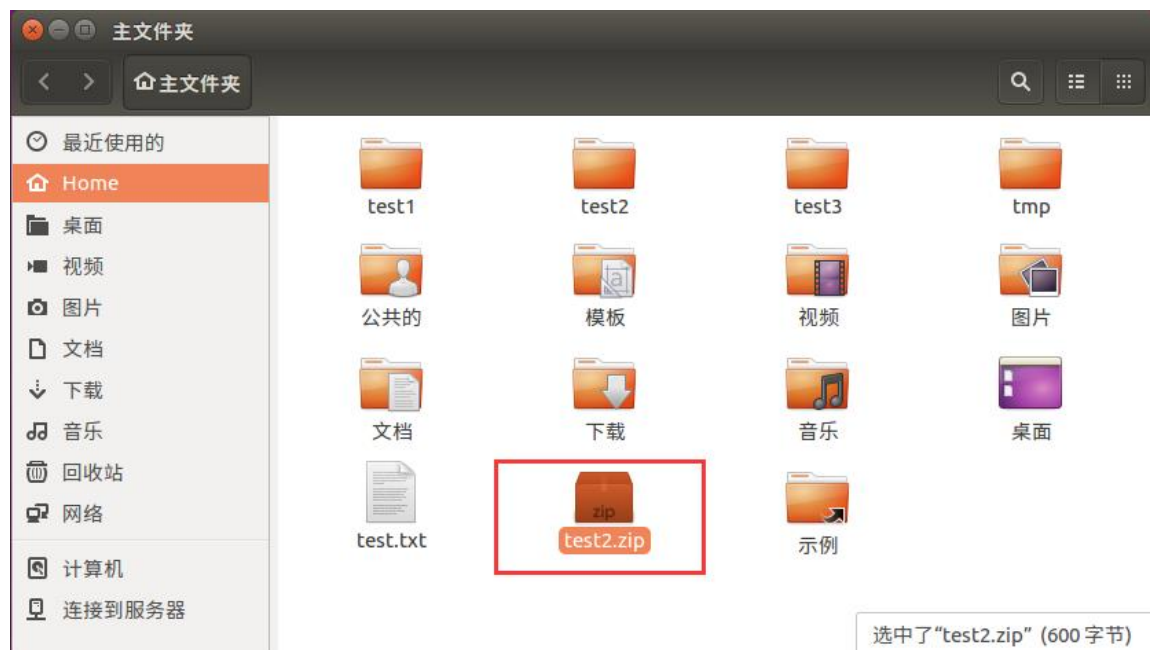


图 2.6.3.4 压缩完成的文件

上面就是使用图形化进行文件压缩的过程，我们接下来对刚刚压缩的 test2.zip 进行解压缩，鼠标放到 test2.zip 上然后点击鼠标右键，选择“提取到此处”，如图 2.6.3.5 所示：



图 2.6.3.5 解压缩文件

点击图 2.6.3.5 中的“提取到此处”以后，系统就会自动进行解压缩，上面就是在 Ubuntu 中使用图形化工具进行文件的压缩和解压缩。

2、命令行进行文件的压缩和解压缩

上面我们学习了如何使用图形化工具在 Ubuntu 下进行文件的压缩和解压缩，本节我们学习如何使用命令行进行压缩和解压缩，我们后面的开发中所有涉及到压缩和解压缩的操作都是

在命令行下完成的。命令行下进行压缩和解压缩常用的命令有三个：zip、unzip 和 tar，我们依次来学习：

3、命令 zip

zip 命令看名字就知道是针对 .zip 文件的，用于将一个或者多个文件压缩成一个 .zip 结尾的文件，命令格式如下：

```
zip [参数] [压缩文件名.zip] [被压缩的文件]
```

主要参数函数如下：

- b<工作目录> 指定暂时存放文件的目录。
- d 从 zip 文件中删除一个文件。
- F 尝试修复已经损毁的压缩文件。
- g 将文件压缩入现有的压缩文件中，不需要新建压缩文件。
- h 帮助。
- j 只保存文件的名，不保存目录。
- m 压缩完成以后删除源文件。
- n<字尾符号> 不压缩特定扩展名的文件。
- q 不显示压缩命令执行过程。
- r 递归压缩，将指定目录下的所有文件和子目录一起压缩。
- v 显示指令执行过程。
- num 压缩率，为 1~9 的数值。

上面讲解了如何使用图形化压缩工具对文件夹 test2 进行压缩，这里我们使用命令“zip”对 test2 文件夹进行压缩，操作如图 2.6.3.6 所示：

```

zuozhongkai@ubuntu:~$ ls //显示当前目录下所有文件
examples.desktop  test2  test.txt  公共的  视频  文档  音乐
test1             test3  tmp       模板   图片  下载  桌面
zuozhongkai@ubuntu:~$ zip -rv test2.zip test2 //使用zip命令进行压缩
adding: test2/          (in=0) (out=0) (stored 0%)
adding: test2/d.c       (in=0) (out=0) (stored 0%)
adding: test2/b.c       (in=0) (out=0) (stored 0%)
adding: test2/a.c       (in=0) (out=0) (stored 0%)
total bytes=0, compressed=0 -> 0% savings
zuozhongkai@ubuntu:~$ ls //显示压缩以后的当前目录下所有文件
examples.desktop  test2  test3  tmp  模板  图片  下载  桌面
test1            test2.zip  test.txt  公共的  视频  文档  音乐

```

图 2.6.3.6 使用 zip 进行文件压缩

图 2.6.3.6 就是使用 zip 命令进行 test2 文件夹的压缩，我们使用的命令如下：

```
zip -rv test2.zip test2
```

上述命令中，-rv 表示递归压缩并且显示压缩命令执行过程。

4、命令 unzip

unzip 命令用于对 .zip 格式的压缩包进行解压，命令格式如下：

```
unzip [参数] [压缩文件名.zip]
```

主要参数如下：

- l 显示压缩文件内所包含的文件。
- t 检查压缩文件是否损坏，但不解压。
- v 显示命令显示的执行过程。
- Z 只显示压缩文件的注解。
- C 压缩文件中的文件名称区分大小写。

- j 不处理压缩文件中的原有目录路径。
- L 将压缩文件中的全部文件名改为小写。
- n 解压缩时不要覆盖原有文件。
- P<密码> 解压密码。
- q 静默执行，不显示任何信息。
- x<文件列表> 指定不要处理.zip 中的哪些文件。
- d<目录> 把压缩文件解到指定目录下。

对上面压缩的 test2.zip 文件使用 unzip 命令进行解压缩，操作如图 2.6.3.7 所示：

```

zuozhongkai@ubuntu:~$ rm -rf test2 //删除以前的test2文件夹，防止干扰解压过程
zuozhongkai@ubuntu:~$ ls //显示当前目录下所有文件
examples.desktop  test2.zip  test.txt  公共的  视频  文档  音乐
test1             test3      tmp       模板  图片  下载  桌面
zuozhongkai@ubuntu:~$ unzip test2.zip //解压test2.zip文件
Archive: test2.zip
  creating: test2/
  extracting: test2/d.c
  extracting: test2/b.c
  extracting: test2/a.c
zuozhongkai@ubuntu:~$ ls //显示当前目录下所有文件
examples.desktop  test2      test3      tmp       模板  图片  下载  桌面
test1             test2.zip  test.txt   公共的  视频  文档  音乐

```

图 2.6.3.7 命令 unzip 使用演示

5、命令 tar

我们前面讲的 zip 和 unzip 这两个命令只适用于.zip 格式的压缩和解压，其它压缩格式就用不了了，比如 Linux 下最常用的.bz2 和.gz 这两种压缩格式。其它格式的压缩和解压使用命令 tar，tar 将压缩和解压缩集合在一起，使用不同的参数即可，命令格式如下：

```
tar [参数] [压缩文件名] [被压缩文件名]
```

常用参数如下：

- c 创建新的压缩文件。
- C<目的目录> 切换到指定的目录。
- f<备份文件> 指定压缩文件。
- j 用 tar 生成压缩文件，然后用 bzip2 进行压缩。
- k 解开备份文件时，不覆盖已有的文件。
- m 还原文件时，不变更文件的更改时间。
- r 新增文件到已存在的备份文件的结尾部分。
- t 列出备份文件内容。
- v 显示指令执行过程。
- w 遇到问题时先询问用户。
- x 从备份文件中释放文件，也就是解压缩文件。
- z 用 tar 生成压缩文件，用 gzip 压缩。
- Z 用 tar 生成压缩文件，用 compress 压缩。

使用 tar 命令来进行.zip 和.gz 格式的文件压缩，操作如图 2.6.3.8 所示：

```

zuozhongkai@ubuntu:~$ ls //显示当前目录下的所有文件
examples.desktop test2 test.txt 公共的 视频 文档 音乐
test1 test3 tmp 模板 图片 下载 桌面
zuozhongkai@ubuntu:~$ tar -vcjf test1.tar.bz2 test1 //压缩为.bz2格式
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls //查看压缩后的文件是否存在
examples.desktop test1.tar.bz2 test3 tmp 模板 图片 下载 桌面
test1 test2 test.txt 公共的 视频 文档 音乐
zuozhongkai@ubuntu:~$ tar -vczf test1.tar.gz test1 //压缩为.gz格式
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls //查看压缩后的文件是否存在
examples.desktop test1.tar.bz2 test2 test.txt 公共的 视频 文档 音乐
test1 test1.tar.gz test3 tmp 模板 图片 下载 桌面

```

图 2.6.3.8 tar 命令进行压缩

在图 2.6.3.8 中，我们使用如下两个命令将 test1 文件夹压缩为.bz2 和.gz 这两个格式：

```

tar -vcjf test1.tar.bz2 test1
tar -vczf test1.tar.gz test1

```

在上面两行命令中，-vcjf 表示创建 bz2 格式的压缩文件，-vczf 表示创建.gz 格式的压缩文件。学习了如何使用 tar 命令来完成压缩，我们再来学习使用 tar 命令完成文件的解压，操作如图 2.6.3.9 所示：

```

zuozhongkai@ubuntu:~$ ls //显示当前目录下所有文件
examples.desktop test2.tar.gz test.txt 公共的 视频 文档 音乐
test1.tar.bz2 test3 tmp 模板 图片 下载 桌面
zuozhongkai@ubuntu:~$ tar -vxjf test1.tar.bz2 //解压缩.bz2格式文件
test1/
test1/c.c
test1/b.c
test1/a.c
zuozhongkai@ubuntu:~$ ls //检查test1.tar.bz2是否解压缩成功
examples.desktop test1.tar.bz2 test3 tmp 模板 图片 下载 桌面
test1 test2.tar.gz test.txt 公共的 视频 文档 音乐
zuozhongkai@ubuntu:~$ tar -vxzf test2.tar.gz //解压缩.gz格式文件
test2/
test2/d.c
test2/b.c
test2/a.c
zuozhongkai@ubuntu:~$ ls //检查test2.tar.gz是否解压缩成功
examples.desktop test2 test.txt 模板 文档 桌面
test1 test2.tar.gz tmp 视频 下载
test1.tar.bz2 test3 公共的 图片 音乐

```

图 2.6.3.9 tar 解压缩命令

图 2.6.3.9 中我们使用如下所示两行命令完成.bz2 和.gz 格式文件的解压缩：

```

tar -vxjf test1.tar.bz2
tar -vxzf test2.tar.gz

```

上述两行命令中，-vxjf 用来完成.bz2 格式压缩文件的解压，-vxzf 用来完成.gz 格式压缩文件的解压。关于 Ubuntu 下的命令行压缩和解压缩就讲解到这里，重点是 tar 命令，要熟练掌握使用 tar 命令来完成.bz2 和.gz 格式的文件压缩和解压缩。

2.6.4 文件查询和搜索

文件的查询和搜索也是最常用的操作，在嵌入式 Linux 开发中常常需要在 Linux 源码文件中查询某个文件是否存在，或者搜索哪些文件都调用了某个函数等等。本节我们就讲解两个最常用的文件查询和搜索命令：`find` 和 `grep`。

1、命令 `find`

`find` 命令用于在目录结构中查找文件，其命令格式如下：

```
find [路径] [参数] [关键字]
```

路径是要查找的目录路径，如果不写的话表示在当前目录下查找，关键字是文件名的一部分，主要参数如下：

-name<filename> 按照文件名称查找，查找与 `filename` 匹配的文件，可使用通配符。

-depth 从指定目录下的最深层的子目录开始查找。

-gid<群组识别码> 查找符合指定的群组识别码的文件或目录。

-group<群组名称> 查找符合指定的群组名称的文件或目录。

-size<文件大小> 查找符合指定文件大小的文件。

-type<文件类型> 查找符合指定文件类型的文件。

-user<拥有者名称> 查找符合指定的拥有者名称的文件或目录。

`find` 命令的参数有很多，常用的就这些，关于其它的参数大家可以自行上网查找，我们来看一下如何使用 `find` 命令进行文件搜索，我们搜索目录 `/etc` 中以 “`vim`” 开头的文件为例，操作如图 2.6.4.1 所示：

```
zuozhongkai@ubuntu:~$ find /etc/ -name vim*
/etc/vim
/etc/vim/vimrc
/etc/vim/vimrc.tiny
find: `/etc/cups/ssl': 权限不够
/etc/alternatives/vim
/etc/alternatives/vimdiff
find: `/etc/polkit-1/localauthority': 权限不够
find: `/etc/ssl/private': 权限不够
```

图 2.6.4.1 `find` 命令操作

从图 2.6.4.1 可以看出，在目录 `/etc` 下，包含以 “`vim*`” 开头的文件有 `/etc/vim`、`/etc/vim/vimrc` 等等，就不一一列出了。

2、命令 `grep`

`find` 命令用于在目录中搜索文件，我们有时候需要在文件中搜索一串关键字，`grep` 就是完成这个功能的，`grep` 命令用于查找包含指定关键字的文件，如果发现某个文件的内容包含所指定的关键字，`grep` 命令就会把包含指定关键字的这一行标记出来，`grep` 命令格式如下：

```
grep [参数] 关键字 文件列表
```

`grep` 命令一次只能查一个关键字，主要参数如下：

-b 在显示符合关键字的那一列前，标记处该列第 1 个字符的位编号。

-c 计算符合关键字的列数。

-d<进行动作> 当指定要查找的是目录而非文件时，必须使用此参数！否则 `grep` 指令将回报信息并停止搜索。

-i 忽略字符大小写。

-v 反转查找，只显示不匹配的行。

-r 在指定目录中递归查找。

比如我们在目录/usr下递归查找包含字符“Ubuntu”的文件，操作如图 2.6.4.2 所示：

```
zuozhongkai@ubuntu:~$ grep -ir "Ubuntu" /usr
匹配到二进制文件 /usr/bin/fcitx-qimpanel-configtool
匹配到二进制文件 /usr/bin/nsupdate
/usr/bin/apport-bug:# Author: Martin Pitt <martin.pitt@ubuntu.com>
/usr/bin/apport-bug:# this so that confined applications using ubuntu-browsers.d
/ubuntu-integration
匹配到二进制文件 /usr/bin/locale
/usr/bin/fcitx-configtool: LXDE|Lubuntu)
匹配到二进制文件 /usr/bin/x86_64-linux-gnu-elfedit
```

图 2.6.4.2 命令 grep 演示

2.6.5 文件类型

这里的文件类型不是说这个文件是音乐文件还是文本文件，在用户根目录下使用命令“ls -l”来查看用户根目录下所有文件的详细信息，如图 2.6.5.1 所示：

```
zuozhongkai@ubuntu:~$ ls -l
总用量 72
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 12月 18 02:08 examples.desktop
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月 24 21:56 test1
-rw-rw-r-- 1 zuozhongkai zuozhongkai 194 12月 24 21:58 test1.tar.bz2
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月 23 01:00 test2
-rw-rw-r-- 1 zuozhongkai zuozhongkai 171 12月 24 22:09 test2.tar.gz
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月 23 00:45 test3
-rw-rw-r-- 1 zuozhongkai zuozhongkai 68 12月 21 01:40 test.txt
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月 19 21:41 tmp
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 22 02:08 桌面
```

图 2.6.5.1 文件详细信息

在图 2.6.5.1 中，每个文件的详细信息占一行，每行最前面的符号标记了当前文件类型，比如 test1 的第一个字符是“d”，test1.tar.bz2 文件第一个字符是“-”。这些字符表示的文件类型如下：

- 普通文件，一些应用程序创建的，比如文档、图片、音乐等等。
- d 目录文件。
- c 字符设备文件，Linux 驱动里面的字符设备驱动，比如串口设备，音频设备等。
- b 块设备文件，存储设备驱动，比如硬盘，U 盘等。
- l 符号连接文件，相当于 Windows 下的快捷方式。
- s 套接字文件。
- p 管道文件，主要指 FIFO 文件。

我们后面学习 Linux 驱动开发的时候基本是在和字符设备文件和块设备文件打交道。

2.7 Linux 用户权限管理

2.7.1 Ubuntu 用户系统

Ubuntu 是一个多用户系统,我们可以给不同的使用者创建不同的用户账号,每个用户使用各自的账号登陆,使用用户账号的目的方便系统管理员管理,控制不同用户对系统的访问权限,另一方面是为用户提供安全性保护。

我们前面在安装 Ubuntu 系统的时候被要求创建一个账户,当我们创建好账号以后,系统会在目录/home 下以该用户名创建一个文件夹,所有与该用户有关的文件都会被存储在这个文件夹中。同样的,创建其它用户账号的时候也会在目录/home 下生成一个文件夹来存储该用户的文件,图 2.7.1.1 就是我的电脑上“zuozhongkai”这个账户的文件夹。

```
zuozhongkai@ubuntu:~$ ls
examples.desktop  tmp  公共的  模板  视频  图片  文档  下载  音乐  桌面
```

图 2.7.1.1 用户账号根目录

装系统的时候创建的用户其权限比后面创建的用户大一点,但是没有 root 用户权限大,Ubuntu 下用户类型分为以下 3 类:

- 初次创建的用户,此用户可以完成比普通用户更多的功能。
- root 用户,系统管理员,系统中的玉皇大帝,拥有至高无上的权利。
- 普通用户,安装完操作系统以后被创建的用户。

以上三种用户,每个用户都有一个 ID 号,称为 UID,操作系统通过 UID 来识别是哪个用户,用户相关信息可以在文件/etc/passwd 中查看到,如图 2.7.1.2 所示:

```
zuozhongkai@ubuntu:/$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
hplip:x:115:7:HPLIP system user,,,:/var/run/hplip:/bin/false
kernoops:x:116:65534:Kernel Oops Tracking Daemon,,,:/bin/false
pulse:x:117:124:PulseAudio daemon,,,:/var/run/pulse:/bin/false
rtkit:x:118:126:RealtimeKit,,,:/proc:/bin/false
saned:x:119:127:./var/lib/saned:/bin/false
usbmux:x:120:46:usbmux daemon,,,:/var/lib/usbmux:/bin/false
zuozhongkai:x:1000:1000:zuozhongkai,,,:/home/zuozhongkai:/bin/bash
quest-sjhdig:x:999:999:访客:/tmp/guest-sjhdig:/bin/bash
```

图 2.7.1.2 passwd 文件内容

从配置文件 passwd 中可以看到,每个用户名后面都有两个数字,比如用户“zuozhongkai”后面“1000:1000”,第一个数字是用户的 ID,另一个是用户的 GID,也就是用户组 ID。Ubuntu 里面每个用户都属于一个用户组里面,用户组就是一组有相同属性的用户集合。

2.7.2 权限管理

在使用 Windows 的时候我们很少接触到用户权限,最多就是打开某个软件出问题的时候会选择以“管理员身份”打开。Ubuntu 下我们会常跟用户权限打交道,权限就是用户对于系统资

源的使用限制情况, root 用户拥有最大的权限, 可以为所欲为, 装系统的时候创建的用户拥有 root 用户的部分权限, 其它普通用户的权限最低。对于我们做嵌入式开发的人一般不关注用户的权限问题, 因为嵌入式基本是单用户, 做嵌入式开发重点关注的是文件的权限问题。

对于一个文件通常有三种权限: 读(r)、写(w)和执行(x), 使用命令 “ls -l” 可以查看某个目录下所有文件的权限信息, 如图 2.7.2.1 所示:

```
zuozhongkai@ubuntu:~$ ls -l
总用量 48
-rw-r--r-- 1 zuozhongkai zuozhongkai 8980 12月 18 02:08 examples.desktop
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 25 20:44 test.c
drwxrwxr-x 2 zuozhongkai zuozhongkai 4096 12月 19 21:41 tmp
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 公共的
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 模板
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 视频
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 图片
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 文档
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 下载
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 18 02:21 音乐
drwxr-xr-x 2 zuozhongkai zuozhongkai 4096 12月 22 02:08 桌面
```

图 2.7.2.1 文件权限信息

在图 2.7.2.1 中我们以文件 test.c 为例讲解, 文件 test.c 文件信息如下:

```
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 25 20:44 test.c
```

其中 “-rw-rw-r--” 表示文件权限与用户和用户组之间的关系, 第一位表示文件类型, 上一小节已经说了。剩下的 9 位以 3 位为一组, 分别表示文件拥有者的权限, 文件拥有者所在用户组的权限以及其它用户权限。后面的 “zuozhongkai zuozhongkai” 分别代表文件拥有者(用户)和该用户所在的用户组, 因此文件 test.c 的权限情况如下:

①、文件 test.c 的拥有者是用户 zuozhongkai, 其对文件 test.c 的权限是 “rw-”, 也就是对该文件拥有读和写两种权限。

②、用户 zuozhongkai 所在的用户组也叫做 zuozhongkai, 其组内用户对于文件 test.c 的权限是 “rw-”, 也是拥有读和写这两种权限。

③、其它用户对于文件 test.c 的权限是 “r-”, 也就是只读权限。

对于文件, 可读权限表示可以打开查看文件内容, 可写权限表示可以对文件进行修改, 可执行权限就是可以运行此文件(如果是软件的话)。对于文件夹, 拥有可读权限才可以使用命令 ls 查看文件夹中的内容, 拥有可执行权限才能进入到文件夹内部。

如果某个用户对某个文件不具有相应的权限的话就不能进行相应的操作, 比如根目录 “/” 下的文件只有 root 用户才有权限进行修改, 如果以普通用户去修改的话就会提示没有权限。比如我们要在根目录 “/” 创建一个文件 mytest, 使用命令 “touch mytest”, 结果如图 2.7.2.2 所示:

```
zuozhongkai@ubuntu:/$ touch mytest
touch: 无法创建 'mytest': 权限不够
```

图 2.7.2.2 创建文件

在图 2.7.2.2 中, 我以用户 “zuozhongkai” 在根目录 “/” 创建文件 mytest, 结果提示我无法创建 “mytest”, 因为权限不够, 因为只有 root 用户才能在根目录 “/” 下创建文件。我们可以使用命令 “sudo” 命令暂时切换到 root 用户, 这样就可以在根目录 “/” 下创建文件 mytest 了, 如图 2.7.2.3 所示:


```

zuozhongkai@ubuntu:/$ sudo touch mytest
[sudo] zuozhongkai 的密码:
zuozhongkai@ubuntu:/$ ls
bin      dev      initrd.img      lib64      mnt      proc      sbin      sys      var
boot     etc      initrd.img.old  lost+found mytest    root      snap      tmp      vmlinuz
cdrom    home     lib             media      opt      run      srv       usr      vmlinuz.old
    
```

图 2.7.2.3 使用 sudo 命令创建文件

在图 2.7.2.3 中我们使用命令“sudo”以后就可以在根目录“/”创建文件 mytest，在进行其它的操作的时候，遇到提示权限不够的时候都可以使用 sudo 命令暂时以 root 用户身份去执行。

上面我们讲了，文件的权限有三种：读(r)、写(w)和执行(x)，除了用 r、w 和 x 表示以外，我们也可以使用二进制数表示，三种权限就可以使用 3 位二进制数来表示，一种权限对应一个二进制位，如果该位为 1 就表示具备此权限，如果该位为 0 就表示不具备此权限，如表 2.7.2.1 所示：

字母	二进制	八进制
r	100	4
w	010	2
x	001	1

表 2.7.2.1 文件权限数字表示方法

如果做过单片机开发的话，就会发现和单片机里面的寄存器位一样，将三种权限 r、w 和 x 进行不同的组合，即可得到不同的二进制数和八进制数，3 位权限可以组出 8 种不同的权限组合，如表 2.7.2.2 所示：

权限	二进制数字	八进制数字
---	000	0
--x	001	1
-w-	010	2
-wx	011	3
r--	100	4
r-x	101	5
rw-	110	6
rwX	111	7

表 2.7.2.2 文件所有权限组合

表 2.7.2.2 中权限所对应的八进制数字就是每个权限对应的位相加，比如权限 rwX 就是 4+2+1=7。前面的文件 test.c 其权限为“rw-rw-r--”，因此其十进制表示就是：664。

另外我们也开始使用 a、u、g 和 o 表示文件的归属关系，用=、+和-表示文件权限的变化，如表 2.7.2.3 所示：

字母	意义
r	可读权限
w	可写权限
x	可执行权限
a	所有用户
u	归属用户
g	归属组
o	其它用户

=	具备权限
+	添加某权限
-	去除某权限

表 2.7.2.3 权限修改字母表示方式

对于文件 test.c，我们想要修改其归属用户(zuozhongkai)对其拥有可执行权限，那么就可以使用：u+x。如果希望设置归属用户及其所在的用户组都对其拥有可执行权限就可以使用：gu+x。

2.7.3 权限管理命令

我们也可以使用 Shell 来操作文件的权限管理，主要用到“chmod”和“chown”这两个命令，我们一个一个来看。

1、权限修改命令 chmod

命令“chmod”用于修改文件或者文件夹的权限，权限可以使用前面讲的数字表示也可以使用字母表示，命令格式如下：

chmod [参数] [文件名/目录名]

主要参数如下：

- c 效果类似“-v”参数，但仅回显更改的部分。
- f 不显示错误信息。
- R 递归处理，指定目录下的所有文件及其子文件目录一起处理。
- v 显示指令的执行过程。

我们先来学习以下如何使用命令“chmod”修改一个文件的权限，在用户根目录下创建一个文件 test，然后查看其默认权限，操作如图 2.7.3.1 所示：

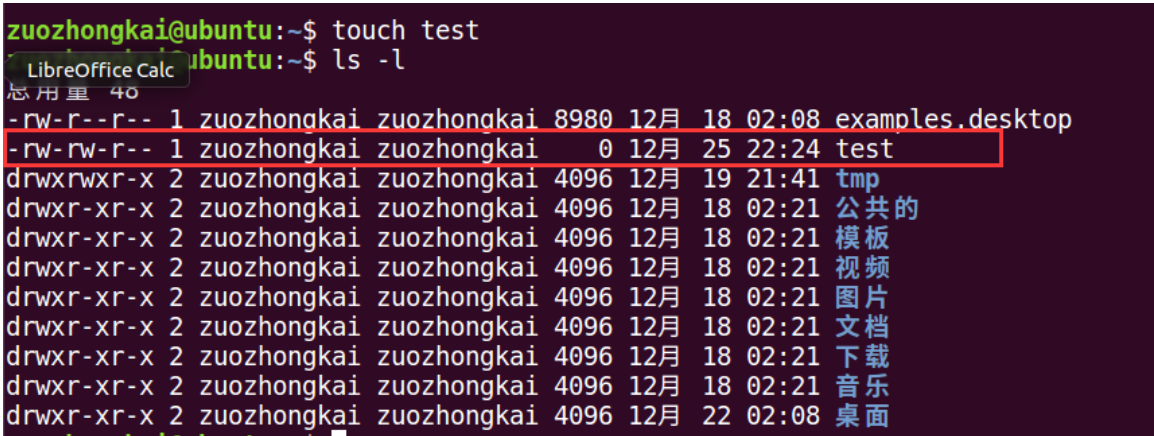


图 2.7.3.1 创建文件 test

在图 2.7.3.1 中我们创建了一个文件：test，这个文件的默认权限为“rw-rw-r--”，我们将其权限改为“rwxrw-rw”，对应数字就是 766，操作如下：

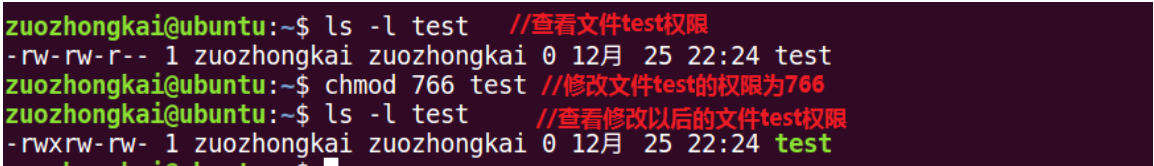


图 2.7.3.2 修改权限

在图 2.7.3.2 中，我们修改文件 test 的权限为 766，修改完成以后的 test 文件权限为“rwxrw-rw-”，和我们设置的一样，说明权限修改成功。

上面我们是通过数字来修改权限的,我们接下来使用字母来修改权限,操作如图 2.7.3.3 所示:

```
zuozhongkai@ubuntu:~$ touch a.c //创建文件a.c
zuozhongkai@ubuntu:~$ ls -l a.c //显示文件a.c的权限
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 25 22:33 a.c
zuozhongkai@ubuntu:~$ chmod u+x a.c //给文件a.c的归属用户添加可执行权限
zuozhongkai@ubuntu:~$ ls -l a.c //显示修改权限后的文件a.c
-rwxrw-r-- 1 zuozhongkai zuozhongkai 0 12月 25 22:33 a.c
```

图 2.7.3.3 使用字母修改文件权限

上面两个例子都是修改文件的权限,接下来我们修改文件夹的权限,新建一个 test 文件夹,在文件夹 test 里面创建 a.c、b.c 和 c.c 三个文件,如图 2.7.3.4 所示:

```
zuozhongkai@ubuntu:~$ ls -l test/
总用量 0
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 26 00:20 a.c
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 26 00:20 b.c
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 26 00:20 c.c
```

图 2.7.3.4 test 文件夹

在图 2.7.3.4 中 test 文件夹下的文件 a.c、b.c 和 c.c 的权限均为“rw-rw-r--”,我们将 test 文件夹下的所有文件权限都改为“rwxrwxrwx”,也就是数字 777,操作如图 2.7.3.5 所示:

```
zuozhongkai@ubuntu:~$ chmod -R 777 test/ //递归修改文件权限
zuozhongkai@ubuntu:~$ ls -l test/ //查看修改权限以后的文件
总用量 0
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月 26 00:20 a.c
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月 26 00:20 b.c
-rwxrwxrwx 1 zuozhongkai zuozhongkai 0 12月 26 00:20 c.c
```

图 2.7.3.5 递归修改文件夹权限

2、文件归属者修改命令 chown

命令 chown 用来修改某个文件或者目录的归属者用户或者用户组,命令格式如下:

```
chown [参数] [用户名.<组名>] [文件名/目录]
```

其中[用户名.<组名>]表示要将文件或者目录改为哪一个用户或者用户组,用户名和组名用“.”隔开,其中用户名和组名中的任何一个都可以省略,命令主要参数如下:

- c 效果同-v 类似,但仅显示更改的部分。
- f 不显示错误信息。
- h 只对符号连接的文件做修改,不改动其它任何相关的文件。
- R 递归处理,将指定的目录下的所有文件和子目录一起处理。
- v 显示处理过程。

在用户根目录下创建一个 test 文件,查看其文件夹所属用户和用户组,如图 2.7.3.6 所示:

```
zuozhongkai@ubuntu:~$ touch test
zuozhongkai@ubuntu:~$ ls -l test
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 26 00:36 test
```

图 2.7.3.6 test 文件信息查询

从图 2.7.3.6 中可以看出,文件 test 的归属用户为 zuozhongkai,所属的用户组为 zuozhongkai,将文件 test 归属用户改为 root 用户,所属的用户组也改为 root,操作如图 2.7.3.7 所示:

```

zuozhongkai@ubuntu:~$ ls -l test //显示文件test的归属用户和归属组
-rw-rw-r-- 1 zuozhongkai zuozhongkai 0 12月 26 00:45 test
zuozhongkai@ubuntu:~$ sudo chown root.root test //修改文件test的归属用户和归属组
zuozhongkai@ubuntu:~$ ls -l test //查看修改以后的文件test归属用户和归属组
-rw-rw-r-- 1 root root 0 12月 26 00:45 test

```

图 2.7.3.7 修改文件归属用户和归属组

命令 shown 同样也可以递归处理来修改文件夹的归属用户和用户组，用法和命令 chown 一样，这里就不演示了。

2.8 Linux 磁盘管理

2.8.1 Linux 磁盘管理基本概念

Linux 的磁盘管理体系和 Windows 有很大的区别，在 Windows 下经常会遇到“分区”这个概念，在 Linux 中一般不叫“分区”而叫“挂载点”。“挂载点”就是将一个硬盘的一部分做成文件夹的形式，这个文件夹的名字就是“挂载点”，不管在哪个发行版的 Linux 中，用户是绝对看到不到 C 盘、D 盘这样的概念的，只能看到以文件夹形式存在的“挂载点”。

文件/etc/fstab 详细的记录了 Ubuntu 中硬盘分区的情况，如图 2.8.1.1 所示：

```

zuozhongkai@ubuntu:~$ cat /etc/fstab
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point> <type> <options>      <dump> <pass>
# / was on /dev/sda1 during installation
UUID=fcd4e2a0-05f1-48fa-a6c8-b4efff06a180 /
# swap was on /dev/sda5 during installation
UUID=c9eb5d28-50fe-4e35-951d-ea8a1c4b7810 none swap sw 0 0

```

图 2.8.1.1 文件 fstab

在图 2.8.1.1 中有一行“/ was on /dev/sda1 during installation”，意思是根目录“/”是在/dev/sda1 上的，其中“/”是挂载点，“/dev/sda1”就是我们装 Ubuntu 系统的硬盘。由于我们的系统是安装在虚拟机中的，因此图 2.8.1.1 没有出现实际的硬盘。可以通过如下命令查看当前系统中的磁盘：

```
ls /dev/sd*
```

上述命令就是打印出所有以/dev/sd 开头的设备文件，如图 2.8.1.2 所示：

```

zuozhongkai@ubuntu:~$ ls /dev/sd*
/dev/sda /dev/sda1 /dev/sda2 /dev/sda5

```

图 2.8.1.2 查看硬盘设备文件

在图 2.8.1.2 中有四个磁盘设备文件，其中 sd 表示是 SATA 硬盘或者其它外部设备，最后面的数字表示该硬盘上的第 n 个分区，比如/dev/sda1 就表示磁盘 sda 上的第一个分区。图 2.8.1.2 中都是以/dev/sda 开头的，说明当前只有一个硬盘。如果再插上 U 盘、SD 卡啥的就可能会出现 /dev/sdb, /dev/sdc 等等。如果你的 U 盘有两个分区那么可能就会出现/dev/sdb1、dev/sdb2 这样的设备文件。比如我现在插入我的 U 盘，插入 U 盘会提示 U 盘是接到主机还是虚拟机，如图 2.8.1.3 所示：



图 2.8.1.3 U 盘连接选择

设置好图 2.8.1.3 以后, 点击“确定”按钮 U 盘就会自动连接到虚拟机中, 也就是连接到 Ubuntu 系统中, 我们再次使用命令“ls /dev/sd*”来查看当前的“/dev/sd*”设备文件, 如图 2.8.1.4 所示:

```
zuozhongkai@ubuntu:~$ ls /dev/sd*  
/dev/sda /dev/sda1 /dev/sda2 /dev/sda5 /dev/sdb /dev/sdb1
```

图 2.8.1.4 插入 U 盘后的设备文件

从图 2.8.1.4 可以看出, 相比图 2.8.1.2 多了/dev/sdb 和/dev/sdb1 这两个文件, 其中/dev/sdb 就是 U 盘文件, /dev/sdb1 表示 U 盘的第一个分区, 因为我的 U 盘就一个分区。

2.8.2 磁盘管理命令

本节我们学习一下跟磁盘操作有关的命令, 这些命令如下:

1、磁盘分区命令 fdisk

如果要对某个磁盘进行分区, 可以使用命令 fdisk, 命令格如下:

fdisk [参数]

主要参数如下:

-b<分区大小> 指定每个分区的大小。

-l 列出指定设备的分区表。

-s<分区编号> 将指定的分区大小输出到标准的输出上, 单位为块。

-u 搭配“-l”参数, 会用分区数目取代柱面数目, 来表示每个分区的起始地址。

比如我要对 U 盘进行分区, **千万不要对自己装 Ubuntu 系统进行分区!!!** 可以使用如下命令:

```
sudo fdisk /dev/sdb
```

结果如图 2.8.2.1 所示:

```
zuozhongkai@ubuntu:~$ sudo fdisk /dev/sdb
[sudo] zuozhongkai 的密码:

Welcome to fdisk (util-linux 2.27.1).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

命令(输入 m 获取帮助):
```

图 2.8.2.1 U 盘分区界面

在图 2.8.2.1 中提示我们输入“m”可以查看帮助, 因为 fdisk 还有一些子命令, 通过输入“m”可以查看都有哪些子命令, 如图 2.8.2.2 所示:

```
命令(输入 m 获取帮助): m
Help:

DOS (MBR)
a toggle a bootable flag
b edit nested BSD disklabel
c toggle the dos compatibility flag

Generic
d delete a partition
F list free unpartitioned space
l list known partition types
n add a new partition
p print the partition table
t change a partition type
v verify the partition table
i print information about a partition

Misc
m print this menu
u change display/entry units
x extra functionality (experts only)

Script
I load disk layout from sfdisk script file
O dump disk layout to sfdisk script file

Save & Exit
w write table to disk and exit
q quit without saving changes

Create a new label
g create a new empty GPT partition table
G create a new empty SGI (IRIX) partition table
o create a new empty DOS partition table
s create a new empty Sun partition table
```

图 2.8.2.2 fdisk 命令的子命令

图 2.8.2.2 中常用的命令如下:

- p** 显示现有的分区
- n** 建立新分区

- t** 更改分区类型
- d** 删除现有的分区
- a** 更改分区启动标志
- w** 对分区的更改写入到硬盘或者存储器中。
- q** 不保存退出。

由于我的 U 盘里面还有一些重要的文件，所以现在不能进行分区，这里现在就不演示 fdisk 的分区操作了，后面我们讲解裸机例程的时候需要将可执行的 bin 文件烧写到 SD 卡中，烧写到 SD 卡之前需要对 SD 卡进行分区，到时候在详细讲解如何使用 fdisk 命令对磁盘进行分区。

2、格式化命令 mkfs

使用命令 fdisk 创建好一个分区以后，我们需要对其格式化，也就是在这个分区上创建一个文件系统，Linux 下的格式化命令为 mkfs，命令格式如下：

```
mkfs [参数] [-t 文件系统类型] [分区名称]
```

主要参数如下：

- fs** 指定建立文件系统时的参数
- V** 显示版本信息和简要的使用方法。
- v** 显示版本信息和详细的使用方法。

比如我们要格式化 U 盘的分区/dev/sdb1 为 FAT 格式，那么就可以使用如下命令：

```
mkfs -t vfat /dev/sdb1
```

3、挂载分区命令 mount

我们创建好分区并且格式化以后肯定是要使用硬盘或者 U 盘的，那么如何访问磁盘呢？比如我的 U 盘就一个分区，为/dev/sdb1，如果直接打开文件/dev/sdb1 会发现根本就不是我们要的结果。我们需要将/dev/sdb1 这个分区挂载到一个文件夹中，然后通过这个文件访问 U 盘，磁盘挂载命令为 mount，命令格式如下：

```
mount [参数] [-t [类型] [设备名称] [目的文件夹]
```

命令主要参数有：

- V** 显示程序版本。
- h** 显示辅助信息。
- v** 显示执行过程详细信息。
- o ro** 只读模式挂载。
- o rw** 读写模式挂载。
- s-r** 等于-o ro。
- w** 等于-o rw。

挂载点是一个文件夹，因此在挂载之前先要创建一个文件夹，一般我们把挂载点放到“/mnt”目录下，在“/mnt”下创建一个 tmp 文件夹，然后将 U 盘的/dev/sdb1 分区挂载到/mnt/tmp 文件夹里面，操作如图 2.8.2.3 所示：

```

zuozhongkai@ubuntu:~$ ls /mnt //查看/mnt目录下的所有文件
zuozhongkai@ubuntu:~$
zuozhongkai@ubuntu:~$ sudo mkdir /mnt/tmp //创建文件夹/mnt/tmp
[sudo] zuozhongkai 的密码:
zuozhongkai@ubuntu:~$ ls /mnt //查看/mnt目录下的所有文件
tmp
zuozhongkai@ubuntu:~$ sudo mount -t vfat /dev/sdb1 /mnt/tmp //挂载U盘到/mnt/tmp中
zuozhongkai@ubuntu:~$ ls /mnt/tmp //查看/mnt/tmp中的文件,也就是U盘里面的文件
AD IMX6UL_ZERO_V1.0.zip 开发板光盘
ARM裸机与嵌入式Linux驱动开发V1.0.docx System Volume Information

```

图 2.8.2.3 挂载 U 盘

在图 2.8.2.3 中我们将 U 盘以 fat 格式挂载到目录/mnt/tmp 中,然后我们就可以通过访问 /mnt/tmp 来访问 U 盘了。

4、卸载命令 umount

当我们不再需要访问已经挂载的 U 盘,可以通过 umount 将其从卸载点卸载,命令格式如下:

```

umount [参数] -t [文件系统类型] [设备名称]
-a 卸载/etc/mntab 中的所有文件系统。
-h 显示帮助。
-n 卸载时不要将信息存入到/etc/mntab 文件中
-r 如果无法成功卸载,则尝试以只读的方式重新挂载。
-t<文件系统类型> 仅卸载选项中指定的文件系统。
-v 显示执行过程。

```

上面我们将 U 盘挂载到了文件夹/mnt/tmp 里面,这里我们使用命令 umount 将其卸载掉,操作如图 2.8.2.4 所示:

```

zuozhongkai@ubuntu:~$ ls /mnt/tmp //查看卸载之前的文件夹/mnt/tmp
AD IMX6UL_ZERO_V1.0.zip 开发板光盘
ARM裸机与嵌入式Linux驱动开发V1.0.docx System Volume Information
zuozhongkai@ubuntu:~$ sudo umount -t vfat /dev/sdb1 //卸载U盘的/dev/sdb1分区
zuozhongkai@ubuntu:~$ ls /mnt/tmp //查看卸载以后的/mnt/tmp文件夹
zuozhongkai@ubuntu:~$
zuozhongkai@ubuntu:~$

```

图 2.8.2.4 卸载 U 盘

在图 2.8.2.4 中,我们使用命令 umount 卸载了 U 盘,卸载以后当我们再去访问文件夹/mnt/tmp 的时候发现里面没有任何文件了,说明我们卸载成功了。

第三章 Linux C 编程入门

在 Windows 下我们可以使用各种各样的 IDE 进行编程，比如强大的 Visual Studio。但是在 Ubuntu 下如何进行编程呢？Ubuntu 下也有一些可以进行编程的工具，但是大多都只是编辑器，也就是只能进行代码编辑，如果要编译的话就需要用到 GCC 编译器，使用 GCC 编译器肯定就要接触到 Makefile。本章就讲解如何在 Ubuntu 下进行 C 语言的编辑和编译、GCC 和 Makefile 的使用和编写。通过本章的学习可以掌握 Linux 进行 C 编程的基本方法，为以后的 ARM 裸机和 Linux 驱动学习做准备。

3.1 Hello World !

我们所说的编写代码包括两部分：代码编写和编译，在 Windows 下可以使用 Visual Studio 来完成这两部分，可以在 Visual Studio 下编写代码然后直接点击编译就可以了。但是在 Linux 下这两部分是分开的，比如我们用 VIM 进行代码编写，编写完成以后再使用 GCC 编译器进行编译，其中代码编写工具很多，比如 VIM 编辑器、Emacs 编辑器、VScode 编辑器等等，本教程使用 Ubuntu 自带的 VIM 编辑器。先来编写一个最简单的“Hello World”程序，把 Linux 下的 C 编程完整的走一遍。

3.1.1 编写代码

先在用户根目录下创建一个工作文件夹：C_Program，所有的 C 语言练习都保存到这个工作文件夹下，创建过程如图 3.1.1.1 所示：

```
zuozhongkai@ubuntu:~$ mkdir C_Program
zuozhongkai@ubuntu:~$ ls
C_Program      test      模板      图片      下载      桌面
examples.desktop 公共的    视频      文档      音乐
```

图 3.1.1.1 创建工作目录

进入图 3.1.1.1 创建的 C_Program 工作文件夹，为了方便管理，我们后面每个例程都创建一个文件夹来保存所有与本例程有关的文件，创建一个名为“3.1”的文件夹来保存我们的“Hello World”程序相关的文件，操作如图 3.1.1.2 所示：

```
zuozhongkai@ubuntu:~$ cd C_Program/
zuozhongkai@ubuntu:~/C_Program$ mkdir 3.1
zuozhongkai@ubuntu:~/C_Program$ ls
3.1
zuozhongkai@ubuntu:~/C_Program$
```

图 3.1.1.2 创建工程文件夹

前面说了我们使用 VI 编辑器，在使用 VI 编辑器之前我们先做如下设置：

1、设置 TAB 键为 4 字节

VI 编辑器默认 TAB 键为 8 空格，我们改成 4 空格，用 vi 打开文件/etc/vim/vimrc，在此文件最后面输入如下代码：

```
set ts=4
```

添加完成如图 3.1.1.3 所示：

```
" Source a global configuration file if available
if filereadable("/etc/vim/vimrc.local")
    source /etc/vim/vimrc.local
endif

set ts=4
```

图 3.1.1.3 设置 TAB 为四个空格

修改完成以后保存并关闭文件。

2、VIM 编辑器显示行号

VIM 编辑器默认是不显示行号的，不显示行号不利于代码查看，我们设置 VIM 编辑器显示行号，同样是通过在文件/etc/vim/vimrc 中添加代码来实现，在文件最后面加入下面一行代码

即可:

```
set nu
```

添加完成以后的/etc/vim/vimrc 文件如图 3.1.1.4 所示:

```
" Source a global configuration file if available
if filereadable("/etc/vim/vimrc.local")
    source /etc/vim/vimrc.local
endif

set ts=4
set nu
```

图 3.1.1.4 设置 VIM 编辑器显示行号

VIM 编辑器可以自行定制,网上有很多的博客讲解如何设置 VIM,感兴趣的可以上网看一下。设置好 VIM 编辑器以后就可以正式开始编写代码了,进入前面创建的“3.1”这个工程文件夹里面,使用 vi 指令创建一个名为“main.c”的文件,然后在里面输入如下代码:

示例代码 3.1.1.1 main.c 文件代码

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[])
4 {
5     printf("Hello World!\n");
6 }
```

编写完成以后保存退出 vi 编辑器,可以使用“cat”命令查看代码是否编写成功,如图 3.1.1.5 所示:

```
zuozhongkai@ubuntu:~/C_Program/3.1$ cat main.c
#include <stdio.h>

int main(int argc, char *argv[])
{
    printf("Hello World!\n");
}
```

图 3.1.1.5 查阅程序源码

从图 3.1.1.5 可以看出 main.c 文件编辑完成,代码编辑完成以后我们需要对其进行编译。

3.1.2 编译代码

Ubuntu 下的 C 语言编译器是 GCC, GCC 编译器在我们 Ubuntu 的时候就已经默认安装好了,可以通过如下命令查看 GCC 编译器的版本号:

```
gcc -v
```

在终端中输入上述命令以后终端输出如图 3.1.2.1 所示:

```

zuozhongkai@ubuntu:~/C_Program/3.1$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/5/lto-wrapper
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 5.4.0-6ubuntu1-16.04.11' --with-bugurl=file:///usr/share/doc/gcc-5/README.Bugs --enable-languages=c,ada,c++,java,go,d,fortran,objc,obj-c++ --prefix=/usr --program-suffix=-5 --enable-shared --enable-linker-build-id --libexecdir=/usr/lib --without-included-gettext --enable-threads=posix --libdir=/usr/lib --enable-nls --with-sysroot=/ --enable-clocale=gnu --enable-libstdcxx-debug --enable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-gnu-unique-object --disable-vtable-verify --enable-libmpx --enable-plugin --with-system-zlib --disable-browser-plugin --enable-java-awt=gtk --enable-gtk-cairo --with-java-home=/usr/lib/jvm/java-1.5.0-gcj-5-amd64/jre --enable-java-home --with-jvm-root-dir=/usr/lib/jvm/java-1.5.0-gcj-5-amd64 --with-jvm-jar-dir=/usr/lib/jvm-exports/java-1.5.0-gcj-5-amd64 --with-arch-directory=amd64 --with-ecj-jar=/usr/share/java/eclipse-ecj.jar --enable-bj1-gc --enable-multiarch --disable-werror --with-arch=32=i686 --with-abi=m64 --with-multilib-list=m32,m64,mx32 --enable-multilib --with-tune=generic --enable-checking=release --build=x86_64-linux-gnu --host=x86_64-linux-gnu --target=x86_64-linux-gnu
Thread model: posix
gcc version 5.4.0 20160609 (Ubuntu 5.4.0-6ubuntu1-16.04.11)

```

图 3.1.2.1 gcc 版本查询

如果输入命令“gcc -v”命令以后，你的终端输出类似图 3.1.2.1 中的信息，那么说明你的电脑已经有 GCC 编译器了。最后下面的“gcc version 5.4.0”说明本机的 GCC 编译器版本为 5.4.0 的。注意观察在图 3.1.2.1 中有“Target: x86_64-linux-gnu”一行，这说明 Ubuntu 自带的 GCC 编译器是针对 X86 架构的，因此只能编译在 X86 架构 CPU 上运行的程序。如果想要编译在 ARM 上运行的程序就需要针对 ARM 的 GCC 编译器，也就是交叉编译器！我们是 ARM 开发，因此肯定要安装针对 ARM 架构的 GCC 交叉编译器，当然了，这是后面的事，现在我们不用管这些，只要知道不同的目标架构，其 GCC 编译器是不同的。

如何使用 GCC 编译器来编译 main.c 文件呢？GCC 编译器是命令模式的，因此需要输入命令来使用 gcc 编译器来编译文件，输入如下命令：

```
gcc main.c
```

上述命令的功能就是使用 gcc 编译器来编译 main.c 这个 c 文件，过程如图 3.1.2.2 所示：

```

zuozhongkai@ubuntu:~/C_Program/3.1$ gcc main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out main.c

```

图 3.1.2.2 编译 main.c 文件

在图 3.1.2.2 中可以看到，当编译完成以后会生成一个 a.out 文件，这个 a.out 就是编译生成的可执行文件，执行此文件看看是否和我们代码的功能一样，执行的方法很简单使用命令：“./+可执行文件”，比如本例程就是命令：./a.out，操作如图 3.1.2.3 所示：

```

zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ./a.out
Hello World!

```

图 3.1.2.3 执行编译得到的文件

在图 3.1.2.3 中执行 a.out 文件以后终端输出了“Hello World!”，这正是 main.c 要实现的功能，说明我们的程序没有错误。a.out 这个文件的命名是 GCC 编译器自动命名的，那我们能不能决定编译完生成的可执行文件名字呢？肯定可以的，在使用 gcc 命令的时候加上 -o 来指定生成的可执行文件名字，比如编译 main.c 以后生成名为“main”的可执行文件，操作如图 3.1.2.4 所示：

```

zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ gcc main.c -o main //指定编译生成的可执行文件名字为main
zuozhongkai@ubuntu:~/C_Program/3.1$ ls
a.out main main.c
zuozhongkai@ubuntu:~/C_Program/3.1$ ./main
Hello World!

```

图 3.1.2.4 指定可执行文件名字

在图 3.1.2.4 中，我们使用“gcc main.c -o main”来编译 main.c 文件，使用参数“-o”来指定

编译生成的可执行文件名字,至此我们就完成 Linux 下 C 编程和编译的一整套过程。

3.2 GCC 编译器

3.2.1 gcc 命令

在上一小节我们已经使用过 GCC 编译器来编译 C 文件了,我们使用到是 gcc 命令, gcc 命令格式如下:

```
gcc [选项] [文件名字]
```

主要选项如下:

-c: 只编译不链接为可执行文件,编译器将输入的.c 文件编译为.o 的目标文件。

-o: <输出文件名>用来指定编译结束以后的输出文件名,如果不使用这个选项的话 GCC 默认编译出来的可执行文件名字为 a.out。

-g: 添加调试信息,如果要使用调试工具(如 GDB)的话就必须加入此选项,此选项指示编译的时候生成调试所需的符号信息。

-O: 对程序进行优化编译,如果使用此选项的话整个源代码在编译、链接的时候都会进行优化,这样产生的可执行文件执行效率就高。

-O2: 比-O 更幅度更大的优化,生成的可执行效率更高,但是整个编译过程会很慢。

3.2.2 编译错误警告

在 Windows 下不管我们用啥编译器,如果程序有语法错误,那么编译的时候都会指示出来,比如开发 STM32 的时候所使用的 MDK 和 IAR,我们可以根据错误信息方便的修改 bug。那 GCC 编译器有没有错误提示呢?肯定是有,我们可以测试一下,新名为“3.2”的文件夹,使用 vi 在文件夹“3.2”中创建一个 main.c 文件,在文件里面输入如下代码:

示例代码 3.2.2.1 main.c 文件代码

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[])
4 {
5     int a, b;
6
7     a = 3;
8     b = 4
9     printf("a+b=\n", a + b);
10 }
```

在上述代码中有两处错误:

第 8 行、第一处是“b=4”少写了个一个“;”号。

第 9 行、第二处应该是 printf(“a+b=%d\n”, a + b);

我们编译一下上述代码,看看 GCC 编译器是否能够检查出错误,编译结果如图 3.2.2.1 所示:

```
zuo zhong kai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
main.c: In function 'main':
main.c:9:5: error: expected ';' before 'printf'
    printf("a+b=", a + b);
    ^
```

图 3.2.2.1 错误提示

从图 3.2.2.1 中可以看出有一个 error, 提示在 main.c 文件的第 9 行有错误, 错误类型是在 printf 之前没有 “;” 号, 这就是第一处错误, 我们在 “b = 4” 后面加上分号, 然后接着编译, 结果又提示有一个错误, 如图 3.2.2.2 所示:

```
zuozhongkai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
main.c: In function 'main':
main.c:9:12: warning: too many arguments for format [-Wformat-extra-args]
printf("a+b=\n", a + b);
    ^
```

图 3.2.2.2 错误提示

在图 3.2.2.2 中, 提示我们说文件 main.c 的第 9 行: printf(“a+b=\n”, a + b)有 error, 错误是因为太多参数了, 我们将其改为:

```
printf(“a+b=%d\n”, a + b);
```

修改完成以后接着重新编译一下, 结果如图 3.2.2.3 所示:

```
zuozhongkai@ubuntu:~/C_Program/3.2$ gcc main.c -o main
zuozhongkai@ubuntu:~/C_Program/3.2$ ls
main  main.c
```

图 3.2.2.3 编译成功

在图 3.2.2.3 中我们编译成功, 生成了可执行文件 main, 执行一下 main, 看看结果和我们设计的是否一样, 如图 3.2.2.4 所示:

```
zuozhongkai@ubuntu:~/C_Program/3.2$ ./main
a+b=7
```

图 3.2.2.4 执行结果

可以看出, GCC 编译器和其它编译器一样, 不仅能够检测出错误类型, 而且标记除了错误发生在哪个文件、哪一行, 方便我们去修改代码。

3.2.3 编译流程

GCC 编译器的编译流程是: 预处理、编译、汇编和链接。预处理就是展开所有的头文件、替换程序中的宏、解析条件编译并添加到文件中。编译是将经过预编译处理的代码编译成汇编代码, 也就是我们常说的程序编译。汇编就是将汇编语言文件编译成二进制目标文件。链接就是将汇编出来的多个二进制目标文件链接在一起, 形成最终的可执行文件, 链接的时候还会涉及到静态库和动态库等问题。

上一小节演示的例程都只有一个文件, 而且文件非常简单, 因此可以直接使用 gcc 命令生成可执行文件, 并没有先将 c 文件编译成.o 文件, 然后再链接在一起。

3.3 Makefile 基础

3.3.1 何为 Makefile

上一小节我们讲了如何使用 GCC 编译器在 Linux 进行 C 语言编译, 通过在终端执行 gcc 命令来完成 C 文件的编译, 如果我们的工程只有一两个 C 文件还好, 需要输入的命令不多, 当文件有几十、上百甚至上万个的时候用终端输入 GCC 命令的方法显然是不现实的。如果我们能够编写一个文件, 这个文件描述了编译哪些源码文件、如何编译那就好了, 每次需要编译工程的时候只需要使用这个文件就行了。这种问题怎么可能难倒聪明的程序员, 为此提出了一个解决大工程编译的工具: make, 描述哪些文件需要编译、哪些需要重新编译的文件就叫做 Makefile, Makefile 就跟脚本文件一样, Makefile 里面还可以执行系统命令。使用的时候只需要一个 make

命令即可完成整个工程的自动编译,极大的提高了软件开发的效率。如果大家以前一直使用 IDE 来编写 C 语言的话肯定没有听说过 Makefile 这个东西,其实这些 IDE 是有的,只不过这些 IDE 对其进行了封装,提供给大家的是已经经过封装后的图形界面了,我们在 IDE 中添加要编译的 C 文件,然后点击按钮就完成了编译。在 Linux 下用的最多的是 GCC 编译器,这是个没有 UI 的编译器,因此 Makefile 就需要我们自己来编写了。作为一个专业的程序员,是一定要懂得 Makefile 的,一是因为在 Linux 下你不得不懂 Makefile,再就是通过 Makefile 你就能了解整个工程的处理过程。

由于 Makefile 的知识比较多,完全可以单独写本书,因此本章我们只讲解 Makefile 基础入门,如果想详细的研究 Makefile,推荐大家阅读《跟我一起写 Makefile》这份文档,文档已经放到了[开发板光盘->4、参考资料](#)里面了,本章也有很多地方参考了此文档。

3.3.2 Makefile 的引入

我们完成这样一个小工程,通过键盘输入两个整形数字,然后计算他们的和并将结果显示在屏幕上,在这个工程中我们有 main.c、input.c 和 calcu.c 这三个 C 文件和 input.h、calcu.h 这两个头文件。其中 main.c 是主体,input.c 负责接收从键盘输入的数值,calcu.c 进行任意两个数相加,其中 main.c 文件内容如下:

示例代码 3.3.2.1 main.c 文件代码

```
1 #include <stdio.h>
2 #include "input.h"
3 #include "calcu.h"
4
5 int main(int argc, char *argv[])
6 {
7     int a, b, num;
8
9     input_int(&a, &b);
10    num = calcu(a, b);
11    printf("%d + %d = %d\r\n", a, b, num);
12 }
```

input.c 文件内容如下:

示例代码 3.3.2.2 input.c 文件代码

```
1 #include <stdio.h>
2 #include "input.h"
3
4 void input_int(int *a, int *b)
5 {
6     printf("input two num:");
7     scanf("%d %d", a, b);
8     printf("\r\n");
9 }
```

calcu.c 文件内容如下:

示例代码 3.3.2.3 calcu.c 文件代码

```
1 #include "calcu.h"
```



```
2
3 int calcu(int a, int b)
4 {
5     return (a + b);
6 }
```

文件 input.h 内容如下:

示例代码 3.3.2.4 input.h 文件代码

```
1 #ifndef _INPUT_H
2 #define _INPUT_H
3
4 void input_int(int *a, int *b);
5 #endif
```

文件 calcu.h 内容如下:

示例代码 3.3.2.5 calcu.h 文件代码

```
1 #ifndef _CALCU_H
2 #define _CALCU_H
3
4 int calcu(int a, int b);
5 #endif
```

以上就是我们这个小工程的所有源文件,我们接下来使用 3.1 节讲的方法来对其进行编译,在终端输入如下命令:

```
gcc main.c calcu.c input.c -o main
```

上面命令的意思就是使用 gcc 编译器对 main.c、calcu.c 和 input.c 这三个文件进行编译,编译生成的可执行文件叫做 main。编译完成以后执行 main 这个程序,测试一下软件是否工作正常,结果如图 3.3.2.1 所示:

```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcu.c calcu.h input.c input.h main main.c
zuozhongkai@ubuntu:~/C_Program/3.3$ ./main
input two num:5 6

5 + 6 = 11
```

图 3.3.2.1 程序测试

可以看出我们的代码按照我们所设想的工作了,使用命令“gcc main.c calcu.c input.c -o main”看起来很简单是吧,只需要一行就可以完成编译,但是我们这个工程只有三个文件啊!如果几千个文件呢?再就是如果有一个文件被修改了以,使用上面的命令编译的时候所有的文件都会重新编译,如果工程有几万个文件(Linux 源码就有这么多文件!),想想这几万个文件编译一次所需要的时间就可怕。最好的办法肯定是哪个文件被修改了,只编译这个被修改的文件即可,其它没有修改的文件就不需要再次重新编译了,为此我们改变我们的编译方法,如果第一次编译工程,我们先将工程中的文件都编译一遍,然后后面修改了哪个文件就编译哪个文件,命令如下:

```
gcc -c main.c
gcc -c input.c
gcc -c calcu.c
gcc main.o input.o calcu.o -o main
```

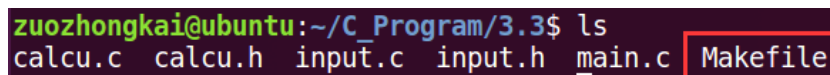
上述命令前三行分别是将 main.c、input.c 和 calcul.c 编译成对应的.o 文件，所以使用了“-c”选项，“-c”选项我们上面说了，是只编译不链接。最后一行命令是将编译出来的所有.o 文件链接成可执行文件 main。假如我们现在修改了 calcul.c 这个文件，只需要将 calcul.c 这一个文件重新编译成.o 文件，然后在将所有的.o 文件链接成可执行文件即，只需要下面两条命令即可：

```
gcc -c calcul.c
gcc main.o input.o calcul.o -o main
```

但是这样就又有一个问题，如果修改的文件一多，我自己可能都不记得哪个文件修改过了，然后忘记编译，然后……，为此我们需要这样一个工具：

- 1、如果工程没有编译过，那么工程中的所有.c 文件都要被编译并且链接成可执行程序。
- 2、如果工程中只有个别 C 文件被修改了，那么只编译这些被修改的 C 文件即可。
- 3、如果工程的头文件被修改了，那么我们需要编译所有引用这个头文件的 C 文件，并且链接成可执行文件。

很明显，能够完成这个功能的就是 Makefile 了，在工程目录下创建名为“Makefile”的文件，文件名一定要叫做“Makefile”!!! 区分大小写的哦！如图 3.3.2.2 所示：



```
zuo zhong kai@ubuntu:~/C_Program/3.3$ ls
calcul.c  calcul.h  input.c  input.h  main.c  Makefile
```

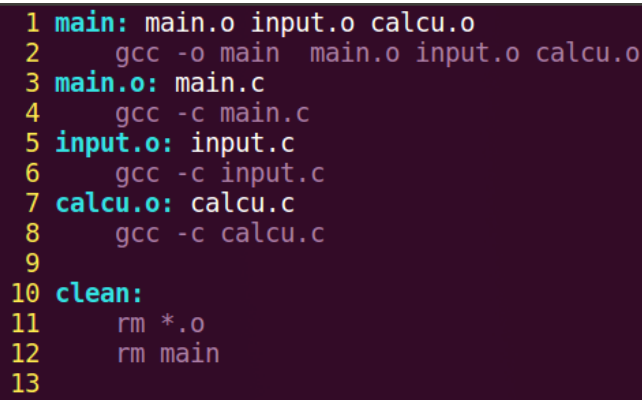
图 3.3.2.2 Makefile 文件

在图 3.3.2.2 中 Makefile 和 C 文件是处于同一个目录的，在 Makefile 文件中输入如下代码：

示例代码 3.3.2.6 Makefile 文件代码

```
1 main: main.o input.o calcul.o
2     gcc -o main main.o input.o calcul.o
3 main.o: main.c
4     gcc -c main.c
5 input.o: input.c
6     gcc -c input.c
7 calcul.o: calcul.c
8     gcc -c calcul.c
9
10 clean:
11     rm *.o
12     rm main
```

上述代码中所有行首需要空出来的地方一定要使用“TAB”键！不要使用空格键！这是 Makefile 的语法要求，编写好得 Makefile 如图 3.3.2.3 所示：



```
1 main: main.o input.o calcul.o
2     gcc -o main main.o input.o calcul.o
3 main.o: main.c
4     gcc -c main.c
5 input.o: input.c
6     gcc -c input.c
7 calcul.o: calcul.c
8     gcc -c calcul.c
9
10 clean:
11     rm *.o
12     rm main
13
```

图 3.3.2.3 Makefile 源码

Makefile 编写好以后我们就可以使用 `make` 命令来编译我们的工程了，直接在命令行中输入“`make`”即可，`make` 命令会在当前目录下查找是否存在“`Makefile`”这个文件，如果存在的话就会按照 `Makefile` 里面定义的编译方式进行编译，如图 3.3.2.4 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls //查看目录下所有文件
calcu.c  calcul.h  input.c  input.h  main.c  Makefile
zuozhongkai@ubuntu:~/C_Program/3.3$ make //make命令编译工程
gcc -c main.c
gcc -c input.c
gcc -c calcul.c
gcc -o main main.o input.o calcul.o
zuozhongkai@ubuntu:~/C_Program/3.3$ ls //查看编译完成后的文件夹，看看编译是否成功
calcu.c  calcul.o  input.h  main  main.o
calcu.h  input.c  input.o  main.c  Makefile
```

图 3.3.2.4 Make 编译工程

在图 3.3.2.4 中，使用命令“`make`”编译完成以后就会在当前工程目录下生成各种.o 和可执行文件，说明我们编译成功了。使用 `make` 命令编译工程的时候可能会提示如图 3.3.2.5 所示错误：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ make
Makefile:2: *** missing separator. 停止。
```

图 3.3.2.5 Make 失败

图 3.3.2.5 中的错误来源一般有两点：

1、`Makefile` 中命令缩进没有使用 `TAB` 键！

2、VI/VIM 编辑器使用空格代替了 `TAB` 键，修改文件/etc/vim/vimrc，在文件最后面加上如下所示代码：

```
set noexpandtab
```

我们修改一下 `input.c` 文件源码，随便加几行空行就行了，保证 `input.c` 被修改过即可，修改完成以后再执行一下“`make`”命令重新编译一下工程，结果如图 3.3.2.6 所示：

```
zuozhongkai@ubuntu:~/C_Program/3.3$ make
gcc -c input.c
gcc -o main main.o input.o calcul.o
```

图 3.3.2.6 重新编译工程

从图 3.3.2.6 中可以看出因为我们修改了 `input.c` 这个文件，所以 `input.c` 和最后的可执行文件 `main` 重新编译了，其它没有修改过的文件就没有编译。而且我们只需要输入“`make`”这个命令即可，非常方便，但是 `Makefile` 里面的代码都是什么意思呢？这就是接下来我们要讲解的。

3.4 Makefile 语法

3.4.1 Makefile 规则格式

`Makefile` 里面是由一系列的规则组成的，这些规则格式如下：

目标……： 依赖文件集合……

命令 1

命令 2

……

比如下面这条规则:

```
main: main.o input.o calcul.o
    gcc -o main main.o input.o calcul.o
```

这条规则的目标是 main, main.o、input.o 和 calcul.o 是生成 main 的依赖文件, 如果要更新目标 main, 就必须先更新它的所有依赖文件, 如果依赖文件中的任何一个有更新, 那么目标也必须更新, “更新”就是执行一遍规则中的命令列表。

命令列表中的每条命令必须以 TAB 键开始, 不能使用空格!

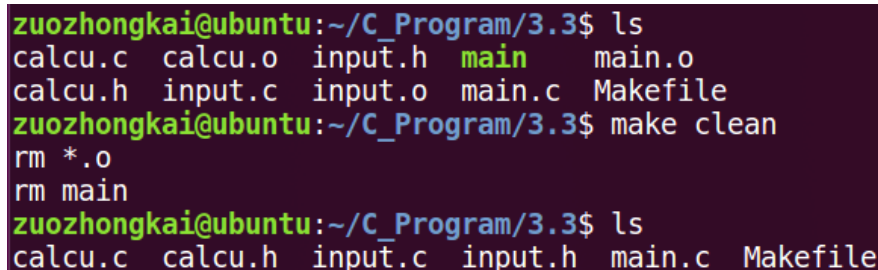
make 命令会为 Makefile 中的每个以 TAB 开始的命令创建一个 Shell 进程去执行。

了解了 Makefile 的基本运行规则以后我们再来分析一下 3.3 节中“示例代码 3.3.2.6”中的 Makefile, 代码如下:

```
1 main: main.o input.o calcul.o
2     gcc -o main main.o input.o calcul.o
3 main.o: main.c
4     gcc -c main.c
5 input.o: input.c
6     gcc -c input.c
7 calcul.o: calcul.c
8     gcc -c calcul.c
9
10 clean:
11     rm *.o
12     rm main
```

上述代码中一共有 5 条规则, 1~2 行为第一条规则, 3~4 行为第二条规则, 5~6 行为第三条规则, 7~8 行为第四条规则, 10~12 为第五条规则, make 命令在执行这个 Makefile 的时候其执行步骤如下:

首先更新第一条规则中的 main, 第一条规则的目标成为默认目标, 只要默认目标更新了那么就认为 Makefile 的工作。在第一次编译的时候由于 main 还不存在, 因此第一条规则会执行, 第一条规则依赖于文件 main.o、input.o 和 calcul.o 这三个.o 文件, 这三个.o 文件目前还没有, 因此必须先更新这三个文件。make 会查找以这三个.o 文件为目标的规则并执行。以 main.o 为例, 发现更新 main.o 的是第二条规则, 因此会执行第二条规则, 第二条规则里面的命令为“gcc -c main.c”, 这行命令很熟悉了吧, 就是不链接编译 main.c, 生成 main.o, 其它两个.o 文件同理。最后一个规则目标是 clean, 它没有依赖文件, 因此会默认为依赖文件都是最新的, 所以其对应的命令不会执行, 当我们想要执行 clean 的话可以直接使用命令“make clean”, 执行以后就会删除当前目录下所有的.o 文件以及 main, 因此 clean 的功能就是完成工程的清理, “make clean”的执行过程如图 3.4.1.1 所示:



```
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcul.c  calcul.o  input.h  main      main.o
calcul.h  input.c  input.o  main.c    Makefile
zuozhongkai@ubuntu:~/C_Program/3.3$ make clean
rm *.o
rm main
zuozhongkai@ubuntu:~/C_Program/3.3$ ls
calcul.c  calcul.h  input.c  input.h  main.c  Makefile
```

图 3.4.1.1 make clean 执行过程

从图 3.4.1.1 可以看出, 当执行“make clean”命令以后, 前面编译出来的.o 和 main 可执行文件都被删除掉了, 也就是完成了工程清理工作。

我们在来总结一下 Make 的执行过程:

- 1、make 命令会在当前目录下查找以 Makefile(makefile 其实也可以)命名的文件。
- 2、当找到 Makefile 文件以后就会按照 Makefile 中定义的规则去编译生成最终的目标文件。
- 3、当发现目标文件不存在, 或者目标所依赖的文件比目标文件新(也就是最后修改时间比目标文件晚)的话就会执行后面的命令来更新目标。

这就是 make 的执行过程, make 工具就是在 Makefile 中一层一层的查找依赖关系, 并执行相应的命令。编译出最终的可执行文件。Makefile 的好处就是“自动化编译”, 一旦写好了 Makefile 文件, 以后只需要一个 make 命令即可完成整个工程的编译, 极大的提高了开发效率。把 make 和 Makefile 和做菜类似, 目标都是呈现出一场盛宴, 它们之间的对比关系如表 3.4.1.1 所示:

make	做菜	描述
make 工具	厨师	负责将材料加工成“美味”。
gcc 编译器	厨具	加工“美味”的工具。
Makefile	食材	加工美味所需的原材料。
最终的可执行文件	最终的菜肴	最终目的, 呈现盛宴

表 3.4.1.1 make 和做菜对比

总结一下, Makefile 中规则用来描述在什么情况下使用什么命令来构建一个特定的文件, 这个文件就是规则的“目标”, 为了生成这个“目标”而作为材料的其它文件称为“目标”的依赖, 规则的命令是用来创建或者更新目标的。

除了 Makefile 的“终极目标”所在的规则以外, 其它规则的顺序在 Makefile 中是没有意义的, “终极目标”就是指在使用 make 命令的时候没有指定具体的目标时, make 默认的那个目标, 它是 Makefile 文件中第一个规则的目标, 如果 Makefile 中的第一个规则有多个目标, 那么这些目标中的第一个目标就是 make 的“终极目标”。

3.4.2 Makefile 变量

跟 C 语言一样 Makefile 也支持变量的, 先看一下前面的例子:

```
main: main.o input.o calcu.o
gcc -o main main.o input.o calcu.o
```

上述 Makefile 语句中, main.o input.o 和 calcue.o 这三个依赖文件, 我们输入了两遍, 我们这个 Makefile 比较小, 如果 Makefile 复杂的时候这种重复输入的工作就会非常费时间, 而且非常容易输错, 为了解决这个问题, Makefile 加入了变量支持。不像 C 语言中的变量有 int、char 等各种类型, Makefile 中的变量都是字符串! 类似 C 语言中的宏。使用变量将上面的代码修改, 修改以后如下所示:

示例代码 3.4.2.1 Makefile 变量使用

```
1 #Makefile 变量的使用
2 objects = main.o input.o calcu.o
3 main: $(objects)
4 gcc -o main $(objects)
```

我们来分析一下“示例代码 3.4.2.1”, 第 1 行是注释, Makefile 中可以写注释, 注释开头要用符号“#”, 不能用 C 语言中的“//”或者“/!”! 第 2 行我们定义了一个变量 objects, 并且给这个变量进行了赋值, 其值为字符串“main.o input.o calcu.o”, 第 3 和 4 行使用到了变量 objects, Makefile 中变量的引用方法是“\$(变量名)”, 比如本例中的“\$(objects)”就是使用变量 objects。

在“示例代码 3.4.2.1”中我们在定义变量 `objects` 的时候使用“=”对其进行了赋值，Makefile 变量的赋值符还有其它两个“:=”和“?=", 我们来看一下这三种赋值符的区别：

1、赋值符“=”

使用“=”在给变量的赋值的时候，不一定要用已经定义好的值，也可以使用后面定义的值，比如如下代码：

示例代码 3.4.2.1 赋值符“=”使用

```
1 name = zzk
2 curname = $(name)
3 name = zuozhongkai
4
5 print:
6     @echo curname: $(curname)
```

我们来分析一下上述代码，第 1 行定义了一个变量 `name`，变量值为“zzk”，第 2 行也定义了一个变量 `curname`，`curname` 的变量值引用了变量 `name`，按照我们 C 写语言的经验此时 `curname` 的值就是“zzk”。第 3 行将变量 `name` 的值改为了“zuozhongkai”，第 5、6 行是输出变量 `curname` 的值。在 Makefile 要输出一串字符的话使用“echo”，就和 C 语言中的“printf”一样，第 6 行中的“echo”前面加了个“@”符号，因为 Make 在执行的过程中会自动输出命令执行过程，在命令前面加上“@”的话就不会输出命令执行过程，大家可以测试一下不加“@”的效果。使用命令“make print”来执行上述代码，结果如图 3.4.2.1：

```
zuozhongkai@ubuntu:~/C_Program$ make print
curname: zuozhongkai
zuozhongkai@ubuntu:~/C_Program$
```

图 3.4.2.1 make 执行结果

在图 3.4.2.1 中可以看到 `curname` 的值不是“zzk”，竟然是“zuozhongkai”，也就是变量“name”最后一次赋值的结果，这就是赋值符“=”的神奇之处！借助另外一个变量，可以将变量的真实值推到后面去定义。也就是变量的真实值取决于它所引用的变量的最后一次有效值。

2、赋值符“:=”

在“示例代码 3.4.2.1”上来测试赋值符“:=”，修改“示例代码 3.4.2.1”中的第 2 行，将其中的“=”改为“:=”，修改完成以后的代码如下：

示例代码 3.4.2.2 “:=”的使用

```
1 name = zzk
2 curname := $(name)
3 name = zuozhongkai
4
5 print:
6     @echo curname: $(curname)
```

修改完成以后重新执行一下 Makefile，结果如图 3.4.2.2 所示：

```
zuozhongkai@ubuntu:~/C_Program$ make print
curname: zzk
zuozhongkai@ubuntu:~/C_Program$
```

图 3.4.2.2 make 执行结果

从图 3.4.2.2 中可以看到此时的 `curname` 是 `zzk`，不是 `zuozhongkai` 了。这是因为赋值符“:=”

不会使用后面定义的变量，只能使用前面已经定义好的，这就是“=”和“:=”两个的区别。

3、赋值符“?=”

“?=”是一个很有用的赋值符，比如下面这行代码：

```
curname ?= zuozhongkai
```

上述代码的意思就是，如果变量 `curname` 前面没有被赋值，那么此变量就是“zuozhongkai”，如果前面已经赋过值了，那么就使用前面赋的值。

4、变量追加“+=”

Makefile 中的变量是字符串，有时候我们需要给前面已经定义好的变量添加一些字符串进去，此时就要使用到符号“+=”，比如如下所示代码：

```
objects = main.o input.o
objects += calcu.o
```

一开始变量 `objects` 的值为“main.o input.o”，后面我们给他追加了一个“calcu.o”，因此变量 `objects` 变成了“main.o input.o calcu.o”，这个就是变量的追加。

3.4.3 Makefile 模式规则

在 3.3.2 小节中我们编写了一个 Makefile 文件用来编译工程，这个 Makefile 的内容如下：

示例代码 3.4.3.1 Makefile 文件代码

```
1 main: main.o input.o calcu.o
2     gcc -o main main.o input.o calcu.o
3 main.o: main.c
4     gcc -c main.c
5 input.o: input.c
6     gcc -c input.c
7 calcu.o: calcu.c
8     gcc -c calcu.c
9
10 clean:
11     rm *.o
12     rm main
```

上述 Makefile 中第 3~8 行是将对应的 .c 源文件编译为 .o 文件，每一个 C 文件都要写一个对应的规则，如果工程中 C 文件很多的话显然不能这么做。为此，我们可以使用 Makefile 中的模式规则，通过模式规则我们就可以使用一条规则来将所有的 .c 文件编译为对应的 .o 文件。

模式规则中，至少在规则的目标定义中要包涵“%”，否则就是一般规则，目标中的“%”表示对文件名的匹配，“%”表示长度任意的非空字符串，比如“%.c”就是所有的以 .c 结尾的文件，类似与通配符，a.%c 就表示以 a. 开头，以 .c 结束的所有文件。

当“%”出现在目标中的时候，目标中“%”所代表的值决定了依赖中的“%”值，使用方法如下：

```
%o: %.c
    命令
```

因此“示例代码 3.4.3.1”中的 Makefile 可以改为如下形式：

示例代码 3.4.3.2 模式规则使用

```
1 objects = main.o input.o calcu.o
```



```

2 main: $(objects)
3     gcc -o main $(objects)
4
5 %.o : %.c
6     #命令
7
8 clean:
9     rm *.o
10    rm main

```

“示例代码 3.4.3.2”中第 5、6 这两行代码替代了“示例代码 3.4.3.1”中的 3~8 行代码，修改以后的 Makefile 还不能运行，因为第 6 行的命令我们还没写呢，第 6 行的命令我们需要借助另外一种强大的变量—自动化变量。

3.4.4 Makefile 自动化变量

上面讲的模式规则中，目标和依赖都是一系列的文件，每一次对模式规则进行解析的时候都会是不同的目标和依赖文件，而命令只有一行，如何通过一行命令来从不同的依赖文件中生成对应的目标？自动化变量就是完成这个功能的！所谓自动化变量就是这种变量会把模式中所定义的一系列的文件自动的挨个取出，直至所有的符合模式的文件都取完，自动化变量只应该出现在规则的命令中，常用的自动化变量如表 3.4.4.1：

自动化变量	描述
\$@	规则中的目标集合，在模式规则中，如果有多个目标的话，“\$@”表示匹配模式中定义的目标集合。
%	当目标是函数库的时候表示规则中的目标成员名，如果目标不是函数库文件，那么其值为空。
\$<	依赖文件集合中的第一个文件，如果依赖文件是以模式(即“%”)定义的，那么“\$<”就是符合模式的一系列的文件集合。
\$?	所有比目标新的依赖目标集合，以空格分开。
^	所有依赖文件的集合，使用空格分开，如果在依赖文件中有多个重复的文件，“\$^”会去除重复的依赖文件，值保留一份。
+	和“\$^”类似，但是当依赖文件存在重复的话不会去除重复的依赖文件。
*	这个变量表示目标模式中“%”及其之前的部分，如果目标是 test/a.test.c，目标模式为 a.%c，那么“\$*”就是 test/a.test。

表 3.4.4.1 自动化变量

表 3.4.4.1 中的 7 个自动化变量中，常用的三种：\$@、\$<和\$^，我们使用自动化变量来完成“示例代码 3.4.3.2”中的 Makefile，最终的完整代码如下所示：

示例代码 3.4.4.1 自动化变量

```

1 objects = main.o input.o calcul.o
2 main: $(objects)
3     gcc -o main $(objects)
4
5 %.o : %.c
6     gcc -c $<
7

```

```
8 clean:
9     rm *.o
10    rm main
```

上述代码就是修改后的完成的 Makefile，可以看出相比 3.3.2 小节中的要精简了很多，核心就在于第 5、6 这两行，第 5 行使用了模式规则，第 6 行使用了自动化变量。

3.4.5 Makefile 伪目标

Makefile 有一种特殊的目标——伪目标，一般的目标名都是要生成的文件，而伪目标不代表真正的目标名，在执行 make 命令的时候通过指定这个伪目标来执行其所在规则的定义的命令。

使用伪目标主要是为了避免 Makefile 中定义的执行命令的目标和工作目录下的实际文件出现名字冲突，有时候我们需要编写一个规则用来执行一些命令，但是这个规则不是用来创建文件的，比如在前面的“示例代码 3.4.4.1”中有如下代码用来完成清理工程的功能：

```
clean:
    rm *.o
    rm main
```

上述规则中并没有创建文件 clean 的命令，因此工作目录下永远都不会存在文件 clean，当我们输入“make clean”以后，后面的“rm *.o”和“rm main”总是会执行。可是如果我们“手贱”，在工作目录下创建一个名为“clean”的文件，那就不一样了，当执行“make clean”的时候，规则因为没有依赖文件，所以目标被认为是最新的，因此后面的 rm 命令也就不会执行，我们预先设想的清理工程的功能也就无法完成。为了避免这个问题，我们可以将 clean 声明为伪目标，声明方式如下：

```
.PHONY : clean
```

我们使用伪目标来更改“示例代码 3.4.4.1”，修改完成以后如下：

示例代码 3.4.5.1 伪目标

```
1 objects = main.o input.o calcul.o
2 main: $(objects)
3     gcc -o main $(objects)
4
5 .PHONY : clean
6
7 %.o : %.c
8     gcc -c $<
9
10 clean:
11     rm *.o
12     rm main
```

上述代码第 5 行声明 clean 为伪目标，声明 clean 为伪目标以后不管当前目录下是否存在名为“clean”的文件，输入“make clean”的话规则后面的 rm 命令都会执行。

3.4.6 Makefile 条件判断

在 C 语言中我们通过条件判断语句来根据不同的情况来执行不同的分支，Makefile 也支持

条件判断，语法有两种如下：

```
<条件关键字>
    <条件为真时执行的语句>
endif
```

以及：

```
<条件关键字>
    <条件为真时执行的语句>
else
    <条件为假时执行的语句>
endif
```

其中条件关键字有 4 个：ifeq、ifneq、ifdef 和 ifndef，这四个关键字其实分为两对、ifeq 与 ifneq、ifdef 与 ifndef，先来看一下 ifeq 和 ifneq，ifeq 用来判断是否相等，ifneq 就是判断是否不相等，ifeq 用法如下：

```
ifeq (<参数 1>, <参数 2>)
ifeq '<参数 1>', '<参数 2>'
ifeq "<参数 1>", "<参数 2>"
ifeq "<参数 1>", '<参数 2>'
ifeq '<参数 1>', "<参数 2>"
```

上述用法中都是用来比较“参数 1”和“参数 2”是否相同，如果相同则为真，“参数 1”和“参数 2”可以为函数返回值。ifneq 的用法类似，只不过 ifneq 是用来比较“参数 1”和“参数 2”是否不相等，如果不相等的话就为真。

ifdef 和 ifndef 的用法如下：

```
ifdef <变量名>
```

如果“变量名”的值非空，那么表示表达式为真，否则表达式为假。“变量名”同样可以是一个函数的返回值。ifndef 用法类似，但是含义与 ifdef 相反。

3.4.7 Makefile 函数使用

Makefile 支持函数，类似 C 语言一样，Makefile 中的函数是已经定义好的，我们直接使用，不支持我们自定义函数。make 所支持的函数不多，但是绝对够我们使用了，函数的用法如下：

```
$(函数名 参数集合)
```

或者：

```
${函数名 参数集合}
```

可以看出，调用函数和调用普通变量一样，使用符号“\$”来标识。参数集合是函数的多个参数，参数之间以逗号“,”隔开，函数名和参数之间以“空格”分隔开，函数的调用以“\$”开头。接下来我们介绍几个常用的函数，其它的函数大家可以参考《跟我一起写 Makefile》这份文档。

1、函数 subst

函数 subst 用来完成字符串替换，调用形式如下：

```
$(subst <from>,<to>,<text>)
```

此函数的功能是将字符串<text>中的<from>内容替换为<to>，函数返回被替换以后的字符串，比如如下示例：

```
$(subst zzk,ZZK,my name is zzk)
```

把字符串“my name is zzk”中的“zzk”替换为“ZZK”，替换完成以后的字符串为“my name is ZZK”。

2、函数 patsubst

函数 patsubst 用来完成模式字符串替换，使用方法如下：

```
$(patsubst <pattern>,<replacement>,<text>)
```

此函数查找字符串<text>中的单词是否符合模式<pattern>，如果匹配就用<replacement>来替换掉，<pattern>可以使用通配符“%”，表示任意长度的字符串，函数返回值就是替换后的字符串。如果<replacement>中也包涵“%”，那么<replacement>中的“%”将是<pattern>中的那个“%”所代表的字符串，比如：

```
$(patsubst %.c,%.o,a.c b.c c.c)
```

将字符串“a.c b.c c.c”中的所有符合“%.c”的字符串，替换为“%.o”，替换完成以后的字符串为“a.o b.o c.o”。

3、函数 dir

函数 dir 用来获取目录，使用方法如下：

```
$(dir <names...>)
```

此函数用来从文件名序列<names>中提取出目录部分，返回值是文件名序列<names>的目录部分，比如：

```
$(dir </src/a.c>)
```

提取文件“/src/a.c”的目录部分，也就是“/src”。

4、函数 notdir

函数 notdir 看名字就是知道去除文件中的目录部分，也就是提取文件名，用法如下：

```
$(notdir <names...>)
```

此函数用与从文件名序列<names>中提取出文件名非目录部分，比如：

```
$(notdir </src/a.c>)
```

提取文件“/src/a.c”中的非目录部分，也就是文件名“a.c”。

5、函数 foreach

foreach 函数用来完成循环，用法如下：

```
$(foreach <var>,<list>,<text>)
```

此函数的意思就是把参数<list>中的单词逐一取出来放到参数<var>中，然后再执行<text>所包含的表达式。每次<text>都会返回一个字符串，循环的过程中，<text>中所包含的每个字符串会以空格隔开，最后当整个循环结束时，<text>所返回的每个字符串所组成的整个字符串将会是函数 foreach 函数的返回值。

6、函数 wildcard

通配符“%”只能用在规则中，只有在规则中它才会展开，如果在变量定义和函数使用时，通配符不会自动展开，这个时候就要用到函数 wildcard，使用方法如下：

```
$(wildcard PATTERN...)
```

比如：

```
$(wildcard *.c)
```

上面的代码是用来获取当前目录下所有的.c 文件，类似“%”。

关于 Makefile 的相关内容就讲解到这里，本节只是对 Makefile 做了最基本的讲解，确保大家能够完成后续的学习，Makefile 还有大量的知识没有提到，有兴趣的可以自行参考《跟我一

起写 Makefile》这份文档来深入学习 Makefile。